



SAPIENZA
UNIVERSITÀ DI ROMA

Progettazione di Algoritmi

UNIVERSITÀ DEGLI STUDI DI ROMA "LA
SAPIENZA"

FACOLTÀ DI INGEGNERIA DELL'INFORMAZIONE,
INFORMATICA E STATISTICA

CORSO DI LAUREA TRIENNALE IN INFORMATICA

Autore
LORENZO SABATINO

Basato sulle lezioni di
PROF. ANGELO MONTI
ESERC. IVANO SALVO

24 luglio 2024



Prefazione

Salve a tutti!

Mi chiamo Lorenzo Sabatino e sono uno studente del corso di Informatica presso l'Università La Sapienza di Roma. Durante il mio percorso accademico, ho affrontato numerosi esami trascrivendo gli appunti presi a lezione e trasformandoli in veri e propri libri di studio.

Ho voluto condividere questo materiale con voi perché credo che possa essere un valido aiuto per tutti gli studenti di Informatica, sia quelli alle prime armi che chi è già avanti nel percorso di studi. Ho scritto e formattato questi testi con cura, utilizzando il linguaggio di markup $\text{\LaTeX}$ per garantire chiarezza e leggibilità. Ho anche arricchito il contenuto con diagrammi e schemi e ho integrato spunti tratti da lezioni online e da libri di testo.

Tuttavia, è importante sottolineare che questi libri non sostituiscono le lezioni frontali e non sono privi di errori. La mia intenzione è stata quella di fornire un supporto aggiuntivo allo studio, ma è indispensabile partecipare alle lezioni e approfondire i concetti con la guida dei docenti. Inoltre, tenete presente che, data la natura in continua evoluzione dell'informatica, alcune informazioni potrebbero diventare obsolete col passare del tempo.

Nonostante ciò, ho deciso di condividere questi libri perché credo che possano essere d'aiuto a molti studenti e spero che li troviate utili per il vostro percorso accademico.

Se desiderate contattarmi per qualsiasi domanda, commento o correzione, potete scrivere all'indirizzo email scrittinformatica@gmail.com. Sarò lieto di rispondere alle vostre richieste.

Grazie per aver scelto di leggere questi libri e in bocca al lupo per i vostri studi!

Indice

Prefazione	1
1 Introduzione al corso	7
1.1 Algoritmi efficienti	7
2 Grafi	9
2.1 Rappresentare un grafo	10
2.1.1 Ricerca di un pozzo	11
2.2 Visite in profondità (DFS)	11
2.2.1 Vettore dei padri	13
2.2.2 Colorazione dei grafi	14
2.2.3 Componenti connesse	15
2.2.4 Ordinamento topologico	18
2.2.5 Ricerca dei cicli	20
2.2.6 Ricerca dei ponti	22
2.3 Visita in ampiezza (BFS)	24
2.4 Grafi pesati	26
2.4.1 Algoritmo di Dijkstra	27
2.4.2 Minimo albero di copertura	29
2.5 Union-Find	31
3 Greedy	35
3.1 Algoritmi greedy	35
3.1.1 Selezione di attività	35
3.1.2 Assegnazione di attività	36
3.1.3 File su disco	37
3.1.4 Elementi in una lista	38
3.2 Algoritmi di approssimazione	39
3.2.1 Copertura tramite nodi minima	39
3.2.2 Il problema del massimo taglio	41
3.2.3 Il problema dell'impacchettamento (BP)	42
4 Divide et impera	45
4.1 Algoritmi divide et impera	45
4.1.1 Problema della selezione	45
4.1.2 Potenza di un numero	49
4.1.3 Valore massimo nelle sottoliste	49
4.1.4 Sottostringhe binarie	52
5 Programmazione dinamica	55
5.1 Algoritmi di programmazione dinamica	55
5.1.1 Sequenza dei numeri di Fibonacci	55
5.1.2 Sottoinsieme di file	57
5.1.3 Sequenza di zeri in stringhe binarie	61
5.1.4 Persone in un albergo	61
5.1.5 Somma massima per una sottosequenza	62
5.1.6 Tassellamenti di una superficie	62

5.1.7	Massimo vettore	63
5.1.8	Sottosequenza crescente più lunga	64
5.1.9	Quadrati di un numero	64
5.1.10	Sequenze decimali non decrescenti	65
5.1.11	Percorso nella matrice	66
5.1.12	Sottomatrici quadrate di dimensione massima	67
5.1.13	Problema dello zaino	67
5.1.14	Problema del resto	68
5.1.15	Guadagno massimo con le transazioni	69
5.1.16	Guadagno massimo con k transazioni	69
5.1.17	Problema della massima sottosequenza comune	70
5.1.18	Ricerca di cammini su grafi con pesi negativi	71
5.1.19	Gioco delle carte agli estremi	73
6	Backtracking	75
6.1	Algoritmi di backtracking	75
6.1.1	Stringhe binarie con almeno k uni	75
6.1.2	Stringhe binarie con esattamente k uni	76
6.1.3	Stringhe sull'alfabeto	76
6.1.4	Sequenze valide	77
6.1.5	Stringhe di $*$ e di caratteri crescenti	78
6.1.6	Matrici binarie $n \times n$	79
6.1.7	Matrici binarie $n \times n$ senza uno adiacenti	79
6.1.8	Matrici binarie $n \times n$ con righe e colonne non decrescenti	80
6.1.9	Permutazioni da 0 a $n - 1$	81
6.1.10	Permutazioni da 0 a $n - 1$ con posizioni e numeri pari	82
6.1.11	Problema di ottimizzazione dello Zaino	82
6.1.12	Problema della 3-colorazione	85
6.1.13	Il problema del ciclo Hamiltoniano	86
6.1.14	Il problema delle n regine	87

-

Capitolo 1

Introduzione al corso

In questo corso descriveremo alcuni algoritmi "classici" per risolvere problemi di base e vedremo particolari tecniche che si sono dimostrate utili per la soluzione di una vasta gamma di problemi: tecnica greedy, divide et impera, programmazione dinamica, backtracking.

1.1 Algoritmi efficienti

Un algoritmo si dice efficiente se la sua complessità è di ordine polinomiale nella dimensione n dell'input. Ovvero di complessità $O(n^c)$ per una qualche costante c .

Perché per un algoritmo efficiente si intende un algoritmo a complessità $O(n^c)$ per un generico c e non più semplicemente $O(n^2)$ o $O(n^3)$? Secondo la tesi di Church-Turing¹ la complessità di un algoritmo è indipendente dal sistema di calcolo utilizzato quindi è inutile dare un valore alla variabile c , ma l'importante è che la funzione sia polinomiale. Questa tesi è tutt'altro che accettata universalmente. Molti ad esempio ritengono che con l'avvento dei calcolatori quantistici molti problemi che attualmente risultano intrattabili diventeranno trattabili. Ma allo stato attuale per questi problemi non si può escludere l'esistenza di un algoritmo classico efficiente.

Di conseguenza un algoritmo è inefficiente se la sua complessità è di ordine super-polinomiale, ovvero se è compresa tra $O(n^c)$ e $2^{\Theta(n)}$, ad esempio $O(n^{\log n})$ dato che $n^{\log n} = 2^{\log^2 n} = 2^{\Theta(\log^2 n)}$ oppure i problemi di fattorizzazione con $2^{\Theta(\sqrt{n})}$.

Esercizio 1.1.1 – Test di primalità

Ho un numero e voglio verificare che sia primo o meno.

Con un algoritmo banale, dato n cerco tutti i divisori tra 2 e $n - 1$, posso pensare erroneamente che abbia complessità $O(n)$. In realtà in $O(n^c)$, n rappresenta il numero di valori in input, ma in questo problema la dimensione dell'input non è n ma $\log n$. Quindi questo algoritmo ha complessità $O(2^{\log n})$.

Nella ricerca dell'eventuale divisore posso fermarmi alla radice in modo da velocizzare l'algoritmo, ma non ne cambia la complessità asintotica che resta esponenziale:

$$O(\sqrt{n}) = O(2^{\log \sqrt{n}}) = O(2^{\frac{1}{2} \log n}) = 2^{\Theta(\log n)}$$

Quindi in entrambe i casi il costo dell'algoritmo è esponenziale $O(2^{\Theta(n)})$ quindi inefficiente.

Non bisogna confondere la complessità dell'algoritmo con la complessità del problema. Un algoritmo di complessità $O(g(n))$ per un problema produce una limitazione superiore alla complessità del problema. Se si dimostra che qualunque algoritmo per quel problema ha complessità $\Omega(f(n))$, si è stabilita una limitazione inferiore alla complessità del problema. Se $f(n) = g(n)$ allora l'algoritmo è detto ottimo, perché la sua complessità in ordine di grandezza risulta la migliore possibile.

¹"I modelli di calcolo realistici sono tra loro polinomialmente correlati. Il concetto di trattabilità è dunque indipendente dalla macchina."

Esercizio 1.1.2 – Maggiore assoluto

Data una lista di n interi voglio determinare se la lista ha un elemento di maggioranza assoluta, ovvero un elemento la cui frequenza nella lista è maggiore della metà (almeno $\frac{n}{2} + 1$).

Intuitivamente questo problema non potrà avere complessità minore di $\Omega(n)$, dato che è necessario almeno uno scorrimento di tutti gli elementi nel vettore.

Si può creare un algoritmo ottimo che usa una lista inizialmente vuota (B) e che scorrendo tra gli elementi della lista A aggiunge l'elemento a B se esso è uguale all'ultimo elemento di B, altrimenti viene cancellato. Dopo questo procedimento, la lista B, se contiene elementi allora sono tutti uguali e, se c'è un elemento a maggioranza, allora è nella lista. Quindi non resta da verificare se la frequenza degli elementi di B è almeno la metà. Il seguente algoritmo risolve il problema in $O(n)$:

```
def es(A):
    B=[]
    n=len(A)
    for i in range(n):
        if B==[] or B[-1]==A[i]:
            B.append(A[i])
        else:
            B.pop()
    if B==[]:
        return None
    x=A.count(B[0])
    if x>=n//2+1:
        return B[0]
    else:
        return None
```

Capitolo 2

Grafi

Un grafo è una struttura formata da una coppia $G = (V, E)$ dove $V = n$ rappresenta il numero di nodi e $E = m$ il numero di archi.

I grafi possono essere *diretti*, dove gli archi hanno un verso e $0 \leq m \leq n(n-1) = O(n^2)$, oppure *non diretti*, dove $0 \leq m \leq \frac{n(n-1)}{2} = O(n^2)$.

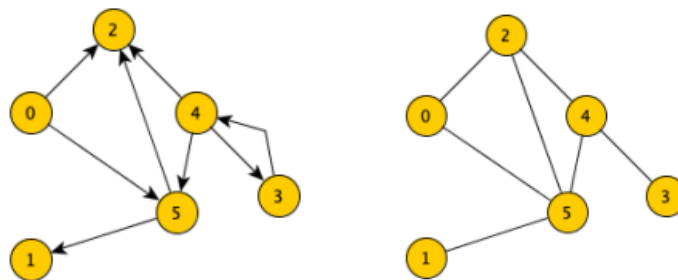


Figura 2.1: Grafo diretto e grafo non diretto

Un grafo si dice *sparso* se il numero di archi è dello stesso ordine del numero dei nodi $m = O(n)$, oppure si dice *denso* se il numero di archi è quadratico rispetto al numero di nodi $m = \Omega(n^2)$. Ad esempio un grafo vuoto (senza archi) è sparso, mentre un grafo completo (con il massimo numero di archi) è denso. Inoltre un grafo non sparso non è necessariamente denso, ad esempio si possono avere $\Theta(n \log n)$ archi.

Un albero è un grafo sparso, connesso (partendo da qualunque nodo si possono raggiungere tutti gli altri) e senza cicli (aciclico).

Lemma 2.0.1 – Archi in un albero

Un albero ha sempre $m = n - 1$ archi la cui prova è fornita per induzione sul numero n di nodi.

Dimostrazione.

- Caso base: in un albero con $n = 1$ nodi ci saranno $m = 0$ archi;
- Ipotesi induttiva: assumiamo vero che in un albero con n nodi ci sono $m = n - 1$ archi;
- Passo induttivo: in un albero con $n + 1$ nodi, tolgo una foglia e il rispettivo arco, mi rimane un albero aciclico con n nodi che per ipotesi induttiva ha $m = n - 1$ archi. Aggiungendo la foglia e l'arco eliminati precedentemente ottengo un albero con $n + 1$ nodi e $m = n$ archi.

Un grafo *planare* è un grafo sparso che può essere rappresentato su una superficie piana in modo che gli archi non si sovrappongano. Secondo il teorema di Eulero un grafo planare di $n \geq 3$ nodi non

ha al più di $3n - 6$ archi, e se $n > 3$ e non vi sono cicli di lunghezza 3, allora si hanno massimo $2n - 4$ archi.

2.1 Rappresentare un grafo

I metodi che studieremo per rappresentare un grafo sono due: con matrice di adiacenza e con liste di adiacenza.

- Il modo più semplice per rappresentare un grafo è con una matrice di adiacenza $n \times n$ dove la cella $M[i][j]$ è 1 solo se c'è un arco diretto da i a j . Nel caso di grafi non diretti questa matrice è simmetrica perché la cella $M[i][j]$ contiene lo stesso valore della cella $M[j][i]$. Lo spazio richiesto per questa struttura è $O(n^2)$.
- Un altro modo di rappresentare un grafo è tramite le liste di adiacenza che utilizzano una lista di liste G . La lista ha tanti elementi quanti sono i nodi del grafo e, $G[x]$ è a sua volta una lista contenente i nodi adiacenti al nodo x , vale a dire quelli da archi che partono da x .

Con questo secondo metodo è più efficiente memorizzare grafi sparsi e, in un grafo diretto, ogni arco occuperà soltanto una lista (nella lista da dove l'arco parte) $O(n + m)$, mentre nei grafi non diretti, ogni arco comparirà due volte nelle liste (in entrambe le liste dei nodi che sono collegati dall'arco) $O(n + 2m)$.

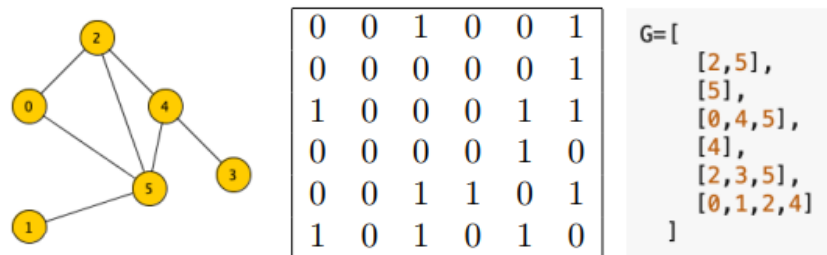


Figura 2.2: Rappresentazione di un grafo non diretto

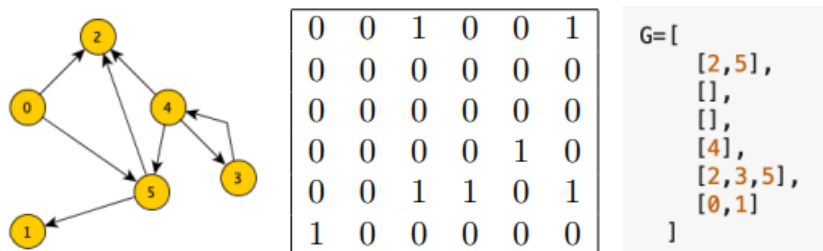


Figura 2.3: Rappresentazione di un grafo diretto

Lemma 2.1.1 – Nodi con lo stesso grado

In un grafo con $n \geq 2$ nodi ci sono almeno due nodi che hanno lo stesso grado (stesso numero di archi che incidono sul nodo).

Dimostrazione. Assumiamo il grafo connesso. Se non lo fosse, prendiamo una componente connessa qualsiasi con almeno due nodi (se non c'è, ci sono almeno due nodi con grado 0). Se n è il numero dei nodi della componente connessa, i gradi sono nell'intervallo $[1, n - 1]$. Applicando il principio dei cassetti (o legge del buco della piccionaia)^a si ha necessariamente che almeno due nodi devono avere lo stesso grado.

^aSe $n + k$ oggetti sono messi in n cassetti, allora almeno un cassetto deve contenere più di un oggetto.

2.1.1 Ricerca di un pozzo

In un grafo diretto un *pozzo* è un nodo senza archi uscenti, mentre, è un *pozzo universale* un pozzo verso cui gli altri nodi hanno un arco. In un grafo diretto possono essere presenti fino ad n pozzi e il pozzo universale se c'è, è unico.

Algoritmo 2.1.1 – Verifica pozzo universale in $O(n^2)$

Un algoritmo di tempo $O(n^2)$ per verificare se un grafo diretto ha un pozzo universale, controlla in ogni nodo se la riga della matrice contiene tutti 0, ovvero se il nodo non ha archi uscenti, e se la colonna della matrice è formata da tutti 1 (tranne per l'elemento $M[u][u]$), ovvero se il nodo ha solo archi entranti.

```
def pozzoU(u, M):
    if sum(M[u]) != 0:
        return false
    for i in range(len(M)):
        if i != u and M[i][u] != 1:
            return false
    return true

def PU(M):
    for i in range(len(M)):
        if pozzoU(i, M):
            return true
    return false
```

0	0	1	0	0	0	0	0
0	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0
0	0	1	0	0	0	0	0
0	0	1	0	1	0	0	0
0	0	1	0	0	0	0	0
0	0	1	0	0	0	0	0

Algoritmo 2.1.2 – Verifica pozzo universale in $\Theta(n)$

Un algoritmo di tempo $\Theta(n)$ per verificare se un grafo diretto ha un pozzo universale, utilizza una lista A con tutti i nodi e li confronta tra di loro. Se il nodo a ha un arco verso il nodo b , a viene eliminato dalla lista, altrimenti viene eliminato b . Al termine di questo procedimento resta un unico nodo da controllare se è un pozzo universale.

```
def PU1(M):
    A = [x for x in range(len(M))]
    while len(A) > 1:
        a = A.pop()
        b = A.pop()
        if M[a][b] == 1:
            A.append(b)
        else:
            A.append(a)
        a = A.pop()
    return pozzoU(a, M)
```

Ad ogni passo l'algoritmo elimina un nodo dalla lista $\Theta(n - 1)$ e per verificare se un nodo è un pozzo universale impiega $O(n)$. Quindi la complessità totale è $\Theta(n)$.

2.2 Visite in profondità (DFS)

Algoritmo 2.2.1 – Visita in profondità (DFS) tramite matrice e liste di adiacenza

Dato un grafo G ed un suo nodo u vogliamo sapere quali nodi del grafo sono raggiungibili a partire dal nodo u .

Per risolvere il problema visitiamo i nodi del grafo tramite la ricorsione, con l'aggiunta di una lista di nodi già visitati V per non entrare in un loop del grafo. La corretta soluzione con la

matrice di adiacenza è:

```
def DFS(u, M):
    def DFSr(u, M, V):
        V[u]=1
        for i in range(len(M)):
            if M[u][i]==1 and V[i]==0:
                DFSr(i, M, V)

    V=[0 for i in range(len(M))]
    DFSr(u, M, V)
    return V
```

La complessità di questo algoritmo è $O(n)\Theta(n) = O(n^2)$.

Si può risolvere lo stesso problema anche con le liste di adiacenza:

```
def DFS(u, G):
    def DFSr(u, G, V):
        V[u]=1
        for x in G[u]:
            if V[x]==0:
                DFSr(x, G, V)

    V=[0 for i in G]
    DFSr(u, G, V)
    return V
```

La complessità di questo algoritmo è $O(n + m)$.

Teorema 2.2.1 – Correttezza dell'algoritmo di visita in profondità (DFS)

Dato un grafo G ed un suo nodo u , l'algoritmo di visita in profondità (DFS) ci permette di conoscere quali nodi sono raggiungibili a partire dal nodo u , e questi nodi si troveranno nel vettore V al termine dell'algoritmo con valore 1.

Dimostrazione. Supponiamo per assurdo che x è un nodo raggiungibile ma alla fine dell'algoritmo abbiamo $V[x]=0$. Tornando all'indietro sul percorso dei nodi, essendo $V[x]=0$ non è possibile arrivare al nodo di partenza u . Viceversa il nodo x non è raggiungibile partendo dal nodo u (contraddizione).

Con una visita DFS gli archi del grafo si bipartiscono in quelli che nel corso della visita sono stati attraversati (perché permettevano di raggiungere nuovi nodi) e gli altri. I nodi visitati e gli archi effettivamente attraversati formano un albero detto *albero DFS*.

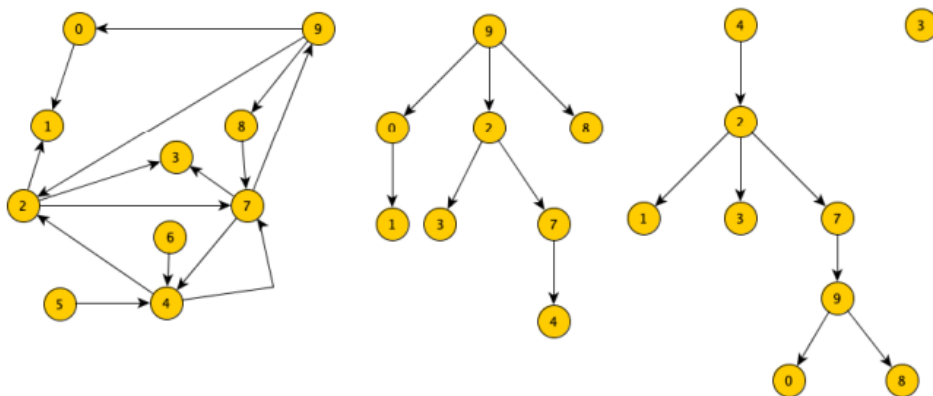


Figura 2.4: Alberi DFS

2.2.1 Vettore dei padri

Un albero può essere memorizzato tramite il *vettore dei padri*. Il vettore dei padri P di un albero T di n nodi ha almeno n componenti: se i è un nodo dell'albero $P[i]$ contiene il padre del nodo i (il padre della radice per convenzione è la radice stessa); se i non è un nodo dell'albero $P[i]$ per convenzione contiene -1.

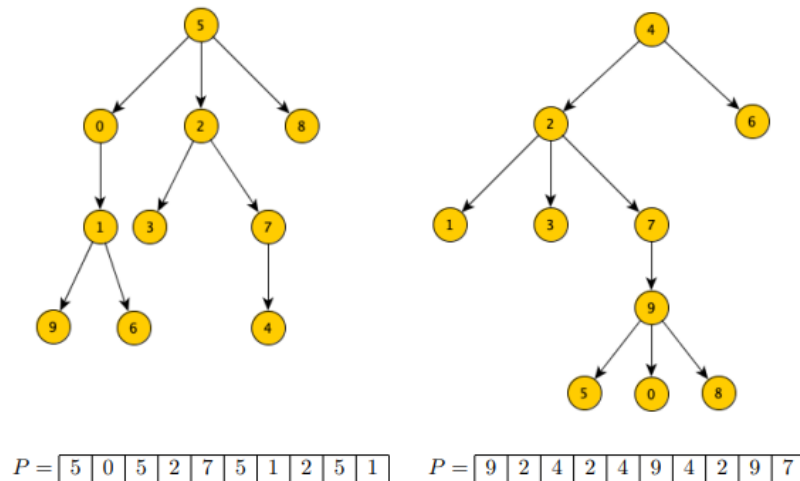


Figura 2.5: Vettore dei padri

Algoritmo 2.2.2 – Visita in profondità (DFS) per il vettore dei padri

Modificando lievemente la procedura DFS è possibile fare in modo che restituisca il vettore dei padri P anziché il vettore dei visitati.

```
def Padri(u, G):
    def DFSr(u, G, P):
        for x in G[u]:
            if P[x] == -1:
                P[x] = u
                DFSr(x, G, P)

    P = [-1] * len(G)
    P[u] = u
    DFSr(u, G, P)
    return P
```

Al termine $P[v]$ contiene -1 se v non è stato visitato altrimenti contiene il padre di v nell'albero DFS.

Spesso non ci basta sapere se un nodo y è raggiungibile a partire dal nodo x del grafo: nel caso la risposta sia positiva vogliamo anche un cammino che ci permetta di andare da x a y .

Algoritmo 2.2.3 – Cammino con il vettore dei padri

Il vettore dei padri dell'albero DFS radicato in x permette di ricavare facilmente il cammino per y : basta controllare che il nodo y sia nell'albero e poi da y risalire alla radice ed effettuare il "reverse" dei nodi incontrati.

Questo problema può essere risolto in tempo $O(n)$ con un approccio iterativo o ricorsivo:

```

def Cammino(u, P):
    if P[u]==-1:
        return []
    path = []
    while P[u]!=u:
        path.append(u)
        u=P[u]
    path.append(u)
    path.reverse()
    return path

def Cammino(u, P):
    if P[u]==-1:
        return []
    if P[u]==u:
        return [u]
    return Cammino(P[u], P)+[u]

```

Se esistono più cammini che dal nodo x portano al nodo y la procedura appena vista non garantisce di restituire il cammino minimo.

2.2.2 Colorazione dei grafi

Dato un grafo connesso G ed un intero k vogliamo sapere se è possibile colorare i nodi del grafo in modo che nodi adiacenti abbiano sempre colori distinti.

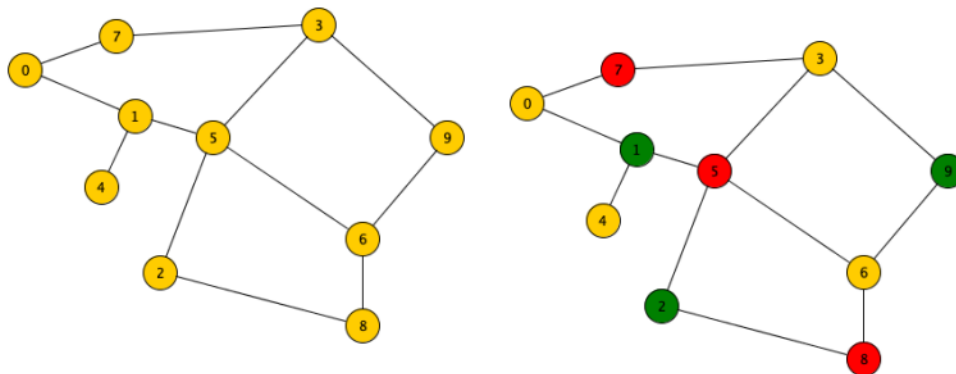


Figura 2.6: Esempio di grafo 3-colorabile

Teorema 2.2.2 – Teorema dei quattro colori

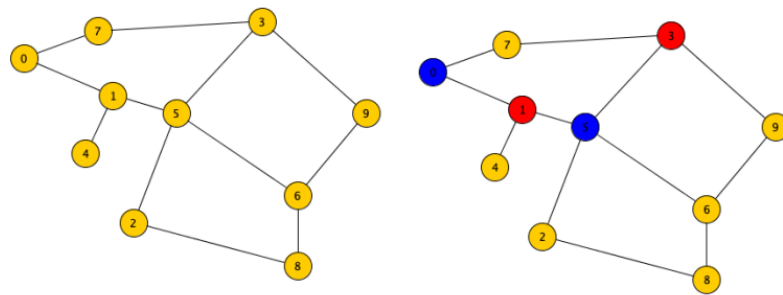
Secondo il teorema dei 4 colori: un grafo planare richiede al più 4 colori per essere colorato.

In generale un grafo può richiedere anche $\Theta(n)$ colori. Non è noto nessun algoritmo polinomiale che, dato un grafo planare G , determini se G è 3-colorabile.

Lemma 2.2.1 – Grafo 2-colorabile

Un grafo è 2-colorabile se e solo se non contiene cicli di lunghezza dispari.

Dimostrazione. Un ciclo di lunghezza dispari rende impossibile la colorazione del grafo con due colori.



L'algoritmo di bi-colorazione che segue prova che un grafo senza cicli dispari può sempre essere 2-colorato.

Algoritmo 2.2.4 – Algoritmo di bi-colorazione

L'algoritmo opera nel seguente modo:

1. Colora il nodo 0 col colore 0.
2. Effettua una visita in profondità del grafo a partire dal nodo 0. Nel corso della visita, a ciascun nodo x che incontri assegna uno dei due colori 0 e 1. Scegli il colore da assegnare in modo che sia diverso dal colore assegnato al nodo padre che ti ha portato a visitare x .

```
def colora(G):
    def DFSr(x, G, colore, c):
        colore[x] = c
        for y in G[x]:
            if colore[y] == -1:
                DFSr(y, G, colore, 1-c)

    colore = [-1 for v in G]
    DFSr(0, G, colore, 0)
    return colore
```

La complessità dell'algoritmo è quella di una semplice visita del grafo connesso da colorare: $O(n + m) = O(m)$ (dipende dal fatto che in un grafo connesso $m \geq n - 1$).

Teorema 2.2.3 – Correttezza dell'algoritmo di bi-colorazione

Siano x e y due nodi adiacenti in G , consideriamo i due possibili casi e facciamo vedere che in entrambi i casi i due nodi al termine avranno colori opposti.

Dimostrazione.

1. L'arco (x, y) viene attraversato durante la visita. In questo caso banalmente i due nodi hanno colori distinti.
2. L'arco (x, y) non viene attraversato durante la visita. Sia x il nodo visitato prima. Esiste un cammino in G che da x porta a y (quello seguito dalla visita), questo cammino si chiude a formare un ciclo con l'arco (y, x) . Il ciclo è di lunghezza pari per ipotesi, quindi il cammino è di lunghezza dispari. Poiché sul cammino i colori si alternano il primo nodo (x) e l'ultimo nodo (y) del cammino avranno colori diversi.

2.2.3 Componenti connesse

Una *componente connessa* di un grafo non diretto è un sotto-grafo composto da un insieme massimale di nodi connessi da cammini. Un grafo si dice connesso se ha una sola componente connessa.

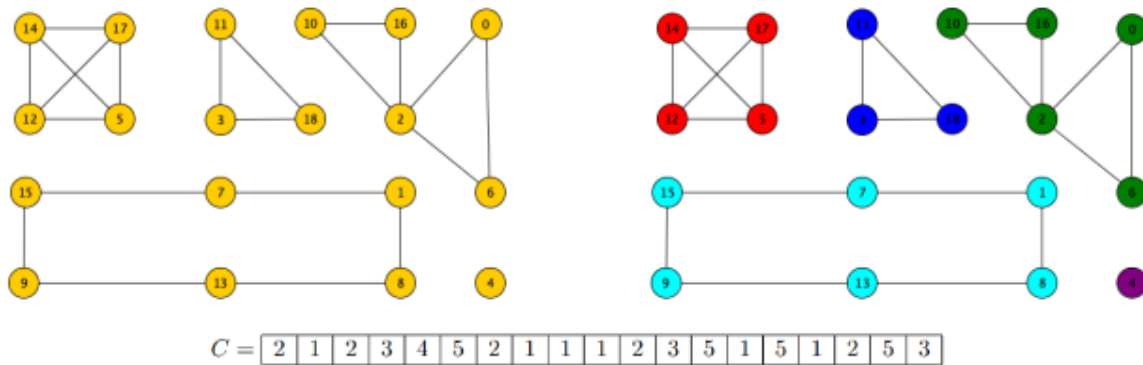


Figura 2.7: Esempio di grafo con componenti connesse

Vogliamo calcolare il vettore C delle componenti connesse di un grafo G . Vale a dire il vettore C che ha tanti elementi quanti sono i nodi del grafo e $C[u] = C[v]$ se e solo se u e v sono nella stessa componente connessa.

Algoritmo 2.2.5 – Algoritmo di ricerca delle componenti connesse

Il seguente algoritmo calcola il vettore C delle componenti connesse di un grafo G .

```

def Componenti(G):
    def DFSr(x, G, C, c):
        C[x]=c
        for y in G[x]:
            if C[y]==0:
                DFSr(y, G, C, c)

    C=[0]*len(G)
    c=0
    for x in range(len(G)):
        if C[x]==0:
            c+=1
            DFSr(x, G, C, c)
    return C

```

La complessità della procedura è $O(n + m)$.

Componenti fortemente connesse

Una *componente fortemente connessa* di un grafo diretto è un sotto-grafo composto da un insieme massimale di nodi connessi da cammini, ovvero se da un nodo posso arrivare a tutti gli altri e da tutti gli altri posso arrivare a quel nodo. Un grafo diretto si dice *fortemente connesso* se ha una sola componente.

Vogliamo calcolare il vettore C delle componenti fortemente connesse di un grafo G . Vale a dire il vettore C che ha tanti elementi quanti sono i nodi del grafo e $C[u] = C[v]$ se e solo se u e v sono nella stessa componente fortemente connessa.

L'algoritmo utilizzato per le componenti connesse non funziona nel caso di componenti fortemente connesse. Il problema risiede nel fatto che se c'è un cammino che da x mi porta a y questo non implica che x e y sono nella stessa componente fortemente connessa, perché questo sia vero è necessario che ci sia anche un cammino che da y mi porti a x .

Dato un grafo diretto G il *grafo trasposto* (o complementare) di G ($G^C = (V, E^C)$) ha gli stessi nodi di G ma archi con direzione opposta.

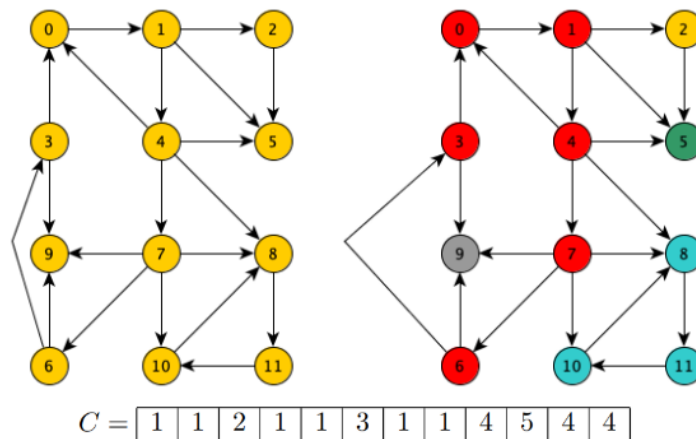


Figura 2.8: Esempio di grafo con componenti fortemente connesse



Figura 2.9: Esempio di grafo trasposto

Lemma 2.2.2 – Grafo complementare connesso

Almeno uno tra G e G^C è connesso.

Dimostrazione. Consideriamo il caso in cui G non sia connesso (altrimenti non c'è nulla da dimostrare perché G è connesso). Siano u, v due nodi qualsiasi di G . Se u e v appartengono a due diverse componenti connesse di G , allora c'è l'arco (u, v) in G^C . Se sono nella stessa componente connessa diciamo G_1 , prendiamo un nodo qualsiasi z in un'altra componente connessa G_2 . Gli archi (u, z) e (z, v) sono allora in G^C e quindi c'è il cammino $u \rightarrow z \rightarrow v$ in G^C .

Algoritmo 2.2.6 – Algoritmo di ricerca delle componenti fortemente connesse

Dato il grafo diretto G ed un suo nodo u , l'algoritmo restituisce i nodi della componente fortemente connessa che contiene u in tre passi:

1. calcola l'insieme A dei nodi di G raggiungibili da u ($O(n + m)$), effettuando una visita sul nodo e ottenendo il vettore dei visitati;
2. calcola l'insieme B dei nodi di G che portano a u . Per eseguire efficientemente questo passo si può ricorrere al grafo trasposto di G ($O(n + m)$);

```
def Trasposto(G):
    GT=[]
    for v in G:
        for u in G[v]:
            GT[u].append(v)
    return GT
```

3. restituisci l'intersezione dei due insiemi A e B ($O(n)$).

```
def Intersezione(V1, V2):
    V=[0]*len(V1)
    for i in range(len(V1)):
        if V1[i]==V2[i]==1:
```

```

        V[i]=1
    return V

```

L'algoritmo risultante è il seguente:

```

def ComponenteFC(x, G):
    visitati1=DFS(x, G)
    G1=Trasposto(G)
    visitati2=DFS(x, G1)
    return Intersezione(visitati1, visitati2)

```

La sua complessità è data dalla somma dei tre passi:

$$O(n + m) + O(n + m) + O(n + m) + O(n) = O(n + m)$$

Utilizzando l'algoritmo `ComponenteFC(x, G)` come subroutine posso ottenere un algoritmo per calcolare il vettore CF delle componenti fortemente connesse:

```

def compFC(G):
    FC=[0]*len(G)
    c=0
    for i in range(len(G)):
        if FC[i]==0:
            E=ComponenteFC(i, G)
            c+=1
            for x in E:
                FC[x]=c
    return FC

```

La complessità dell'algoritmo è $\Theta(n) * O(n + m) = O(n^2 + nm) = O(n^3)$.

Per il calcolo del vettore CF delle componenti fortemente connesse sono noti diversi algoritmi non banali che lavorano in tempo $O(n + m)$ (ad esempio l'algoritmo di Tarjan o quello di Kosaraju).

2.2.4 Ordinamento topologico

Spesso un grafo diretto cattura relazioni di propedeuticità (un arco da a a b indica che a è propedeutico a b). Potrò rispettare tutte le propedeuticità se riesco ad ordinare i nodi del grafo in modo che gli archi (le frecce) vadano tutti da sinistra verso destra. Questa ordinamento è detto *ordinamento topologico*.

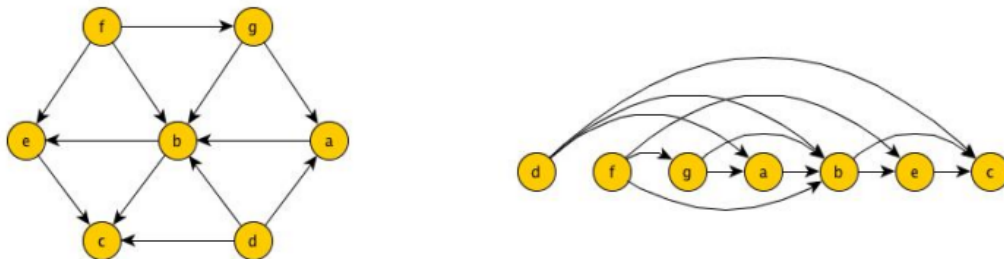


Figura 2.10: Esempio di ordinamento topologico

Un grafo diretto può avere da 0 ad $n!$ ordinamenti topologici. Perché G possa avere un ordinamento topologico è necessario che sia un DAG (ovvero un grafo diretto aciclico). La presenza di un ciclo nel grafo implica che nessuno dei nodi del ciclo possa comparire nell'ordine giusto. Un DAG ha sempre un nodo sorgente (un nodo in cui non entrano archi).

Algoritmo 2.2.7 – Algoritmo di ordinamento topologico

Posso costruire l'ordinamento topologico dei nodi del DAG in questo modo:

1. Inizio la sequenza dei nodi con una sorgente;
2. Cancello dal DAG quel nodo sorgente e le frecce che partono da lui. Ottengo un nuovo DAG;
3. Itero questo ragionamento finché non ho sistemato in ordine lineare tutti i nodi.

```
def SortTop(G):
    gradoEnt=[0]*len(G)
    for i in range(len(G)):
        for j in G[i]:
            gradoEnt[j]+=1
    sorgenti=[]
    for i in range(len(G)):
        if gradoEnt[i]==0:
            sorgenti.append(i)
    ST=[]
    while len(sorgenti)>0:
        x=sorgenti.pop()
        ST.append(x)
        for y in G[x]:
            gradoEnt[y]-=1
            if gradoEnt[y]==0:
                sorgenti.append(y)
    if len(ST)==len(G):
        return ST
    return []
```

Inizializzare il vettore dei gradi entranti costa $O(n + m)$, inizializzare l'insieme delle sorgenti costa $O(n)$, il while viene iterato $O(n)$ volte e il costo totale del for al termine del while è $O(m)$. Il costo dell'algoritmo è $O(n + m)$.

Algoritmo 2.2.8 – Algoritmo alternativo di ordinamento topologico con DFS

Posso costruire un algoritmo alternativo basato sulla visita DFS per l'ordinamento topologico dei nodi del DAG in questo modo:

1. Effettua una visita del DAG a partire dal nodo 0;
2. Man mano che termina la visita dei vari nodi, inseriscili in una lista;
3. Restituisci come ordinamento dei nodi il reverse della lista.

```
def SortT(G):
    def DFSr(x, G, visitati, lista):
        visitati[x]=1
        for y in G[x]:
            if visitati[y]==0:
                DFSr(y, G, visitati, lista)
        lista.append(x)

    visitati=[0 for _ in G]
    lista=[]
    for x in range(len(G)):
        if visitati[x]==0:
            DFSr(x, G, visitati, lista)
```

```
lista.reverse()
return lista
```

La complessità è $O(n + m) + O(n) = O(n + m)$.

Teorema 2.2.4 – Correttezza dell’algoritmo alternativo di ordinamento topologico con DFS

Siano x e y due nodi in G , con arco che va da x a y . In entrambi i casi nella lista, prima di effettuare il reverse, y precede x .

Dimostrazione. Se l’arco (x, y) viene attraversato durante la visita. In questo caso banalmente la visita di y finisce prima della visita di x e y finisce nella lista prima che ci finisca x .

Se l’arco (x, y) non viene attraversato durante la visita. Durante la visita di x il nodo y è già stato visitato e la sua visita è anche già terminata (infatti da y non c’è un cammino che porta a x , in caso contrario nel DAG ci sarebbe un ciclo), anche in questo caso y finisce nella lista prima che ci finisca x .

2.2.5 Ricerca dei cicli

Dato un grafo G (diretto o non diretto) ed un suo nodo u vogliamo sapere se da u è possibile raggiungere un ciclo in G .

Algoritmo 2.2.9 – Algoritmo sbagliato per i cicli

L’idea di partenza (sbagliata) è: visita il grafo, e se nel corso della visita incontri un nodo già visitato interrompi e restituisci True, se al contrario la visita termina regolarmente restituisci False.

```
def ciclo(u, G):
    def DFSr(u, G, visitati):
        visitati[u]=1
        for v in G[u]:
            if visitati[v]==1:
                return True
            if DFSr(v, G, visitati):
                return True
        return False

    visitati=[0]*len(G)
    return DFSr(u, G, visitati)
```

Il codice è sbagliato sia con grafi non diretti che con grafi diretti:

- Il codice è sbagliato perché (per come è codificato il grafo non diretto) se x ha un arco che lo collega a y nella lista di adiacenza di x è presente y e nella lista di adiacenza di y è presente x . Questo viene interpretato erroneamente come un ciclo di lunghezza 2 e la procedura proposta termina quindi sempre con True.
- Il codice è sbagliato perché incontrare in un grafo diretto un nodo già visitato non significa necessariamente che si è in presenza di un ciclo e la procedura può terminare con True anche in assenza di ciclo.

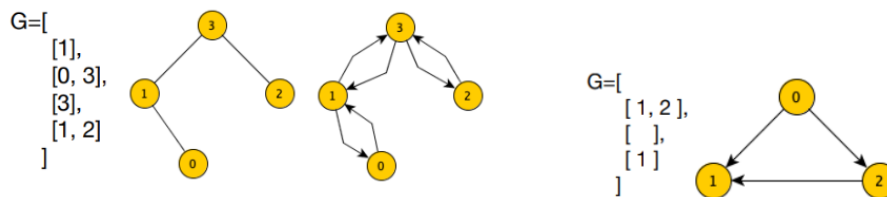


Figura 2.11: Cicli sbagliati di grafi non diretti e diretti

Algoritmo 2.2.10 – Algoritmo per i cicli con i grafi non diretti

Per risolvere il problema, durante la visita alla ricerca del ciclo, devo distinguere nella lista di adiacenza di ciascun nodo y che incontro il nodo x che mi ha portato a visitarlo (il padre di y nell'albero DFS).

```
def ciclo(u, G):
    def DFSr(u, padre, G, visitati):
        visitati[u]=1
        for v in G[u]:
            if visitati[v]==1:
                if v!=padre:
                    return True
            else:
                if DFSr(v, u, G, visitati):
                    return True
        return False

    visitati=[0]*len(G)
    return DFSr(u, u, G, visitati)
```

La complessità $O(n)$ è dovuta al fatto che se il grafo non contiene cicli allora ha al più $n - 1$ archi e quindi $O(n + m) = O(n)$. Se al contrario il grafo contiene cicli se ne trova uno dopo aver considerato al più n archi e a quel punto la procedura termina la visita.

Algoritmo 2.2.11 – Algoritmo per i cicli con i grafi diretti

Durante la visita DFS posso incontrare nodi già visitati in tre modi diversi: archi in avanti (cioè le frecce dirette da un antenato a un discendente), archi all'indietro (cioè le frecce dirette da un discendente ad un antenato) e archi di attraversamento. Solo la presenza di archi all'indietro testimonia la presenza di un ciclo.

Per risolvere il problema, durante la visita DFS alla ricerca del ciclo, devo poter distinguere la scoperta di nodi già visitati grazie ad un arco all'indietro dagli altri. Posso individuare i visitati da archi all'indietro notando che solo nel caso di archi all'indietro la visita del nodo già visitato ha terminato la sua ricorsione.

Per il vettore V dei visitati uso tre step:

- in V un nodo vale 0 se il nodo non è stato ancora visitato;
- in V un nodo vale 1 se il nodo è stato visitato ma la ricorsione su quel nodo non è ancora finita;
- in V un nodo vale 2 se il nodo è stato visitato e la ricorsione su quel nodo è finita.

In questo modo scopro un ciclo quando trovo un arco diretto verso un nodo già visitato che si trova nello stato 1.

```
def ciclo(u, G):
    def DFSr(u, G, visitati):
```

```

    visitati[u]=1
    for v in G[u]:
        if visitati[v]==1:
            return True
        if visitati[v]==0:
            if DFSr(v, G, visitati):
                return True
    visitati[u]=2
    return False

    visitati=[0]*len(G)
    return DFSr(u, G, visitati)

```

La complessità è $O(n + m)$.

Se voglio sapere se un grafo (diretto o non diretto) contiene un ciclo o meno, devo visitarlo tutto non importa il punto da cui parto. Non è difficile modificare le procedure appena viste senza alterarne la complessità $O(n + m)$.

2.2.6 Ricerca dei ponti

In un grafo la connessione è una proprietà che può andar persa con la perdita di un arco. Un arco la cui eliminazione disconnette il grafo è detto *ponte*. I ponti rappresentano criticità del grafo ed è quindi utile identificarli. Il grafo può non avere neanche un ponte così come avere tutti i suoi archi come ponti.

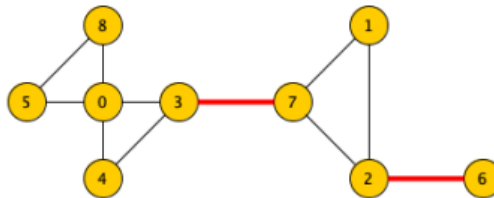


Figura 2.12: Esempio di ponte

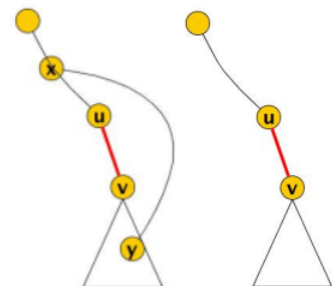
Lemma 2.2.3 – Ponte in un grafo

Sia $\{u, v\}$ un arco dell'albero DFS con u padre di v . L'arco $\{u, v\}$ è un ponte se e solo se non ci sono archi tra i nodi del sottoalbero radicato in v e il nodo u o nodi antenati di u .

Dimostrazione. Sia $\{x, y\}$ un arco tra un antenato di u e un discendente di v . Dopo l'eliminazione dell'arco $\{u, v\}$ tutti i nodi dell'albero restano comunque connessi grazie all'arco $\{x, y\}$.

Se, al contrario $\{x, y\}$ non è un arco tra un antenato di u e un discendente di v , l'eliminazione dell'arco $\{u, v\}$ disconnette i nodi dell'albero radicato in v dal resto del grafo.

Osservazione. Nel momento in cui l'arco $\{u, v\}$ viene attraversato per raggiungere il nodo v da visitare, il nodo u non ha informazioni per decidere se dal sotto-albero radicato in v partono archi verso suoi antenati (quindi scoprire se $\{u, v\}$ è ponte o meno). Ma gli sarà possibile scoprirlo al termine della visita di v se riceve da v la giusta informazione.

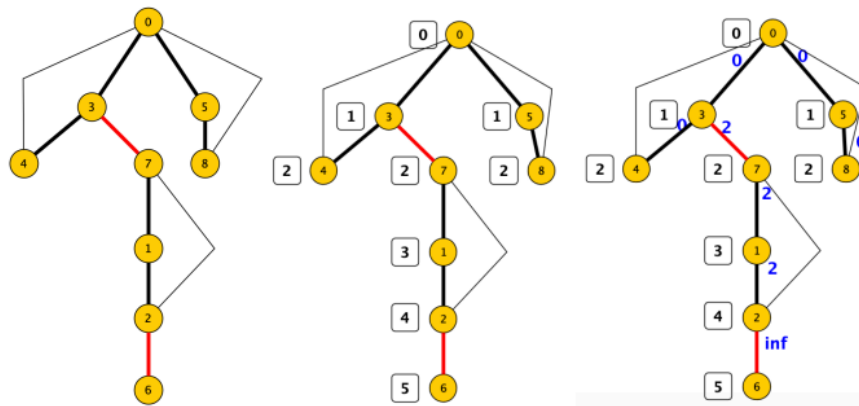


Algoritmo 2.2.12 – Ricerca dei ponti in un grafo

Dato un grafo connesso G determinare l'insieme di ponti del grafo.

- Una prima soluzione basata sulla ricerca esaustiva: prova per ogni arco del grafo se questo è ponte o meno. Verificare se un arco $\{a, b\}$ è un ponte per G richiede tempo $O(m)$. Basta ad esempio eliminare l'arco $\{a, b\}$ da G e, con una visita DFS, controllare se b è raggiungibile a partire da a . La complessità di questo algoritmo è $m * O(m) = O(m^2) = O(n^4)$.
- Vedremo ora che il problema è risolvibile in tempo lineare nella dimensione del grafo $O(m)$. I ponti vanno ricercati unicamente tra gli $n - 1$ archi dell'albero DFS. Infatti un arco non presente nell'albero DFS non può essere ponte (se lo elimino gli archi dell'albero DFS continuano a garantire la connessione).

Durante la visita ogni nodo x calcola la sua altezza nell'albero DFS e a fine visita restituisce l'altezza minima raggiungibile da nodi del sottoalbero radicato in x tramite gli archi non presenti nell'albero DFS. Se non ci sono archi di questo tipo allora viene restituito $+\infty$. Il nodo u confrontando la sua altezza con il valore che gli viene segnalato dai figlio v è in grado di decidere se l'arco $\{u, v\}$ è ponte o meno. $\{u, v\}$ è ponte se e solo se l'altezza di u è strettamente minore del valore restituito da v .



L'algoritmo che risolve il problema con complessità $O(n + m)$ è il seguente:

```
def TrovaPonti(G):
    def DFSModificata(x, t, G, Altezza, Ponti):
        from math import inf
        Altezza[x] = t
        ret = inf
        for y in G[x]:
            if Altezza[y] == -1:
                a = DFSModificata(y, t+1, G, Altezza, Ponti)
                if t < a:
                    Ponti.append((x, y))
                ret = min(ret, a)
            elif Altezza[y] != t-1:
                ret = min(ret, Altezza[y])
        return ret

    Altezza = [-1] * len(G)
    Ponti = []
    DFSModificata(0, 0, G, Altezza, Ponti)
    return Ponti
```

2.3 Visita in ampiezza (BFS)

Dati due nodi a e b di un grafo G , definiamo distanza (minima) in G di a da b il numero minimo di archi che bisogna attraversare per raggiungere b a partire da a . La distanza è posta a $+\infty$ se b non è raggiungibile partendo da a .

La visita in ampiezza (Breadth First Search) esplora i nodi del grafo partendo da quelli a distanza 1 dalla sorgente s . Poi visita quelli a distanza 2 e così via. L'algoritmo visita tutti i vertici a livello k prima di passare a quelli a livello $k+1$. Si genera un albero detto albero BFS dove si visitano nodi sempre più distanti dalla radice.

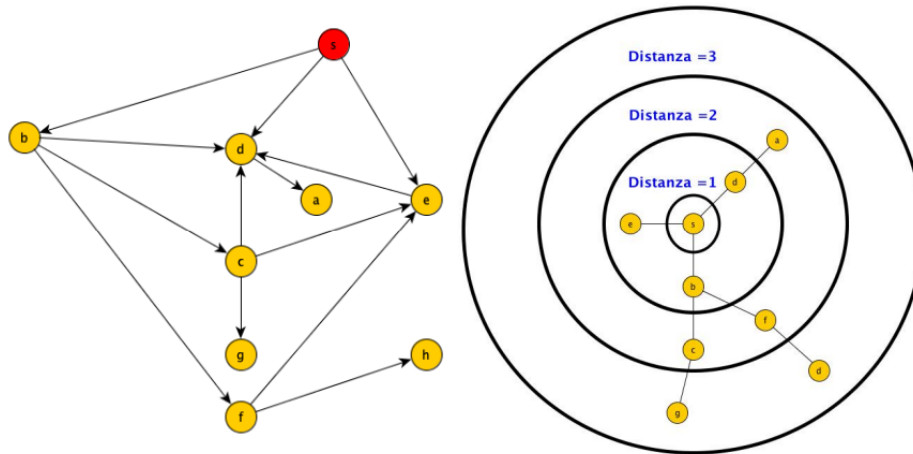


Figura 2.13: Esempio di visita in ampiezza (BFS)

Algoritmo 2.3.1 – Visita in ampiezza (BFS)

Dato un grafo G ed un suo nodo x vogliamo conoscere le distanze di tutti i nodi di G da x . Più precisamente vogliamo calcolare il vettore delle distanze D , dove in $D[y]$ troviamo la distanza di y da x .

Per effettuare questo tipo di visita manteniamo in una coda i nodi visitati i cui adiacenti non sono stati ancora esaminati. Ad ogni passo, preleviamo il primo nodo dalla coda, esaminiamo i suoi adiacenti e se scopriamo un nuovo nodo lo visitiamo e lo aggiungiamo alla coda.

```
def BFS(x, G):
    visitati = [0] * len(G)
    visitati[x] = 1
    coda = [x]
    while coda:
        x = coda.pop(0)
        for y in G[x]:
            if visitati[y] == 0:
                visitati[y] = 1
                coda.append(y)
    return visitati
```

Un nodo finisce in coda al più una volta (poi risulterà visitato e non ci finisce più) quindi il while verrà eseguito $O(n)$ volte. Le liste di adiacenza dei nodi verranno scorse al più una volta quindi il costo totale dei `for y in G[x]` sarà $O(m)$. Se le operazioni di inserimento e cancellazione dalla coda richiedessero tempo $\Theta(1)$ avremmo complessità $O(n + m)$. Ma abbiamo implementato la coda tramite lista inserendo in coda e cancellando in testa, dove ogni inserimento in coda `append()` costa $O(1)$, ogni estrazione in testa `pop(0)` ha un costo proporzionale al numero di elementi presenti in quel momento nella coda/lista e questi elementi possono essere anche $O(n)$. Quindi il costo di questa implementazione dell'algoritmo è $O(n^2)$.

L'implementazione alternativa di costo $O(n + m)$ consiste di effettuare solo cancellazioni logiche e non effettive: usando un puntatore che indica l'inizio della coda all'interno della lista,

il puntatore si incrementa ogni volta che si cancella dalla coda. Alternativamente si può utilizzare la struttura dati deque dal modulo `collections` che permette di effettuare inserimenti e cancellazioni da entrambi i lati di una lista in tempo $O(1)$. In particolare può estrarre a sinistra `popleft()` e inserire a sinistra `appendleft()`.

```
def BFS(x, G):
    visitati=[0]*len(G)
    visitati[x]=1
    coda=[x]
    i=0
    while len(coda)>i:
        u=coda[i]
        i+=1
        for y in G[u]:
            if visitati[y]==0:
                visitati[y]=1
                coda.append(y)
    return visitati

def BFS(x, G):
    visitati=[0]*len(G)
    visitati[x]=1
    from collections import deque
    coda=deque([x])
    while coda:
        u=coda.popleft()
        for y in G[u]:
            if visitati[y]==0:
                visitati[y]=1
                coda.append(y)
    return visitati
```

Teorema 2.3.1 – Correttezza dell’algoritmo di visita in ampiezza (BFS)

Alla fine dell’algoritmo `visitati[u]` contiene 1 se il nodo u è raggiungibile da x , 0 altrimenti.

Dimostrazione.

- Se u è raggiungibile a partire da x allora `visitati[u]=1`. Se u è raggiungibile c’è un cammino P da x a u . Supponiamo per assurdo che al termine `visitati[u]=0`. Sia b il primo nodo che incontro nel cammino P con `v[b]=0` e a il suo predecessore (questi due nodi esistono perché il cammino comincia con x e `visitati[x]=1` e termina con u e `visitati[u]=0`). Un nodo che diventa 1 finisce in coda, quindi a è finito in coda, ma l’algoritmo si ferma quando la coda si svuota quindi c’è stato un momento in cui a è stato estratto dalla coda, in quel momento tutti i suoi vicini sono stati controllati e se non visitati sono stati visitati (e poi inseriti in coda) quindi anche b è stato controllato e se non ancora visitato sarebbe stato visitato e posto a 1.
- Se u non è raggiungibile a partire da x allora `visitati[u]=0`. Supponiamo per assurdo che al termine `visitati[u]=1`. `visitati[u]` diventa 1 quando viene inserito in coda ma questo accade perché risulta adiacente ad un nodo che viene estratto dalla coda, andando all’indietro questo processo deve terminare perché ciascun nodo viene inserito al più una volta nella coda quindi si crea un cammino da x a u il che è assurdo avendo supposto che u non è raggiungibile da x .

Algoritmo 2.3.2 – Vettore dei padri in BFS

Modifichiamo la procedura in modo che restituisca in $O(n + m)$ l’albero di visita BFS rappresentato tramite il vettore dei padri.

```
def BFSpadri(x, G):
    P=[-1]*len(G)
    P[x]=x
    coda=[x]
    i=0
    while len(coda)>i:
        u=coda[i]
        i+=1
        for y in G[u]:
            if P[y]==-1:
```

```

        P[y]=u
        coda.append(y)
    return P

```

Grazie al vettore dei padri P in $O(n)$ potremo ottenere un cammino in G dalla radice dell'albero al nodo x (posto che sia raggiungibile).

Lemma 2.3.1 – Profondità minima nell'albero BFS

La distanza minima di un vertice x da s nel grafo G equivale alla profondità di x nell'albero BFS.

Dimostrazione. Si può dimostrare per induzione sulla distanza d di x da s . E' ovviamente vero per $d = 0$ in quanto l'unico vertice a distanza 0 da s è s stesso che è a profondità 0 nell'albero. Supponiamo sia vero per tutti i vertici con distanza al più $d - 1$ e consideriamo quindi un vertice x a distanza d . Sia P un cammino minimo da s a x e sia v il predecessore di x in questo cammino. Per ipotesi induttiva è a profondità $d - 1$ nell'albero e BFS. Se x è stato inserito nell'albero grazie a v allora si troverà a profondità d , assumiamo quindi che sia stato inserito nell'albero grazie ad un nodo $u \neq v$.

La profondità di u non può essere inferiore a $d - 1$ altrimenti avremmo trovato un cammino che parte da s e porta a v (tramite u) di lunghezza inferiore a d . D'altra parte non può essere la profondità di u maggiore di $d - 1$ perché il nodo v sarebbe stato visitato prima di u e x sarebbe stato inserito grazie al nodo v . Si deve necessariamente avere che la profondità di u è $d - 1$ e quindi la profondità di v è comunque d .

Grazie alla proprietà appena dimostrata i cammini prodotti grazie all'albero sono quelli di lunghezza minima ecco perché l'albero BFS che si ottiene dalla visita è detto anche albero dei cammini minimi.

Algoritmo 2.3.3 – Vettore delle distanze in BFS

Modifichiamo la procedura in modo che restituisca in $O(n + m)$ il vettore delle distanze D . Al nodo x viene assegnata distanza zero e a tutti gli altri nodi il valore -1. A ciascun nodo via via visitato viene assegnata la distanza corrispondente al padre incrementata di 1. Al termine $D[u]$ conterrà -1 se il nodo u non è raggiungibile a partire da x , la distanza minima di u da x altrimenti.

```

def BFSdistanze(x, G):
    D=[-1]*len(G)
    D[x]=0
    coda=[x]
    i=0
    while len(coda)>i:
        u=coda[i]
        i+=1
        for y in G[u]:
            if D[y]==-1:
                D[y]=D[u]+1
                coda.append(y)
    return D

```

2.4 Grafi pesati

Ci sono problemi che possono essere naturalmente rappresentati tramite un grafo pesato dove ogni arco ha associato un valore numerico. Per rappresentare questo tipo di grafi per l'arco (x, y) di peso c nella lista di adiacenza di x invece che il solo nodo destinazione y ci sarà la coppia (y, c) con l'informazione sul nodo destinazione e il peso dell'arco.

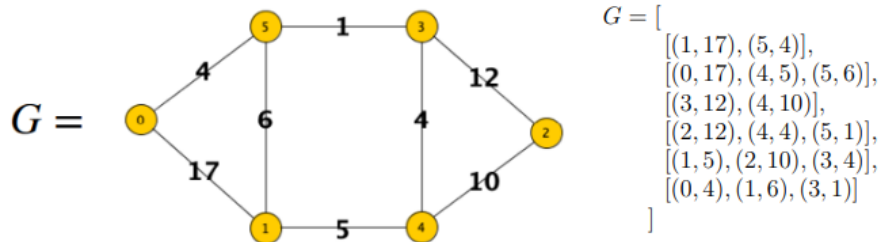


Figura 2.14: Rappresentazione di grafi pesati

Vedremo ora un algoritmo che permette di trovare i cammini minimi lavorando direttamente su grafi pesati (con pesi anche non interi) l'algoritmo è noto come algoritmo di Dijkstra.

2.4.1 Algoritmo di Dijkstra

Algoritmo 2.4.1 – Algoritmo di Dijkstra

Dato un grafo pesato vogliamo trovare i cammini minimi e quindi anche le distanze da un certo nodo s (detto sorgente) a tutti gli altri nodi del grafo.

Si seguono questi passi:

- costruisci l'albero dei cammini minimi un arco per volta partendo dal nodo sorgente;
- ad ogni passo aggiungi all'albero l'arco che produce il nuovo cammino più economico;
- alla nuova destinazione assegna come distanza il costo del cammino.

Un vettore binario `Inserito[x]` ci dice se x è già nell'albero o meno. In `Lista[x]` è presente la coppia (*distanza_minima*, *padre*) il cui significato è il seguente:

1. se x è inserito nell'albero: *padre* è il suo nodo padre e *distanza_minima* la sua vera distanza dalla sorgente;
2. se x non è inserito nell'albero e non è adiacente ad un nodo inserito nell'albero allora (*distanza_minima*, *padre*) = (∞ , -1);
3. se x non è inserito nell'albero ma è adiacente a nodi dell'albero allora *padre* è il nodo dell'albero più conveniente a cui attaccare x e *distanza* è la distanza minima che ne risulterebbe.

All'inizio l'unico nodo nell'albero è la sorgente. Nel seguito i nodi verranno inseriti via via nell'albero e si terminerà solo quando non ci saranno più nodi da inserire o tutti i nodi non ancora inseriti avranno valore $+\infty$ (il che significa che non sono nella stessa componente della sorgente s).

```
def dijkstra(s, G):
    Inserito=[False]*len(G)
    from math import inf
    Lista=[(inf, -1)]*len(G)
    Lista[s],Inserito[s]=(0, s),True
    for y, costo in G[s]:
        Lista[y]=(costo, s)
    while True:
        minimo=inf
        for i in range(len(Lista)):
            if not Inserito[i] and Lista[i][0]<minimo:
                minimo,x=Lista[i][0],i
        if minimo==inf:
```

```

    break
    Inserito[x]=True
    for y, costo in G[x]:
        if not Inserito[y] and minimo+costo<Lista[y][0]:
            Lista[y]=(minimo+costo, x)
    D=[costo for costo, _ in Lista]
    P=[padre for _, padre in Lista]
    return D, P

```

Il costo totale dell'esecuzione delle istruzioni al di fuori del while è $\Theta(n)$. Abbiamo poi un while che viene eseguito al più $n - 1$ volte visto che ad ogni iterazione un nodo non inserito viene inserito nell'albero. All'interno del while c'è un primo for che viene iterato ogni volta esattamente n volte ed un secondo for che viene eseguito tante volte quanti sono gli adiacenti presenti nella lista del nodo appena inserito nell'albero (quindi il numero di queste iterazioni è $O(n)$). In pratica ognuna delle $O(n)$ iterazioni del while costa $\Theta(n)$ e quindi il costo del while è $O(n^2)$. La complessità di questa implementazione è dunque $\Theta(n) + O(n^2) = O(n^2)$. Questa implementazione è ottima nel caso di grafi densi dove $m = \Theta(n^2)$.

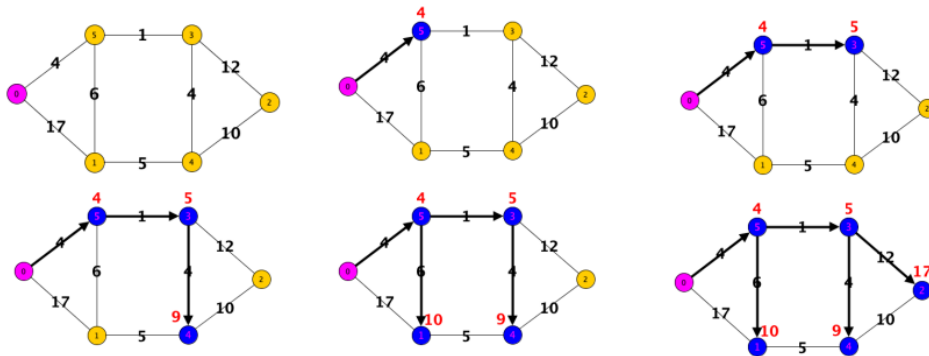


Figura 2.15: Esempio algoritmo di Dijkstra

Teorema 2.4.1 – Correttezza algoritmo di Dijkstra

La prima cosa da notare è che l'algoritmo non è corretto nel caso di grafi con pesi anche negativi. Nel caso di pesi positivi, ad ogni iterazione del while viene assegnata una nuova distanza ad un nodo. Per induzione sul numero di iterazioni mostreremo che la distanza assegnata è quella minima.

Il caso base è banale (al passo 0 viene assegnata distanza zero alla sorgente e con pesi positivi non può esserci una distanza inferiore). Sia T_i l'albero dei cammini minimi costruito fino al passo $i > 0$ e (u, v) l'arco aggiunto all'albero al passo $i + 1$. Faremo vedere che $D[v]$ è la distanza minima di v da s . Basterà mostrare che il costo di un eventuale cammino alternativo è sempre superiore o uguale a $D[v]$.

- Sia C un qualsiasi cammino da s a v alternativo a quello presente nell'albero e (x, y) il primo arco che incontriamo percorrendo il cammino C all'indietro tale che x è nell'albero T_i e y no (tale arco deve esistere perché s è in T_i mentre v no).
- Per ipotesi induttiva $\text{costo}(C) \geq \text{Dist}(x) + \text{peso}(x, y)$ (quest'affermazione è vera perché i pesi del grafo sono tutti non negativi).
- L'algoritmo ha preferito estendere l'albero T_i con l'arco (u, v) anziché l'arco (x, y) e in base alla regola con cui l'algoritmo sceglie l'arco con cui estendere l'albero deve quindi aversi $D[x] + p(x, y) \geq D[u] + p(u, v)$. Da cui segue: $\text{costo}(C) \geq D[x] + \text{peso}(x, y) \geq D[u] + p(u, v) = D[v]$ e il cammino alternativo ha un costo superiore a $D[v]$.

Se si potesse evitare di scorrere ogni volta il vettore `Lista` alla ricerca della posizione in cui è presente il minimo potrei evitare di pagare $\Theta(n)$ ad ogni iterazione del `while`.

Algoritmo 2.4.2 – Algoritmo di Dijkstra con heap minimo

Per ogni nodo x del grafo per cui c'è arco (p, x) con p nell'albero e x non nell'albero, posso mantenere in un heap minimo la tripla $(D[p] + \text{costo}(p, x), x, p)$ (indicizzata rispetto alla prima componente). In questo modo ad ogni step possiamo scoprire in tempo logaritmico nella dimensione dell'heap il nodo x da inserire nell'albero, il costo $D[p] + \text{costo}(p, x)$ da assegnargli e il padre p a cui agganciarlo. Ovviamente ogni volta che aggiungo un nodo all'albero dovrò aggiornare anche l'heap inserendovi nuovi valori $(D[x] + \text{costo}(x, y), y, x)$ per ogni vicino y di x (ricorda che ogni inserimento ha costo logaritmico nella dimensione dell'heap).

```
def dijkstra(s, G):
    P=[-1]*len(G)
    from math import inf
    D=[inf]*len(G)
    D[s], P[s]=0, s
    from heapq import heappop, heappush
    H=[]
    for y, costo in G[s]:
        heappush(H, (costo, s, y))
    while H:
        costo, padre, x=heappop(H)
        if P[x]==-1:
            P[x]=padre
            D[x]=costo
            for y, costo1 in G[x]:
                heappush(H, (D[x]+costo1, x, y))
    return D, P
```

Nota che in H possono esserci anche $O(m)$ elementi e quindi i costi di inserimento ed estrazione saranno $O(\log m) = O(\log n^2) = O(\log n)$.

Prima dell'accesso al `while` abbiamo l'inizializzazione di D e P (per un costo $O(n)$) e l'inserimento nell'heap H di $O(n)$ elementi (per un costo $O(n \log n)$). Abbiamo un `while` con all'interno un `for`. Ad ogni iterazione del `while` si elimina un elemento da H e eventualmente per mezzo del `for` annidato si scorre la lista di adiacenza di un nodo e vengono aggiunti elementi ad H . Ogni lista di adiacenza può essere scorsa al più una volta quindi ad H in totale possono essere aggiunti al più $O(m)$ elementi. Per quanto detto il numero di iterazioni del `while` è $O(m)$. Il costo di ciascuna iterazione del `while` richiede $O(\log n)$ a causa dell'estrazione da H . Quindi il costo totale del `while`, senza tener conto del `for`, sarà $O(m \log m)$. Il tempo totale richiesto dalle varie iterazioni del `for` annidato nel `while` è $O(m \log n)$ in quanto ad ogni iterazione scorro una lista di adiacenza diversa. In totale scorrerò $O(m)$ archi e per ogni arco pagherò $O(\log n)$ per inserimento in H . La complessità di questa implementazione è dunque $O(n \log n) + O(m \log n) + O(m \log n) = O((n + m) \log n)$.

Questa soluzione nel caso di grafi sparsi ha una complessità $O(n \log n)$, mentre andrebbe evitata nel caso di grafi densi dove la complessità risultante sarebbe $O(n^2 \log n)$.

2.4.2 Minimo albero di copertura

In alcuni problemi è richiesta la ricerca nel grafo di un albero che “copre” l'intero grafo e la cui somma dei costi dei suoi archi sia minima. Questo problema prende il nome di *minimo albero di copertura*.

Algoritmo 2.4.3 – Algoritmo di Kruskal

Dato un grafo G connesso e pesato cerchiamo un suo minimo albero di copertura.

L'algoritmo opera come segue:

- Parti con il grafo T che contiene tutti i nodi di G e nessun arco di G .

- Considera uno dopo l'altro gli archi del grafo G in ordine di costo crescente.
- Se l'arco forma un ciclo in T con archi già presi allora non prenderlo altrimenti inseriscilo in T .
- Al termine restituisci T .

Quindi con un pre-processing ordino gli archi in E in ordine decrescente cosicché l'estrazione da E dell'arco di costo minimo costi $O(1)$. Verifico che l'arco (x, y) non formi un ciclo in T controllando se y è raggiungibile da x in T .

```
def kruskal(G):
    E=[(c, u, v) for u in range(len(G)) for v, c in G[u] if u<v]
    E.sort(reverse=True)
    T=[[] for _ in G]
    while E:
        c, x, y = E.pop()
        if not connessi(x, y, T):
            T[x].append(y)
            T[y].append(x)
    return T

def connessi(x, y, T):
    def connessiR(a, b, T):
        visitati[a]=1
        for z in T[a]:
            if z==b:
                return True
            if not visitati[z] and connessiR(z, b, T):
                return True
        return False

    visitati=[0]*len(T)
    return connessiR(x, y, T)
```

L'ordinamento esterno al while costa $O(m \log m) = O(m \log n)$, il while viene iterato m volte (non ad ogni passo viene inserito un arco in T). L'estrazione dell'arco (a, b) di costo minimo da E richiede tempo $O(1)$, controllare che l'arco (a, b) non crei ciclo in T con la procedura `connessi(a, b, T)` richiede il costo di una visita di un grafo aciclico quindi $O(n)$. La complessità di questa implementazione è $O(mn)$.

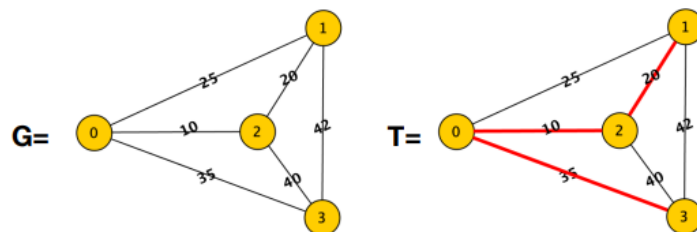


Figura 2.16: Esempio algoritmo di Kruskal

Teorema 2.4.2 – Correttezza dell'algoritmo di Kruskal

Dobbiamo far vedere che al termine dell'algoritmo T è un albero di copertura e che non c'è un altro albero che costa meno.

Dimostrazione.

- *L'algoritmo produce un albero di copertura*: la prova viene fornita per assurdo. Supponiamo che al termine in T ci sia più di una componente. Consideriamone una A , poiché G è connesso nel grafo c'è un arco (x, y) da un nodo x di A ad un nodo y di un'altra componente B ma l'arco (x, y) ad un certo punto è stato estratto da E e se non crea ciclo in T ora che l'algoritmo è terminato non lo creava neanche al momento in cui è stato estratto e quindi sarebbe stato aggiunto a T e non scartato.
- *Non c'è un albero di copertura per G che costa meno di T* : la prova viene fornita per assurdo. Tra tutti gli alberi di copertura di costo minimo per G prendiamo quello che differisce nel minor numero di archi da T . Sia T^* . Supponiamo per assurdo che T differisca da T^* . Faremo vedere che questa assunzione porterebbe all'assurdo perché avrebbe come conseguenza l'esistenza di un altro albero di copertura di costo minimo per G che differisce da T in meno archi di T^* .

Considera l'ordine $e_1, e_2, \dots$ con cui gli archi sono stati presi in considerazione nel corso dell'algoritmo e sia e il primo arco preso che non compare in T^* . Se inserisco e in T^* si forma di certo un ciclo C . Il ciclo C contiene almeno un arco e' che non appartiene a T (infatti non tutti gli archi del ciclo C sono in T altrimenti e non sarebbe stato preso dall'algoritmo di Kruskal). Considera ora l'albero T' che ottengo da T^* inserendo l'arco e ed eliminando e' . Il costo del nuovo albero T' (che è $\text{costo}(T^*) - \text{costo}(e') + \text{costo}(e)$) non può aumentare rispetto a quello di T^* (perché $\text{costo}(e) \leq \text{costo}(e')$, nota infatti che tra i due archi e ed e' che non creavano ciclo Kruskal ha considerato prima l'arco e) ma allora T è un altro albero di copertura ottimo che differisce da T in meno archi di quanto faccia di T^* il che contraddice l'ipotesi che T^* differisce da T nel minor numero di archi.

Algoritmo 2.4.4 – Algoritmo di Kruskal con Union e Find

Per migliorare l'implementazione non possiamo permetterci di spendere $O(n)$ ad ogni iterazione del while a causa della visita per testare la raggiungibilità. Ricorreremo alla struttura dati Union e Find che permette di testare efficientemente se due nodi appartengano o meno alla stessa componente connessa.

```
def kruskal(G):
    E=[(c, u, v) for u in range(len(G)) for v, c in G[u] if u<v]
    E.sort(reverse=True)
    T=[]
    C=crea(T)
    while E!=[]:
        c, x, y = E.pop()
        cx = find(x, C)
        cy = find(y, C)
        if cx!=cy:
            T[x].append(y)
            T[y].append(x)
            union(cx, cy, C)
    return T
```

L'ordinamento costa $O(m \log n)$, il while viene iterato m volte (non ad ogni passo viene inserito un arco in T). L'estrazione dell'arco (a, b) di costo minimo da E richiede $\Theta(1)$, eseguire il Find costa $O(\log n)$, eseguire l'Union costa $\Theta(1)$ (e all'interno del while viene eseguito esattamente $n - 1$ volte).

La complessità di questa implementazione è $O(m \log n)$.

2.5 Union-Find

L'*Union-Find*, noto anche come Disjoint Set Union (DSU), è una struttura utilizzata per operazioni di unione e ricerca efficienti su insiemi disgiunti. Le tre operazioni fondamentali di Union-Find sono:

1. **Crea(S)**: restituisce una struttura dati Union-Find sull'insieme S di elementi dove ciascun elemento è in un insieme separato.
2. **Find(x, C)**: Restituisce il nome dell'insieme della struttura dati C a cui appartiene l'elemento x .
3. **Union(A, B, C)**: modifica la struttura dati C fondendo i due suoi insiemi di nome A e B in un singolo insieme.

Una gestione efficiente di insiemi disgiunti è utile in diversi contesti.

Esempio 2.5.1 – Union-Find con grafi

Si pensi ad esempio allo scenario dove un grafo evolve nel tempo con l'aggiunta di archi.

In questo caso gli insiemi disgiunti sono le componenti connesse del grafo. L'operazione **find(u)** sul nodo u permette di scoprire in quale componente si trova il nodo u . L'operazione può essere usata per testare se due nodi u e v sono nello stessa componente semplicemente verificando se **find(u) == find(v)**.

Se poi l'arco (u, v) viene aggiunto al grafo allora prima si testa se u e v sono nella stessa componente connessa e se risulta $A \neq B$ dove $A = \text{find}(u)$ e $B = \text{find}(v)$ l'operazione **Union(A, B)** può essere usata per fondere le due componenti.

Come primo approccio possiamo pensare di assegnare all'insieme il nome dell'elemento massimo in esso contenuto.

Algoritmo 2.5.1 – Union-Find

Probabilmente il modo più semplice di implementare questa struttura dati per n elementi è di mantenere il vettore C delle n componenti. All'inizio ogni elemento è in un insieme distinto vale a dire $C[i] = i$. Quando la componente i viene fusa con la componente j , se $i > j$ allora tutte le occorrenze di j nel vettore C verranno sostituite da i (se $i < j$ accadrà il contrario).

```
def Crea(G):
    C=[i for i in range(len(G))]
    return C

def Find(u, C):
    return C[u]

def Union(a, b, C):
    if a>b:
        for i in range(len(C)):
            if C[i]==b:
                C[i]=a
    else:
        for i in range(len(C)):
            if C[i]==a:
                C[i]=b
```

Il costo delle operazioni è rispettivamente $\Theta(n)$, $\Theta(1)$, $\Theta(n)$.

Nella implementazione appena vista al **Find** costa $O(1)$ e la **Union** $\Theta(n)$. Meglio bilanciare i costi utilizzando il vettore dei padri.

Algoritmo 2.5.2 – Union-Find con il vettore dei padri

In questa implementazione quando voglio sapere in che componente si trova un nodo devo semplicemente risalire alla sua radice in tempo $O(n)$, mentre quando fondo due componenti una diventa figlia dell'altra in $O(1)$.

```
def Crea(G):
```

```

C=[i for i in range(len(G))]
return C

def Find(u, C):
    while u!=C[u]:
        u=C[u]
    return u

def Union(a, b, C):
    if a>b:
        C[b]=a
    else:
        C[a]=b

```

Se non voglio arrivare a spendere $O(n)$ per la Find i cammini per arrivare alle radici del vettore dei padri non devono essere mai troppo lunghi. Dunque è meglio mantenere gli alberi bilanciati.

Quando con la Union fondo le due componenti rendo sempre radice la componente che ha più elementi. L'intuizione è che in questo modo per almeno la metà dei nodi presenti nelle due componenti coinvolte nella fusione la lunghezza del cammino non aumenta.

Algoritmo 2.5.3 – Union-Find con alberi bilanciati

In questa implementazione della Union-Find devo fare in modo che ai nodi radice sia associato anche il numero di elementi che la componente contiene. Ogni elemento è caratterizzato da una coppia (x, numero) dove x è il nome dell'elemento e numero è il numero di nodi nell'albero radicato in x .

```

def Crea(G):
    C=[(i, 1) for i in range(len(G))]
    return C

def Find(u, C):
    while u!=C[u]:
        u=C[u]
    return u

def Union(a, b, C):
    tota, totb = C[a][1], C[b][1]
    if tota>=totb:
        C[a]=(a, tota+totb)
        C[b]=(a, totb)
    else:
        C[b]=(b, tota+totb)
        C[a]=(b, tota)

```

Con questo accorgimento nella fusione delle componenti vale la seguente proprietà: se una componente ha altezza h allora la componente contiene almeno 2^h nodi. Dalla proprietà deduciamo che l'altezza delle componenti non potrà mai superare $\log n$ (perché in caso contrario avrei nella componente più di n nodi il che è assurdo). Implementando quindi nella Union l'accorgimento descritto la Find richiederà tempo $O(\log n)$ anziché $O(n)$.

Lemma 2.5.1 – Proprietà degli alberi bilanciati

Se una componente ha altezza h allora la componente contiene almeno 2^h nodi.

Dimostrazione. Assumiamo per assurdo durante una delle fusioni si sia formata una nuova componente di altezza h che non rispetta la proprietà. Considera la prima volta che ciò accade

e siano ca e cb le componenti che si fondono. Possono essere accadute due cose:

- ca e cb erano componenti con la stessa altezza allora avevano entrambe altezza $h - 1$ ed ognuna aveva almeno 2^{h-1} elementi (perché nelle fusioni precedenti la proprietà era sempre stata verificata). Quindi il numero totale di elementi della nuova componente è $2^{h-1} + 2^{h-1} = 2^h$ e la proprietà è verificata.
- ca e cb avevano diverse altezze allora l'altezza dopo la fusione è quella della componente di altezza maggiore che doveva essere già altezza h e conteneva da sola già 2^h elementi.

Capitolo 3

Greedy

Per illustrare il progetto e l'analisi di un algoritmo greedy consideriamo qualche problema.

3.1 Algoritmi greedy

3.1.1 Selezione di attività

Abbiamo una lista di n attività da eseguire: ciascuna attività è caratterizzata da una coppia con il suo tempo di inizio ed il suo tempo di fine, due attività sono compatibili se non si sovrappongono. Vogliamo trovare un sottoinsieme di attività compatibili di massima cardinalità.

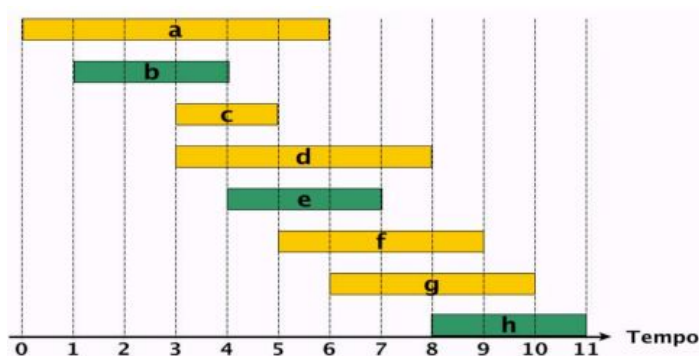


Figura 3.1: Istanza del problema con $n = 8$

Volendo utilizzare il paradigma greedy dovremmo trovare una regola, semplice da calcolare, che ci permetta di effettuare ogni volta la scelta giusta. Per questo problema ci sono diverse potenziali regole di scelta: prendi l'attività compatibile che inizia prima; prendi l'attività compatibile che dura meno; prendi l'attività compatibile che ha meno conflitti con le rimanenti.

La regola giusta sta nel prendere sempre l'attività compatibile che finisce prima.

Dimostrazione. Supponiamo per assurdo che la soluzione greedy SOL trovata con questa regola non sia ottima. Le soluzioni ottime differiscono dunque da SOL . Nel caso ci fossero più di una soluzione ottima prendiamo quella che differisce nel minor numero di attività da SOL , sia SOL^* . Dimosteremo ora che esiste un'altra soluzione ottima SOL' che differisce ancora meno da SOL il che è assurdo.

Siano $A_1, A_2, \dots$ le attività nell'ordine in cui sono state selezionate dal greedy sia A_i la prima attività scelta dal greedy e non dall'ottimo (questa attività deve esistere perché tutte le attività scartate dal greedy erano incompatibili con quelle prese dal greedy e se la soluzione avesse preso tutte le attività scelte dal greedy non potrebbe averne prese di più). Nell'ottimo deve esserci un'altra attività A' che va in conflitto con A_i (altrimenti SOL^* non sarebbe ottima in quanto potrei aggiungervi l'attività A_i). A questo punto posso sostituire in SOL^* l'attività A' con l'attività A_i senza creare conflitti (perché in base alla regola di scelta greedy A_i termina prima di A'). Ottengo in questo modo una soluzione ottima SOL' (infatti le attività in SOL' sono tutte compatibili e la cardinalità di SOL' è la stessa di SOL^*). Ma SOL' differisce da SOL di un'attività in meno rispetto a SOL^* .

Si può proseguire in questo modo:

- cercare in lista l'attività che finisce prima scorrendo la lista costerebbe $O(n)$ ad ogni iterazione del while. Conviene fare un pre-processing ordinando le attività in lista per tempo di fine crescente. In questo modo paghiamo $\Theta(n \log n)$ per l'ordinamento ma poi l'estrazione dell'attività ad ogni iterazione del file costerà $\Theta(1)$.
- Per eseguire efficientemente il test richiesto dall'if conviene mantenere una variabile con il tempo di fine dell'ultima attività inserita in soluzione in questo modo il test richiederà $\Theta(1)$.

L'algoritmo risultante è:

```
def selezione_a(lista):
    lista.sort(key=lambda x:x[1])
    libero=0
    sol=[]
    for inizio, fine in lista:
        if libero<inizio:
            sol.append((inizio, fine))
            libero=fine
    return sol
```

Ordinare la lista delle attività costa $\Theta(n \log n)$. Il for viene eseguito n volte e il costo di ogni iterazione è costante. Il costo finale è $\Theta(n \log n)$.

3.1.2 Assegnazione di attività

Abbiamo una lista di attività, ciascuna caratterizzata da un tempo di inizio ed un tempo di fine. Le attività vanno tutte eseguite e vogliamo assegnarle al minor numero di aule tenendo conto che in una stessa aula non possono eseguirsi più attività in parallelo.



Figura 3.2: Istanza del problema con lista=[(1,4), (1,6), (7,8), (5,10)]

Un possibile algoritmo greedy si basa sull'idea di occupare aule finché ci sono attività da assegnare e ad ogni aula, una volta inaugurata, assegnare il maggior numero di attività non ancora assegnate che è in grado di contenere.

Dimostrazione. Sia k il numero di aule utilizzate dalla soluzione. Faremo vedere che ci sono nella lista k attività incompatibili a coppie e questo ovviamente implica che k aule sono necessarie.

Sia (a, b) l'attività che ha portato all'introduzione nella soluzione della k -ma aula. In quel momento nelle altre $k - 1$ aule era impossibile eseguire l'attività (a, b) ma per il criterio di scelta greedy posso anche dire che nell'istante di tempo a erano tutte occupate (le attività loro assegnate iniziavano prima del tempo a e non erano ancora finite) quindi nell'istante di tempo a a due a due tutte queste k attività sono incompatibili.

Si può proseguire in questo modo:

- per individuare efficientemente l'attività che inizia prima effettuo un pre-processing in cui ordino le attività per tempo di inizio.
- per individuare efficientemente se una delle aule in soluzione è in grado di eseguire un certo lavoro (a, b) uso un heap minimo in cui metto le coppie $(libera, i)$ dove $libera$ indica il

tempo in cui si libera l'aula i . In questo modo in tempo $O(1)$ posso sapere qual'è l'aula che si libera prima semplicemente verificando $H[0][0]$.

Se nell'aula che si libera prima si può eseguire l'attività allora dovrò assegnargliela e aggiornare il valore *libera* della coppia che la rappresenta nell'heap. Se al contrario l'aula non può accogliere l'attività allora non sarà possibile farlo in nessuna delle altre aule e bisognerà assegnare l'attività ad una nuova aula ed inserire nell'heap la coppia che rappresenta questa nuova aula.

L'algoritmo risultante è:

```
def assegnazioneAule(lista):
    from heapq import heappop, heappush
    Sol = [[]]
    H = [(0, 0)]
    lista.sort()
    for inizio, fine in lista:
        libera, aula = H[0]
        if libera < inizio:
            Sol[aula].append((inizio, fine))
            heappop(H)
            heappush(H, (fine, aula))
        else:
            Sol.append([(inizio, fine)])
            heappush(H, (fine, len(Sol) - 1))
    return Sol
```

Ordinare la lista costa $(n \log n)$, il for viene eseguito n volte ed all'interno del for al caso peggio può essere eseguita un'estrazione dall'heap seguita subito dopo da un inserimento nell'heap, entrambe operazioni di costo $O(\log n)$. Il for richiederà quindi tempo $O(n \log n)$. La complessità dell'algoritmo è $O(n \log n)$.

3.1.3 File su disco

Abbiamo n file di dimensioni $d_0, d_1, \dots, d_{n-1}$ che vorremmo memorizzare su un disco di capacità k . Tuttavia la somma delle dimensioni di questi file eccede la capacità del disco. Vogliamo dunque selezionare un sottoinsieme degli n file che abbia cardinalità massima e che possa essere memorizzato sul disco. Descrivere un algoritmo greedy che risolve il problema in tempo $O(n \log n)$ e provarne la correttezza.

Ad esempio: per $D = [5, 6, 3, 5, 4, 7, 3]$ e $k = 11$ la risposta deve essere la tripla di file 2, 4 e 6 che occupa spazio 10.

Un algoritmo greedy per questo problema è quello che si presenta naturalmente: consideriamo i file per dimensione crescente e se c'è spazio per memorizzare il file su disco allora facciamo.

Dimostrazione. Assumiamo per assurdo che la soluzione *sol* prodotta dal greedy non sia ottima, devono quindi esistere insieme con più file di *sol* che rispettano la capacità del disco. Tra questi insieme prendiamo quello con più elementi in comune con *sol* e denotiamolo con *sol**. Nota che:

- esiste un file a che appartiene a *sol** e non a *sol* e che occupa più spazio di qualunque file in *sol* (questo perché tutti gli elementi in *sol* occupano meno spazio di quelli non presenti in *sol*);
- esiste un file b che appartiene a *sol* e non a *sol** (questo perché $\text{sol} \not\subseteq \text{sol}^*$ infatti l'aggiunta di un qualunque elemento a *sol* porterebbe a superare la capacità del disco).

Posso dunque eliminare da *sol** il file a e inserire il file b ottenendo un nuovo insieme di file che rispetta ancora le capacità del disco ed ha un elemento in più in comune con *sol* contraddicendo la nostre ipotesi (il file *sol** è quello con più elementi in comune con *sol*).

L'algoritmo risultante è:

```
def file(D, k):
    n = len(D)
    lista = [(D[i], i) for i in range(n)]
    lista.sort()
    spazio, sol = 0, []
```

```

for d, i in lista:
    if spazio+d<=k:
        sol.append(i)
        spazio+=d
    else:
        return sol

```

La complessità dell'algoritmo è $\Theta(n \log n)$.

3.1.4 Elementi in una lista

Abbiamo due liste di interi A e B ciascuna con $2n$ elementi. Dobbiamo selezionare n elementi dalla lista A ed n elementi dalla lista B in modo che la somma dei $2n$ elementi selezionati sia massima. Non è possibile selezionare elementi che nelle due liste occupino la stessa posizione. Progettare un algoritmo che prende come parametro le due liste A e B e in tempo $O(n \log n)$ restituisce la somma massima che si può ottenere. Motivare la correttezza e la complessità del vostro algoritmo.

Ad esempio: per $A = [10, 2, 4, 6, 1, 7, 3, 4]$ e $B = [6, 6, 1, 0, 3, 8, 5, 7]$ la selezione ottima si ottiene selezionando $A = [10, -, 4, 6, -, 7, -, -]$ e $B = [-, 6, -, -, 3, -, 5, 7]$.

La scelta di un elemento $A[i]$ significa anche perdere la possibilità di scegliere $B[i]$ quindi conviene massimizzare non tanto $A[i]$ quanto $A[i] - B[i]$.

Una implementazione diretta produrrebbe una complessità $O(n^2)$ (infatti costerebbe $O(n)$ la ricerca dell'elemento i per cui è massimo $A[i] - B[i]$) e questa operazione andrebbe fatta n volte all'interno del for. Per ottenere $O(n \log n)$ effettuiamo un pre-processing riordinando gli elementi di A in modo che risulti decrescente il valore $A[i] - B[i]$. Fatto questo ordinamento gli elementi da selezionare in A saranno gli n che risultano al primo posto della lista.

Dimostrazione. Sia Sol^* il valore ottimo e SOL il valore prodotto dall'algoritmo. Assumiamo per assurdo che SOL e Sol^* non coincidano e che quindi le scelte fatte dall'algoritmo e quelle che hanno portato a Sol^* differiscono. Sia Sol^* la sequenza di scelte che porta all'ottimo e che differisce meno dalle scelte fatte dall'algoritmo. Faremo vedere che esiste una sequenza di scelte SOL' che porta all'ottimo e nelle scelte fatte differisce dall'algoritmo greedy meno di quanto faccia Sol^* .

Sia i un elemento in A scelto dal greedy ma non in Sol^* (un tale elemento deve esistere perché la soluzione ottima differisce dalla soluzione greedy). Deve anche esistere in A un elemento j preso dalla soluzione ottima ma non da quella greedy (perché entrambe le soluzioni pescano in ciascuno degli insiemi lo stesso numero di elementi). Deve aversi $A[i] - B[i] \geq A[j] - B[j]$ (grazie alla regola greedy che ha portato a scegliere l'elemento i e non l'elemento j). Da cui deduciamo $A[i] - B[i] - A[j] + B[j] \geq 0$.

Considera ora la soluzione SOL' che si ottiene a partire da Sol^* scambiando la scelta fatta per i e j . Inserendo cioè gli elementi i di B al posto di quello di A e l'elemento j di B al posto di quello di A . La soluzione SOL' differisce meno da quella greedy di quanto faccia Sol^* . Resta da dimostrare che è ottima (vale a dire che produce lo stesso valore di quella prodotta da Sol^*).

La variazione di valore tra SOL' e Sol^* è data da $A[i] - B[i] - A[j] + B[j]$ e noi sappiamo che questo valore è positivo o nullo. Positivo non può essere (avremmo altrimenti trovato una soluzione che vale più dell'ottimo) deve quindi essere nulla. Questo significa che SOL' e Sol^* hanno lo stesso valore.

L'algoritmo risultante è:

```

def scelta(A, B):
    n=len(A)
    lista=[(A[i]-B[i], i) for i in range(n)]
    lista.sort(reverse=True)
    sol=0
    for i in range(n//2):
        sol+=A[lista[i][1]]
    for i in range(n//2, n):
        sol+=B[lista[i][1]]
    return sol

```

La complessità dell'algoritmo è $O(n \log n)$.

3.2 Algoritmi di approssimazione

Ci sono moltissimi problemi di ottimizzazione come la copertura tramite nodi che sono computazionalmente difficili. Per questi problemi non si conoscono algoritmi neanche lontanamente efficienti (sostanzialmente per questi problemi sono noti solo algoritmi esponenziali).

In questi casi potrebbe essere già soddisfacente ottenere una soluzione che sia soltanto "vicina" ad una soluzione ottima e, ovviamente, più è vicina e meglio è. Fra gli algoritmi che non trovano sempre una soluzione ottima, è importante distinguere due categorie piuttosto differenti: algoritmi di approssimazione ed euristiche.

- Gli algoritmi di approssimazione sono algoritmi per cui si dimostra che la soluzione prodotta ha una certa "vicinanza" alla soluzione ottima. In altre parole, è garantito che la soluzione prodotta approssima entro un certo grado una soluzione ottima.
- Le euristiche sono algoritmi per cui non si riesce a dimostrare che la soluzione prodotta ha sempre una certa vicinanza ad una soluzione ottima. Però, sperimentalmente sembrano comportarsi bene. Sono l'ultima spiaggia, quando non si riesce a trovare algoritmi corretti efficienti né algoritmi di approssimazione efficienti che garantiscano un buon grado di approssimazione.

Per una gran parte dei problemi computazionalmente difficili non solo non si conoscono algoritmi corretti efficienti ma neanche buoni algoritmi di approssimazione. Non è quindi sorprendente che fra tutti i tipi di algoritmi, gli algoritmi euristici costituiscano la classe più ampia e che ha dato luogo ad una letteratura sterminata.

Un algoritmo di approssimazione per un dato problema è un algoritmo per cui si dimostra che la soluzione prodotta approssima sempre entro un certo grado una soluzione ottima per il problema. Si tratta quindi di specificare cosa si intende per "approssimazione entro un certo grado".

Iniziamo con problemi di minimizzazione (come ad esempio il problema della copertura tramite vertici) dove ad ogni soluzione ammissibile è associato un costo e cerchiamo quindi la soluzione ammissibile di costo minimo. Sia p un qualsiasi problema di minimizzazione ed I una sua istanza. Indichiamo con $OPT(I)$ il costo di una soluzione ottima per l'istanza e con $A(I)$ la soluzione prodotta dall'algoritmo A per quell'istanza.

Il modo usuale di misurare il grado di approssimazione di un algoritmo di minimizzazione è il rapporto al caso pessimo tra il costo della soluzione prodotta dall'algoritmo e il costo della soluzione ottima: si dice che A approssima P entro un fattore di approssimazione ρ se per ogni istanza I di P vale $\frac{A(I)}{OPT(I)} \leq \rho$.

Nota che risulta sempre $A(I) \geq OPT(I)$ di conseguenza il rapporto di approssimazione è sempre un numero maggiore o uguale ad 1.

- Se A approssima P con fattore 1 allora A è corretto per P perché trova sempre una soluzione ottima.
- Se A approssima P entro un fattore 2 allora A trova sempre una soluzione di costo al più doppio di quello della soluzione ottima.

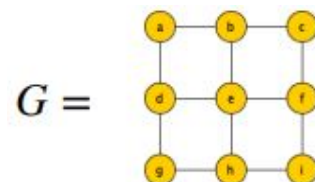
Ovviamente quanto più il rapporto d'approssimazione è vicino a 1 tanto più l'algoritmo d'approssimazione è buono.

Per problemi di massimizzazione dove ad ogni soluzione ammissibile è associato un valore si considera il rapporto inverso vale a dire $\frac{OPT(I)}{A(I)}$.

3.2.1 Copertura tramite nodi minima

Dato un grafo non diretto G una sua copertura tramite nodi è un sottoinsieme S dei suoi nodi tale che tutti gli archi di G hanno almeno un estremo in S . Dato un grafo non diretto G trovare una copertura tramite nodi di minima cardinalità.

Una semplice strategia greedy per il problema potrebbe essere: finché ci sono archi non coperti inserisci in S il nodo che copre il massimo numero di archi da coprire. Questo algoritmo non è corretto. Infatti sul grafo G al primo passo verrebbe inserito in S il nodo e (l'unico a coprire 4 archi) dopodiché tutti i nodi restanti sono



in grado di coprire lo stesso numero di archi e grado. La soluzione prodotta consiste di 5 nodi. Invece la soluzione ottima è formata dai 4 nodi b, d, f, h .

Proviamo a valutare il rapporto d'approssimazione dell'algoritmo greedy visto per il problema del copertura tramite nodi. L'algoritmo greedy che abbiamo proposto non è corretto (abbiamo trovato un'istanza per cui l'algoritmo produce una soluzione con 5 nodi mentre la soluzione ottima ha 4 nodi). Da ciò possiamo dedurre che il fattore di approssimazione dell'algoritmo è almeno $\frac{5}{4} > 1$, ma potrebbe essere peggiore. In effetti per ogni costante R si possono esibire grafi per cui l'algoritmo sbaglia di un fattore superiore ad R .

Lemma 3.2.1 – Rapporto di approssimazione $\Omega(\log l)$ per il copertura tramite nodi

Dimostreremo che per ogni intero l possiamo costruire un grafo G_l su cui l'algoritmo greedy avrà rapporto di approssimazione $\Omega(\log l)$. Il grafo G_l è definito come segue:

- il grafo è strutturato in l livelli che vanno da 1 a l e gli archi del grafo collegano nodi a livello $i > 1$ con nodi a livello 1. Più precisamente: a livello i sono presenti $\frac{l}{i}$ nodi, ciascuno di questi nodi ha grado i e gli $\frac{l}{i} \leq l$ archi vanno a nodi distinti del livello 1.

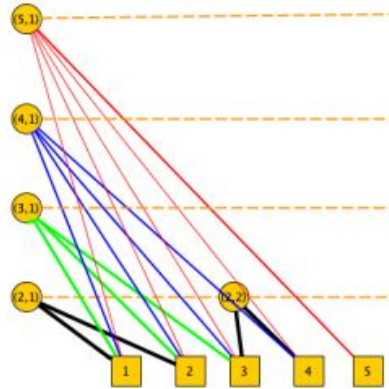


Figura 3.3: Esempio di grafo G_5

Sul grafo G_l l'euristica funziona come segue: il primo nodo ad essere selezionato è il nodo a livello l di grado l (tutti gli altri hanno grado al più $l-1$); verranno poi uno dopo l'altro selezionati tutti i nodi a livello $l-1$ di grado $l-1$ (tutti gli altri hanno grado al più $l-2$); verranno poi uno dopo l'altro selezionati tutti i nodi a livello $l-2$ di grado $l-2$ (tutti gli altri hanno grado al più $l-3$) ... verranno infine selezionati tutti i nodi a livello 2 di grado 2 (tutti gli altri, cioè i nodi a livello 1 hanno grado 1).

Per il costo della soluzione prodotta dall'euristica si ha:

$$c(SOL) = \sum_{i=2}^l \frac{l}{i} \geq \sum_{i=2}^l \left(\frac{l}{i} - 1 \right) \geq l \int_2^{l+1} \frac{1}{x} dx - (l-1) = l \log_e(l+1) - l \log_e 2 - l + 1 \geq \Omega(l \log l)$$

Una possibile soluzione al problema è quella di selezionare gli l nodi a livello 1 (infatti per costruzione tutti gli archi incidono su quegli l nodi). Non è difficile dimostrare che questa è anche la soluzione ottima. Possiamo comunque dire $c(SOL) = O(l)$.

Abbiamo quindi dimostrato che il rapporto di approssimazione dell'euristica è almeno $\frac{c(SOL)}{c(SOL^*)} = \frac{\Omega(l \log l)}{O(l)} = \Omega(\log l)$.

Ovviamente il fatto d'aver dimostrato che per un problema un certo algoritmo d'approssimazione ha un cattivo rapporto d'approssimazione non impedisce che per il problema possano esistere altri algoritmi d'approssimazione con un fattore d'approssimazione costante.

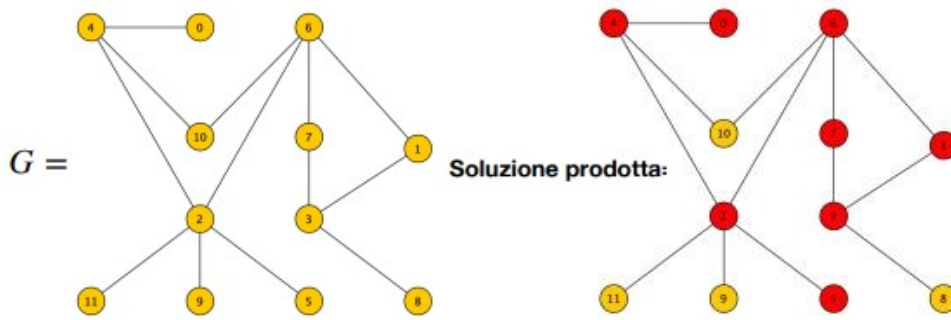
Algoritmo 3.2.1 – Algoritmo per la copertura dei nodi

Consideriamo il seguente algoritmo greedy per la copertura di nodi: considera i vari archi del grafo uno dopo l'altro e ogni volta che ne trovi uno non coperto (vale a dire nessuno dei suoi estremi è in S) aggiungi entrambi gli estremi dell'arco alla soluzione S .

L'algoritmo produce ovviamente una copertura (ogni arco verrà infatti esaminato e se risulterà non coperto verrà coperto da entrambi i lati). La copertura prodotta non è detto sia minima, l'algoritmo ha rapporto d'approssimazione almeno 2 come dimostra il grafo con due soli nodi ed un arco. Dimosteremo che il rapporto d'approssimazione è limitato da 2.

Per rendere efficiente il test sui nodi in S utilizzo un vettore caratteristico $presi$ dove $presi[i]=1$ se il nodo i è in S , 0 altrimenti.

```
def copertura1(G):
    n=len(G)
    E=[(x, y) for x in range(n) for y in G[x] if x<y]
    presi=[0]*n
    Sol=[]
    for a, b in E:
        if presi[a]==presi[b]==0:
            Sol.append(a)
            Sol.append(b)
            presi[a]=presi[b]=1
    return Sol
```



La complessità dell'algoritmo è $O(n + m)$.

Lemma 3.2.2 – Il rapporto d'approssimazione dell'algoritmo greedy è limitato da 2

Siano $e_1, e_2, \dots, e_k$ gli archi di G che vengono trovati non coperti durante l'esecuzione dell'algoritmo greedy.

1. Per come funziona l'algoritmo deduciamo che $A(I) = 2k$.

I k archi non coperti sono tra loro disgiunti (infatti i due estremi di ciascuno di questi archi vengono incontrati per la prima volta quando viene esaminato l'arco) questo significa che in una qualunque delle soluzioni ottime almeno un estremo di ciascuno di questi k archi deve essere presente.

2. Ne deduciamo che $k \leq OTT(I)$.

Dai punti 1) e 2) ricaviamo che $A(I) = 2k \leq 2 * OTT(I)$ da cui segue $\frac{A(I)}{OTT(I)} \leq 2$.

3.2.2 Il problema del massimo taglio

Dato un grafo G non diretto e connesso un taglio di G è un sottoinsieme A dei suoi vertici. Il valore di un taglio è dato dal numero di archi che hanno esattamente un estremo in A . Vogliamo trovare il taglio di valore massimo.

Si propone il seguente algoritmo greedy: parti con due insiemi A e B vuoti. Considera i vertici del grafo G nell'ordine crescente e inseriscili in A o B in base alla regola che segue: ogni nodo i può

contribuire al taglio che si va via via creando apportando potenzialmente nuovi archi (quelli che incidono su di lui e sui nodi già considerati). Sia x il numero di questi archi. Inserisci i in A se così facendo il contributo al taglio è almeno $\frac{x}{2}$, inseriscilo in B altrimenti. Dopo aver inserito ogni nodo nell'insieme opportuno restituisci A .

Viene riportata una possibile implementazione di complessità $O(m)$:

```
def MC(G):
    n=len(G)
    CA=[0]*n
    CB=[0]*n
    A=[]
    for i in range(n):
        if CA[i] >= CB[i]:
            A.append(i)
            for u in G[i]:
                if u>i:
                    CB[u]+=1
        else:
            for u in G[i]:
                if u>i:
                    CA[u]+=1
    return A
```

L'euristica ha un rapporto d'approssimazione limitato da 2.

Sia k il valore della soluzione greedy e k^* quello della soluzione ottima. Assegnamo la responsabilità di ciascun arco al suo estremo di indice massimo. Sia r_i il numero di archi di cui il nodo i è responsabile. Ovviamente vale

$$k^* \leq m = \sum_{i=0}^{n-1} r_i$$

Per la regola con cui viene assegnato il nodo i ad A o a B si ha che il contributo di i al taglio è almeno $\frac{r_i}{2}$. Quindi vale

$$\sum_{i=0}^{n-1} \frac{r_i}{2} \leq k$$

Dalle disequazioni 1) e 2) ricaviamo $k^* \leq \sum_{i=0}^{n-1} r_i = 2 \sum_{i=0}^{n-1} \frac{r_i}{2} \leq 2k$ e quindi $\frac{k^*}{k} \leq 2$.

Il rapporto d'approssimazione non può essere migliorato come mostra l'istanza con un grafo di n nodi con i nodi 0 e 1 di grado $n-1$ e tutti gli altri nodi di grado 2. L'euristica metterebbe il primo nodo in A ed il secondo in B a questo punto tutti i rimanenti nodi contribuirebbero al taglio per esattamente 1 (indipendentemente dall'insieme in cui vengono inseriti). Viene quindi prodotto un taglio di valore $n-1$. Mettendo i primi due nodi del grafo in A e tutti gli altri in B si otterrebbe un taglio di valore $2(n-2)$. Si ha quindi in questo caso

$$\frac{k^*}{k} \geq \frac{2(n-1)}{n-1} - \frac{2}{n-1} = 2 - \frac{2}{n-1}$$

che tende a 2 al crescere di n . Attualmente il miglior algoritmo approssimante per questo problema ha rapporto $\alpha \approx 1.389$.

3.2.3 Il problema dell'impacchettamento (BP)

Il problema dell'impacchettamento è un problema di ottimizzazione che consiste nell'impacchettare in scatole con capacità c un insieme di n oggetti ciascuno di dimensione intera al più c , minimizzando il numero di contenitori utilizzati. Il problema richiede dunque di trovare la distribuzione degli oggetti all'interno dei contenitori in modo da usare il minor numero possibile di contenitori, rispettando le dimensioni dei singoli oggetti e dei contenitori stessi.

Il problema è notoriamente difficile e per ottenere soluzioni soddisfacenti in tempi ragionevoli si ricorre ad algoritmi di approssimazione. La più semplice euristica è di tipo greedy ed è nota come Next Fit, lavora come segue: si considerano gli oggetti da impacchettare in sequenza. All'inizio abbiamo un unico contenitore contenente il primo oggetto. Per ogni altro oggetto, se il contenitore

ha spazio per contenerlo allora l'oggetto viene inserito nel contenitore, in caso contrario il contenitore viene chiuso e viene inaugurato un nuovo contenitore.

La procedura prende in input la lista dei pesi degli oggetti e la capacità delle scatole e restituisce il numero di scatole necessarie. La complessità è $\Theta(n)$:

```
def NF(P, c):
    n=len(P)
    Sol=[[0]]
    spazio=c-P[0]
    for i in range(1, n):
        if spazio-P[i] >= 0:
            Sol[-1].append(i)
            spazio-=P[i]
        else:
            Sol.append([i])
            spazio=c-P[i]
    return len(Sol), Sol
```

L'euristica Next Fit ha un rapporto d'approssimazione limitato da 2.

Sia k il numero di contenitori utilizzato dall'euristica e k^* quello della soluzione ottima inoltre indichiamo con s la somma delle dimensioni degli n oggetti. Deve ovviamente aversi

$$s \leq c * k^*$$

Nota che di due contenitori inaugurati in sequenza almeno uno è pieno per più della metà infatti se il primo dei due contenitori è pieno per meno della metà allora l'oggetto che ha portato all'inaugurazione del contenitore successivo deve avere una dimensione superiore alla metà di c . Questo significa che tutti i contenitori tranne l'ultimo sono pieni per più della metà. Abbiamo dunque

$$(k-1) \frac{c}{2} < s$$

Mettendo insieme le due disuguaglianze abbiamo $(k-1) \frac{c}{2} < c * k^*$ da cui ricaviamo $k-1 < 2k^*$ e, poiché k e k^* sono interi deve aversi $k \leq 2k^*$ da cui finalmente deduciamo $\frac{k}{k^*} \leq 2$.

Il rapporto stabilito dal teorema non può essere migliorato come mostra la seguente istanza: considera una lista di $2c$ oggetti di peso c e $2c$ oggetti di peso 1 da dover inserire in scatole di dimensione $2c$. La soluzione ottima consiste nell'inserire due oggetti di dimensione c in ciascuna scatola e poi utilizzare due scatole per inserire i $2c$ oggetti di dimensione 1. Il numero minimo di scatole k^* necessarie è dunque $c + 2$ scatole. Tuttavia se nella lista gli oggetti delle due diverse dimensioni si alternano l'algoritmo NF utilizzerà $2c$ scatole inserendo in ogni scatola un oggetto di dimensione c ed un oggetto di dimensione 1 producendo una soluzione che utilizza un numero di scatole k pari a $2c$. Per questa istanza si ha dunque rapporto

$$\frac{k}{k^*} = \frac{2c}{c+2} = \frac{2(c+2)}{c+2} - \frac{4}{c+2} = 2 - \frac{4}{c+2}$$

il rapporto al crescere di c tende a 2.

Nell'euristica precedente si considera un oggetto per volta senza conoscere il peso degli oggetti futuri (algoritmo online). Un problema con questi algoritmi è quello di avere oggetti pesanti tardi nella sequenza. Nell'euristica che segue si assume di avere di fronte tutti gli oggetti (algoritmo offline). L'idea dell'euristica FFD è quella di ordinare gli oggetti per peso decrescente e poi procedere con il loro impacchettamento con la seguente strategia: passa in rassegna le eventuali scatole già inaugurate, sistema l'oggetto nella prima che si incontra in grado di contenerlo, se nessuna scatola è in grado di farlo inaugura una nuova scatola. Le scatole vengono dunque sigillate solo quando tutti gli oggetti saranno stati inscatolati.

Considera scatole di dimensione $c = 10$ e lista di oggetti $lista = [5, 4, 3, 2, 2, 2, 2, 2]$. Gli oggetti possono essere partizionati in due scatole $(5, 3, 2)$ e $(4, 2, 2, 2, 2)$ ma il metodo FFD richiede tre scatole poiché piazzerebbe gli oggetti di peso 5 e 4 insieme.

Si riporta una possibile implementazione di complessità $O(n^2)$.

```
def FFD(P, c):
    lista=[(P[i], i) for i in range(len(P))]
```

```

lista.sort(reverse= True)
Sol=[]
C=[]
for spazio, i in lista:
    s=0
    while s<len(C):
        if C[s]+spazio <= c:
            C[s]+=spazio
            Sol[s].append(i)
            break
        s+=1
    if s==len(C):
        Sol.append([i])
        C.append(spazio)
return len(Sol), Sol

```

L'euristica First Fit Decreasing ha rapporto d'approssimazione $\frac{3}{2} + \frac{1}{k^*}$.

Sia k il numero di contenitori utilizzato dall'euristica e k^* quello della soluzione ottima inoltre indichiamo con s la somma delle dimensioni degli n oggetti.

Dividiamo gli oggetti in 4 categorie:

- A = gli oggetti di dimensione $(\frac{2c}{3}, c)$;
- B = gli oggetti di dimensione $(\frac{c}{2}, \frac{2c}{3})$;
- C = gli oggetti di dimensione $(\frac{c}{3}, \frac{c}{2})$;
- D = gli oggetti di dimensione $(0, \frac{c}{3})$.

Cominciamo col far vedere che se nella soluzione greedy non compaiono oggetti di tipo D allora l'euristica non sbaglia e produce la soluzione ottima. Nota che gli oggetti di tipo A devono necessariamente comparire da soli in ciascuna scatola mentre in altre scatole possono esserci coppie di oggetti ma solo se i due oggetti sono uno di tipo B e l'altro di tipo C o entrambi di tipo C . Poiché gli oggetti sono ordinati per dimensione decrescente e l'euristica greedy li sistemerà nelle scatole considerandoli in quell'ordine, non è difficile vedere che l'ottimo e la soluzione greedy utilizzeranno lo stesso numero di scatole.

Considera ora il caso in cui sono presenti oggetti di tipo D ma nella soluzione greedy non c'è alcuna scatola che contiene solo oggetti di questo tipo. In questo caso la presenza degli oggetti di tipo D non influenza il numero di scatole e quindi la soluzione prodotta è quella richiesta da tutti gli altri oggetti e sappiamo già che per quegli oggetti il greedy produce l'impacchettamento l'ottimo.

Resta da dimostrare cosa accade quando la soluzione greedy presenta almeno una scatola con soli oggetti di tipo D . L'ultima scatola utilizzata deve essere certamente di questo tipo ma questo significa che le scatole precedenti saranno piene tutte piene per almeno $\frac{2c}{3}$. Sia k il numero di contenitori utilizzato dall'euristica e k^* quello della soluzione ottima inoltre indichiamo con s la somma delle dimensioni degli n oggetti. Abbiamo quindi

$$(k-1)\frac{2c}{3} < s$$

inoltre

$$s < c * k^*$$

Mettendo insieme le due disuguaglianze otteniamo $(k-1)\frac{2}{3} < k^*$ da cui $\frac{k-1}{k^*} < \frac{3}{2}$ e infine $\frac{k}{k^*} < \frac{3}{2} + \frac{1}{k^*}$. Con una prova molto più complicata si dimostra che il rapporto è limitato da $\frac{11}{9} + \frac{4}{k^*}$.

Capitolo 4

Divide et impera

Un programma sviluppato secondo questa tecnica è sostanzialmente diviso in tre parti:

- *Divide*: in questa parte si procede alla suddivisione dei problemi in problemi di dimensione minore;
- *Impera*: nella seconda parte i problemi vengono risolti in modo ricorsivo. Quando i sottoproblemi arrivano ad avere una dimensione sufficientemente piccola, essi vengono risolti direttamente tramite il caso base;
- *Combina*: l'ultima fase del paradigma prevede di ricombinare l'output ottenuto dalle precedenti chiamate ricorsive al fine di ottenere il risultato finale.

Quali sono i vantaggi offerti da questa tecnica? Di solito, quando la tecnica è applicabile, non è difficile vedere come applicarla. Questo perché, tipicamente, è facile intuire come si potrebbe spezzare l'istanza. Inoltre, è abbastanza facile, una volta che un algoritmo di Divide et Impera è stato ideato, analizzarne sia la correttezza che la complessità. In particolare, la complessità di un algoritmo Divide et Impera, essendo un algoritmo ricorsivo, può essere determinata risolvendo un'opportuna relazione di ricorrenza relativa alla funzione $T(n)$ che dà il massimo tempo di calcolo dell'algoritmo su istanze di dimensione n .

4.1 Algoritmi divide et impera

4.1.1 Problema della selezione

Data una lista A di interi distinti, il rango di un elemento in A è il numero di elementi che sono minori o uguali ad x in A . Ad esempio il minimo in A è l'elemento di rango 1, il mediano in A è l'elemento di rango $\frac{n}{2}$ e il massimo in A è l'elemento di rango n . Abbiamo una lista A di n numeri distinti ed un intero k , $1 \leq k \leq n$. Vogliamo l'elemento di rango k in A .

Un semplice algoritmo di complessità $O(n \log n)$ è il seguente:

```
def selezional(A, k):  
    A.sort()  
    return A[k-1]
```

Se $k = 1$ o $k = n$ il problema si riduce alla ricerca del minimo o del massimo e questi casi sono risolvibili in tempo $\Theta(n)$. Utilizzando la tecnica del divide et impera, vedremo che, anche nel caso generale, il problema può risolversi in $\Theta(n)$. In altre parole dimostreremo che il problema della selezione è computazionalmente più semplice di quello dell'ordinamento (che richiede tempo $\Omega(n \log n)$).

Approccio basato sul divide et impera

L'approccio basato sul divide et impera segue questi passi:

1. scegli nella lista A un elemento x (che chiameremo perno);

2. a partire da A costruisci due liste A_1 ed A_2 , la prima contenente gli elementi di A minori di x e la seconda gli elementi di A maggiori di x .
3. dove si trova l'elemento di rango k ? Se $|A_1| \geq k$ allora l'elemento di rango k è nel vettore A_1 ; se $|A_1| = k - 1$ allora l'elemento di rango k è proprio il perno x ; se $|A_1| < k - 1$ allora l'elemento di rango k è in A_2 è l'elemento di rango $k - |A_1| - 1$ in A_2 .

Nota che dopo aver costruito dalla lista A di elementi le due liste A_1 e A_2 , grazie al test sulla cardinalità di A_1 il problema della selezione dell'elemento di rango k o risulta risolto o viene ricondotto alla selezione di un elemento in una lista con meno di n elementi.

```
def selezione2(A, k):
    if len(A)==1:
        return A[0]
    perno=A[0]
    A1, A2= [], []
    for x in A:
        if x < perno:
            A1.append(x)
        elif x > perno:
            A2.append(x)
    if len(A1) >= k:
        return selezione2(A1, k)
    elif len(A1) == k-1:
        return perno
    return selezione2(A2, k-len(A1)-1)
```

La procedura che tripartisce la lista in A_1 , p e A_2 può restituire una partizione massimamente sbilanciata in cui si ha ad esempio $|A_1| = 0$ e $|A_2| = n - 1$, questo accade quando il perno p risulta l'elemento minimo nella lista.

Qualora questo evento sfortunato si ripettesse sistematicamente nel corso delle varie partizioni eseguite dall'algoritmo (questo può accadere quando cerco il massimo in una lista ordinata), allora la complessità dell'algoritmo al caso peggioro viene catturata dalla seguente equazione:

$$T(n) = T(n - 1) + \Theta(n)$$

che risolta dà

$$T(n) = \Theta(n^2)$$

In generale la complessità superiore alla procedura è catturata dalla ricorrenza

$$T(n) = T(m) + \Theta(n)$$

dove $m = \max\{|A_1|, |A_2|\}$.

Se avessimo una regola di scelta del perno p in grado di garantire una partizione perfettamente bilanciata (vale a dire $m = \max\{|A_1|, |A_2|\} \approx n/2$) allora per la complessità $T(n)$ dell'algoritmo avremmo

$$T(n) = T\left(\frac{n}{2}\right) + \Theta(n)$$

che produce $T(n) = \Theta(n)$.

Chiedere che la partizione sia perfettamente bilanciata è forse chiedere troppo, potremmo allora accontentarci di chiedere che la scelta del perno p garantisca partizioni non troppo sbilanciate come ad esempio quelle per cui $m \approx 3n/4$. In questo caso si ha

$$T(n) \leq T\left(\frac{3}{4}n\right) + \Theta(n)$$

che risolta dà ancora $T(n) = \Theta(n)$. In generale finché m è una frazione di n (anche piuttosto vicina ad n come ad esempio $\frac{99}{100}n$) la ricorrenza da sempre $T(n) = \Theta(n)$.

Scegliamo il perno p a caso in modo equiprobabile tra gli elementi della lista. Anche se la scelta "casuale" non produce necessariamente una partizione bilanciata, quanto visto ci fa intuire che la complessità rimane lineare in n .


```

from random import randint

def selezione2R(A, k)
    if len(A)==1:
        return A[0]
    perno=A[randint(0, len(A)-1)]
    A1, A2 = [], []
    for x in A:
        if x < perno:
            A1.append(x)
        elif x > perno:
            A2.append(x)
    if len(A1)>=k:
        return selezione2R(A1, k)
    elif len(A1)==k-1:
        return perno
    return selezione2R(A2, k-len(A1)-1)

```

Analisi formale del caso medio

Più formalmente, sotto l'assunzione che il pivot possa essere con uguale probabilità un elemento di qualsiasi rango tra 1 e n la ricorrenza da studiare per il caso medio è

$$T(n) \leq \frac{1}{n} \sum_{k=1}^n T(\max(T(k-1), T(n-k))) + \Theta(n) = \frac{1}{n} \sum_{k=1}^{n-1} 2T(k) + \Theta(n) \text{ per } n \geq 2$$

possiamo dimostrare che per questa ricorrenza vale $T(n) = O(n)$ col metodo di sostituzione. Sia $T(n) \leq \frac{1}{n} \sum_{k=\frac{n}{2}}^{n-1} 2T(k) + a * n$ per $n \geq 3$, $T(x) \leq b$ altrimenti e dimostriamo $T(n) \leq cn$ per una $c > 0$ costante. Per $n \leq 3$ abbiamo $T(n) \leq b \leq 3c$ che vera ad esempio per $c \geq b$. Per ipotesi induttiva abbiamo $T(n) \leq \frac{2c}{n} \sum_{k=\frac{n}{2}}^{n-1} k + a * n$ da cui ricaviamo

$$\begin{aligned}
 T(n) &\leq \frac{2c}{n} \left(\sum_{k=1}^{n-1} k - \sum_{k=1}^{\frac{n}{2}-1} k \right) + a * n \leq \frac{2c}{n} \left(\frac{n(n-1)}{2} - \frac{(n/2-1)(n/2-2)}{2} \right) + a * n \\
 &\leq \frac{c}{n} \left(\frac{3^2}{4} + \frac{n}{2} - 2 \right) + a * n \leq \frac{3cn}{4} + \frac{c}{n} + a * n = cn - \left(\frac{cn}{4} - \frac{c}{2} - a * n \right) \leq cn
 \end{aligned}$$

dove l'ultima disequaglianza segue prendendo c in modo che $(\frac{cn}{4} - \frac{c}{2} - a * n) \geq 0$. Basta ad esempio prendere $c \geq 8a$.

L'analisi rigorosa appena fatta dimostra che, se la scelta del perno avviene in modo equiprobabile a caso tra i vari elementi della lista A , il tempo di calcolo dell'algoritmo risulta con alta probabilità lineare in n . Il tempo dell'algoritmo `selezione2R` è dunque $O(n)$ con alta probabilità.

Ovviamente nel caso peggiore, quando nelle varie partizioni che si succedono nell'iterazione dell'algoritmo si verifica che il perno scelto a caso risulta sempre vicino al massimo o al minimo della lista, la complessità dell'algoritmo rimane $O(n^2)$. Però questo accade con probabilità molto piccola.

Algoritmo deterministico in $O(n)$

Sappiamo che riuscire a selezionare un perno in grado di garantire che nessuna delle due sottoliste A_1 e A_2 abbia più di cn elementi per una qualche costante $0 < c < 1$, avrebbe come conseguenza una complessità di calcolo $O(n)$.

Descriviamo ora un metodo (noto come il mediano dei mediani) per selezionare un perno p che garantisce di produrre sottoliste A_1 ed A_2 ciascuna delle quali ha al più $\frac{3}{4}n$ elementi:

1. Dividi gli a elementi di A in gruppi di 5 elementi ciascuno tranne al più l'ultimo che potrebbe averne di meno e considera i primi $\frac{n}{5}$ gruppi ottenuti (tutti di 5 elementi ciascuno).
2. Trova il mediano in ciascuno di questi $\frac{n}{5}$ gruppi.

3. Trova il mediano p dei $\frac{n}{5}$ mediani.
4. Usa p come perno di A .

Se la lista A contiene almeno 120 elementi e il perno con cui partizionarla viene scelto in base alla regola appena descritta si può esser sicuri che la dimensione di ciascuna delle due sottoliste A_1 e A_2 ottenute sarà limitata da $\frac{3}{4}n$.

Il perno scelto p ha la proprietà di trovarsi in posizione $\frac{n}{10}$ nella lista degli $\frac{n}{5}$ mediani selezionati in A . Ci sono dunque $\frac{n}{10} - 1$ mediani di valore inferiore a p e $\frac{n}{5} - \frac{n}{10}$ mediani di valore superiore a p .

- *Prova che $|A_2| < \frac{3}{4}n$.* Considera i $\frac{n}{10} - 1$ mediani di valore inferiore a p . Ognuno di questi mediani appartiene ad un gruppo di 5 elementi in n . Ci sono dunque in A altri 2 elementi inferiori a p per ogni mediano. In totale abbiamo $3(\frac{n}{10} - 1) \geq 3\frac{n}{10} - 3$ elementi di A che finiranno in A_1 .

Abbiamo dunque $|A_2| \leq n - (3\frac{n}{10} - 3) = \frac{7}{10}n + 3 \leq \frac{3}{4}n$ (dove l'ultima disequaglianza segue dal fatto che $n \geq 120$).

- *Prova che $|A_1| < \frac{3}{4}n$.* Ci sono $\frac{n}{5} - \frac{n}{10} \geq (\frac{n}{5} - 1) - (\frac{n}{10} + 1) = \frac{n}{10} - 2$ mediani di valore superiore a p . Ognuno di questi mediani appartiene ad un gruppo di 5 elementi in A . Ci sono dunque in A altri 2 elementi superiori a p per ogni mediano. In totale abbiamo almeno $3\frac{n}{10} - 6$ elementi di A che finiranno in A_2 .

Abbiamo dunque $|A_1| \leq n - (3\frac{n}{10} - 6) = \frac{7}{10}n + 6 \leq \frac{3}{4}n$ (dove ultima disequaglianza segue dal fatto che $n \geq 120$).

```
from math import ceil
```

```
def selezione(A, k):
    if len(A) <= 120:
        A.sort()
        return A[k-1]
    B=[sorted(A[5*i: 5*i+5])[2] for i in range(len(A)/5)]
    perno=selezione(B, ceil(len(A)/10))
    A1, A2 = [], []
    for x in A:
        if x < perno:
            A1.append(x)
        elif x > perno:
            A2.append(x)
    if len(A1) >= k:
        return selezione(A1, k)
    elif len(A1) == k-1:
        return perno
    return selezione(A2, k-len(A1)-1)
```

Nota che ordinare 120 elementi richiede tempo $O(1)$, ordinare una lista di n elementi in gruppetti da 5 richiede tempo $\Theta(n)$, selezionare i mediani dei mediani di gruppi da 5 da una lista in cui gli elementi sono stati ordinati in gruppetti da 5 richiede tempo $\Theta(n)$.

Sappiamo che per $n \geq 120$ risulta $lista1 \leq \frac{3}{4}n$ e $lista2 \leq \frac{3}{4}n$. Dunque per la complessità $T(n)$ dell'algoritmo si ha

$$T(n) \leq \begin{cases} O(1) & \text{se } n \leq 120 \\ T(\frac{n}{5}) + T(\frac{3n}{4}) + \Theta(n) & \text{altrimenti} \end{cases}$$

Notiamo che la ricorrenza è del tipo

$$T(n) = T(\alpha * n) + T(\beta * n) + \Theta(n) \text{ con } \alpha + \beta = 1/5 + 3/4 = 19/20 < 1$$

Se $T(n) = T(\alpha * n) + T(\beta * n) + cn$ e $\alpha + \beta < 1$, allora $T(n) = \Theta(n)$. Il fatto che $\alpha + \beta$ sia inferiore ad 1 gioca un ruolo fondamentale nella prova. Consideriamo l'albero delle chiamate ricorsive generato dalla ricorrenza e analizziamone il costo per livelli. Al primo livello abbiamo un costo $(\alpha + \beta) * n$, al

secondo un costo $(\alpha + \beta)^2 * n$ al terzo un costo $(\alpha + \beta)^3 * n$ e così via. Il tempo di esecuzione totale è la somma dei contributi dei vari livelli:

$$T(n) < cn + c(\alpha + \beta)n + c(\alpha + \beta)^2n \dots = cn \sum_{i=0}^{\infty} (\alpha + \beta)^i = cn \frac{1}{1 - (\alpha + \beta)} = \Theta(n)$$

dove nel calcolare la serie abbiamo sfruttato il fatto che $\alpha + \beta < 1$ e la serie geometrica $\sum_{i=0}^{\infty} x^i$, con $x < 1$, converge a $\frac{1}{1-x}$.

Abbiamo quindi dimostrato che il problema della selezione può essere risolto in tempo lineare. Abbiamo infatti un algoritmo che risolve il problema in $O(n)$ al caso pessimo. Tuttavia, a causa delle grandi costanti moltiplicative nascoste dall' $O(n)$, nella pratica l'algoritmo randomizzato che ha tempo $O(n)$ con alta probabilità si comporta molto meglio.

4.1.2 Potenza di un numero

Dati due interi a ed n progettare un algoritmo che calcoli a^n in tempo $O(\log n)$ (le uniche operazioni aritmetiche permesse sono $+$, $$ e $/$).*

Un semplice algoritmo che calcola a^n utilizzando $n - 1$ prodotti è il seguente:

```
def pow(a, n)
    s=1
    for _ in range(n):
        s*=a
    return s
```

Per applicare il divide et impera osserva che $a^n = a^{\frac{n}{2}} * a^{\frac{n}{2}}$. Ad esempio: $a^{15} = a^7 * a^8$ e $a^{16} = a^8 * a^8$. Inoltre

$$a^{\frac{n}{2}} = \begin{cases} a^{\frac{n}{2}} & \text{se } n \text{ è pari} \\ a^{\frac{n}{2}} * a & \text{se } n \text{ è dispari} \end{cases}$$

quindi

$$a^n = \begin{cases} a^{\frac{n}{2}} * a^{\frac{n}{2}} & \text{se } n \text{ è pari} \\ a^{\frac{n}{2}} * a^{\frac{n}{2}} * a & \text{se } n \text{ è dispari} \end{cases}$$

Il sottoproblema da risolvere è dunque $a^{\frac{n}{2}}$.

```
def pow(a, n):
    if n==0:
        return 1
    x = pow(a, n//2)
    if n%2:
        return x*x*a
    return x*x
```

La ricorrenza da studiare per la complessità dell'algoritmo (e per il numero di prodotti richiesti) è

$$T(n) = T\left(\frac{n}{2}\right) + O(1)$$

che risolta dà $\Theta(\log n)$.

4.1.3 Valore massimo nelle sottoliste

Data una lista di interi A , una sottolista è una sequenza consecutiva di valori della lista. Il valore della sottolista è dato dalla somma degli elementi che comprende. Data una lista di interi vogliamo trovare il valore massimo delle sue sottoliste.

E' chiaro che il problema è banale se i valori della lista sono tutti non negativi o tutti non positivi. Il problema risulta interessante se la lista contiene valori sia positivi che negativi.

Un algoritmo diretto per il problema, basato sulla ricerca esaustiva, consiste nel considerare le somme di tutte le possibili sottoliste e prenderne il massimo:

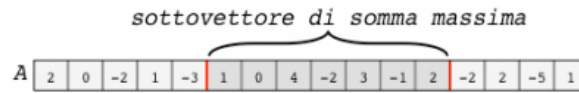


Figura 4.1: Sottolista che contiene valori positivi e negativi

```
def es(A):
    n=len(A)
    val=A[0]
    for i in range(n):
        for j in range(i, n):
            val=max(sum(A[i : j + 1]), val)
    return val
```

L'algoritmo è molto inefficiente: la complessità è $O(n^3)$. Possiamo abbassare la complessità osservando che la somma di una sottolista $[i \dots j]$ è uguale alla somma della sottolista $[i \dots j-1]$ più $A[j]$.

```
def es(A):
    n=len(A)
    val=A[0]
    for i in range(n):
        s=0
        for j in range(i, n):
            s+=A[j]
            val=max(s, val)
    return val
```

L'algoritmo ha ora complessità $\Theta(n^2)$. Possiamo fare di meglio? Le sottoliste possibili sono

$$\frac{n(n-1)}{2} + n = \frac{n(n+1)}{2} = \Theta(n^2)$$

Ma è realmente necessario esaminarle tutte per risolvere il problema? Proviamo ad applicare la tecnica del Divide et Impera. La prima cosa che viene in mente è di dividere a metà la lista, trovare il massimo in ciascuna delle due metà e poi calcolare il massimo delle somme delle sottoliste a cavallo delle due metà. Alla fine restituiamo il massimo dei tre massimi ottenuti.

Sicuramente l'algoritmo è corretto perché considera tutte le possibili sottoliste. Infatti, una sottolista o è contenuta interamente nella prima metà della lista o interamente nella seconda o è cavallo delle due metà.

Qual'è la sua complessità di questo algoritmo? La risposta dipende da come effettuiamo il calcolo delle somme delle sottoliste a cavallo. Se lo facciamo in modo diretto cioè, calcoliamo la somma della sottolista $[i \dots j]$ per ogni coppia di indici i e j con i nella prima metà e j nella seconda, questo richiede tempo $\Theta(n^2)$ (dato che il numero di sottoliste a cavallo è $\frac{n}{2} \frac{n}{2} = \Theta(n^2)$).

Ovviamente questo implica che la complessità $T(n)$ del nostro algoritmo Divide et Impera è determinata dalla relazione $T(n) = 2T(\frac{n}{2}) + \Theta(n^2)$ che risolta dà $T(n) = \Theta(n^2)$.

Possiamo migliorare il calcolo del massimo delle somme delle sottoliste a cavallo delle due metà? Osserviamo che ogni sottolista a cavallo è formato da un suffisso della prima metà della lista ed un prefisso della seconda metà della lista.

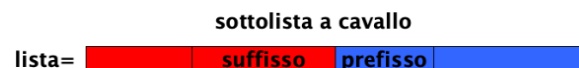


Figura 4.2: Sottolista formata da prefisso e suffisso

La sottolista a cavallo di somma massima ha come valore il valore del suffisso di somma massima della prima metà della lista più il valore del prefisso di somma massima della seconda metà della lista.

```
def es(A):
    if len(A)==1:
```

```

    return A[0]
m=len(A)//2
rs=es(A[:m])
rd=es(A[m:])
pmax=x=A[m-1]
for i in range(m-2,-1,-1):
    x+=A[i]
    pmax=max(pmax, x)
smax=x=A[m]
for i in range(m+1, len(A)):
    x+=A[i]
    smax=max(smax, x)
return max(rs, rd, smax+pmax)

```

Il calcolo di $A[:m]$ e di $A[m:]$ costa $\Theta(n)$. Il calcolo del valore del suffisso massimo della prima metà di A costa $\Theta(n)$. Il calcolo del valore del prefisso massimo della seconda metà di A costa $\Theta(n)$. La ricorrenza che cattura la complessità $T(n)$ della funzione è

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$$

che risolta dà $\Theta(n \log n)$.

Si può fare di meglio? La risposta è sì. Faremo ora vedere una migliore implementazione dell'algoritmo basato sul divide et impera che presenterà una complessità $\Theta(n)$. Nota che l'algoritmo $\Theta(n)$ sarà ottimo per il problema (il lower bound $\Omega(n)$ segue dalla banale osservazione che per risolvere il problema non si può fare a meno di scorrere tutti gli n elementi della lista).

1. Non possiamo permetterci di pagare $\Theta(n)$ in fase di invocazione della funzione sui due sottoproblemi (per la creazione di $A[:m]$ e $A[m:]$) e per ovviare a questo problema indicheremo i sottoproblemi di A con il loro punto di inizio i ed il loro punto di fine j .
2. Non possiamo permetterci di pagare $\Theta(n)$ in fase di combinazione delle risposte fornite dai due sottoproblemi. Per ovviare a questo problema faremo sì che ogni sottoproblema restituisca ulteriore informazione che aiuti nella combinazione.

Facciamo in modo che la procedura ricorsiva per A con la soluzione sol al problema restituisca anche altri tre valori (che semplificheranno poi il costo della combinazione):

- tot : la somma dei valori di A .
- pm : il valore massimo per i prefissi di A .
- sm : il valore massimo per i suffissi di A .

dai valori $solS$, $totS$, pmS e smS restituiti dal primo sottoproblema e dai valori $solD$, $totD$, pmD e smD restituiti dal secondo problema è possibile calcolare in $O(1)$:

- $sol = \max\{solS, solD, smS + pmD\}$
- $tot = totS + totD$
- $pm = \max\{pmS, totS + pmD\}$
- $sm = \max\{smD, smS + totD\}$

$A =$

prefS		suffS	prefD	suffD
-------	--	-------	-------	-------

Figura 4.3: Nuovi valori nella sottolista

```

def es(A, i, j):
    if i==j:
        return A[i], A[i], A[i], A[i]
    m=(i+j)//2

```

```

sols , tots , pms, sms = es(A, i , m)
sold , totd , pmd, smd = es(A, m+1, j)
sol = max(sols , sold , sms + pmd)
tot = tots + totd
pm = max(pms, tots + pmd)
sm = max(smd, totd + sms)
return sol , tot , pm, sm

```

La relazione di ricorrenza per il tempo di calcolo di questa implementazione è

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(1)$$

che risolta dà $\Theta(n)$.

4.1.4 Sottostringhe binarie

Data una stringa binaria S di n bit progettare un algoritmo che calcoli il numero di sottostringhe di S che cominciano con 0 terminano con 1. Progettare due algoritmi basati sulla tecnica del Divide et Impera, uno a complessità $\Theta(n \log n)$ e l'altro a complessità $\Theta(n)$.

Ad esempio: per $S = '0011'$ restituisce 4 (infatti le sottostringhe sono 001_, 0011, _01_, _011). Per $S = '0101'$ restituisce 3 (infatti le sottostringhe sono 01_, 0101, _01). Per $S = '1100'$ restituisce 0.

Divido la stringa in due sottostringhe di lunghezza circa $\frac{n}{2}$. Risolvo il problema in ciascuna delle due sottostringhe ricevendo come soluzione ts e td . Restituisco il valore $ts + td + c$ dove c è il numero di sottostringhe che iniziano nella sottostringa a destra e terminano nella sottostringa a sinistra. Per calcolare c basta contare il numero z degli zeri nella sottostringa di sinistra ed il numero u degli uni nella sottostringa a destra (vale infatti $c = z * u$).

```

def es(S):
    if len(S)==1:
        return 0
    m = len(S)//2
    ts = es(S[:m])
    td = es(S[m:])
    return ts+td+S[:m].count('0')*S[m:].count('1')

```

La relazione di ricorrenza per il tempo di calcolo di questa implementazione è:

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$$

che risolta dà $\Theta(n \log n)$.

Facciamo in modo che la funzione ricorsiva sulla stringa S oltre a restituire il numero t delle sottosequenze che cominciano con zero e finiscono con uno restituisca anche il numero z dei suoi zeri e il numero u dei suoi uni. Con queste informazioni da parte delle due chiamate ricorsive la funzione è in grado di calcolare in $\Theta(1)$ il numero di sottostringhe che cominciano nella sottostringa di sinistra e finiscono nella sottostringa di destra.

Sapendo infatti zs (il numero di zeri della sottostringa di sinistra) e ud (il numero di uni della sottostringa di destra) il numero di sottosequenze che iniziano alla sinistra del taglio e terminano alla destra del taglio sono $zs * ud$.

```

def es(S):
    if len(S)==1:
        if S[0]=='0':
            return (1,0,0)
        else:
            return (0,1,0)
    m = len(S)//2
    zs, us, ts = es(S[:m])
    zd, ud, td = es(S[m:])
    return zs+zd, us+ud, ts+td+zs*ud

```

La relazione di ricorrenza per il tempo di calcolo di questa implementazione è ancora:

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$$

a causa del costo $\Theta(n)$ necessario per la creazione delle due istanze da invocare. Il problema è facilmente risolvibile specificando le sottoliste da passare tramite indici di inizio e fine.

```
def es(S, i, j):
    if i==j:
        if S[i]=='0':
            return (1,0,0)
        else:
            return (0,1,0)
    m = (i+j)//2
    zs, us, ts = es(S, i, m)
    zd, ud, td = es(S, m+1, j)
    return zs+zd, us+ud, ts+td+zs*ud
```

La relazione di ricorrenza per il tempo di calcolo di questa implementazione è:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(1)$$

che risolta dà $\Theta(n)$.

Capitolo 5

Programmazione dinamica

Sappiamo che gli algoritmi basati sulla tecnica del Divide et Impera seguono i 3 punti:

- Dividi il problema in sottoproblemi di taglia inferiore.
- Risolvi (ricorsivamente) i sottoproblemi di taglia inferiore.
- Combina le soluzioni dei sottoproblemi in una soluzione del problema originale.

Negli esempi finora visti i sottoproblemi che si ottenevano dalla applicazione del passo 1 erano tutti diversi, pertanto ciascuno di essi veniva individualmente risolto dalla relativa chiamata ricorsiva del passo 2. In molte situazioni i sottoproblemi ottenuti al passo 1 possono risultare uguali. In tal caso, l'algoritmo basato sulla tecnica del Divide et Impera risolve lo stesso problema più volte svolgendo lavoro inutile.

In termini generali la programmazione dinamica, al pari del divide et impera può essere vista come un metodo che risolve un problema risolvendo problemi di dimensione più piccola e grazie all'utilizzo di tabelle permette di non ricalcolare più volte lo stesso sottoproblema (memorizzazione). I sottoproblemi possono essere risolti a partire dal più grande al più piccolo (approccio top-down) tipico delle versioni ricorsive o dal più piccolo al più grande (approccio bottom-up) tipico delle versioni iterative.

La tabella che riporta le soluzioni di tutti i sottoproblemi è stata fin'ora usata solamente per calcolare il valore della soluzione. Può essere usata anche per ritrovare la soluzione. Come vedremo questa è una proprietà generale degli algoritmi di programmazione dinamica: in un primo momento ci si può concentrare sul calcolo del valore della soluzione e successivamente la tabella utilizzata per ottenere questo valore potrà essere usata per ritrovare la soluzione cui quel valore corrisponde. L'idea è di percorrere all'indietro la tabella, a partire dall'elemento che rappresenta il valore del problema, di elemento in elemento seguendo a ritroso le "decisioni" che hanno portato a determinare quel valore.

5.1 Algoritmi di programmazione dinamica

5.1.1 Sequenza dei numeri di Fibonacci

La sequenza $f_0, f_1, f_2, \dots$ dei numeri di Fibonacci è definita dall'equazione di ricorrenza:

$$f_i = f_{i-1} + f_{i-2} \text{ con } f_0 = f_1 = 1$$

Ad esempio i primi termini della sequenza sono 1, 1, 2, 3, 5, 8, 13,

Dato un intero n progettare un algoritmo che calcola f_n .

Il primo algoritmo che viene in mente per risolvere il problema è basato sul divide et impera e sfrutta la definizione stessa di numero di Fibonacci.

```
def fib(n):  
    if n <= 1:
```

```

    return 1
a=fib (n-1)
b=fib (n-2)
return a+b

```

La relazione di ricorrenza per il tempo di calcolo dell'algoritmo è:

$$T(n) = T(n-1) + T(n-2) + O(1)$$

ovviamente $T(n) \geq 2T(n-2) + O(1)$ e questa ricorrenza può essere facilmente risolta (ad esempio col metodo iterativo) e otteniamo $T(n) = \Omega(2^{\frac{n}{2}})$. Il motivo di tale inefficienza sta nel fatto che il programma `fib` viene chiamato sullo stesso input molte volte e ciò è chiaramente ridondante.

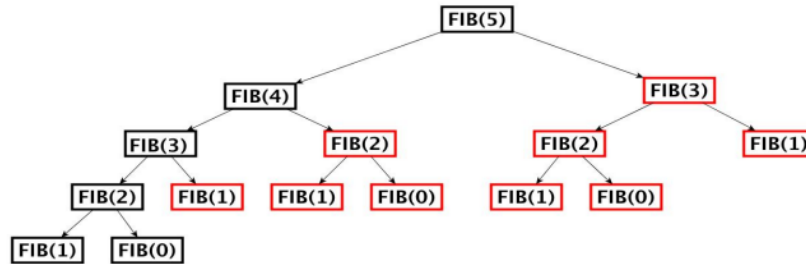


Figura 5.1: Chiamate ricorsive per `fib(5)`

Nel corso della scomposizione in sottoproblemi, stesse quantità vengono calcolate più volte da differenti chiamate ricorsive. I sottoproblemi in cui viene scomposto il problema non sono disgiunti (questo fenomeno prende il nome di *overlapping* di sottoproblemi) mentre l'algoritmo si comporta come se lo fossero e li risolve nuovamente.

Individuata la causa dell'inefficienza dell'algoritmo è facile individuare la cura. Basta memorizzare in una lista i valori `fib(i)` quando li si calcola la prima volta cosicché nelle future chiamate ricorsive a `fib(i)` non ci sarà più bisogno di ricalcolarli ma potranno essere ricavati dalla lista (questa tecnica prende il nome di *memoizzazione*). Si risparmia così tempo di calcolo al costo di un piccolo incremento di occupazione di memoria.

```

def fib1(n):
    F=[-1]*(n+1)
    return memfib(n, F)

```

```

def memFib(n, F):
    if n<=1:
        return 1
    if F[n]==-1:
        a=memFib(n-1, F)
        b=memFib(n-2, F)
        F[n]=a+b
    return F[n]

```

La novità dell'algoritmo `fib1` è che esso, prima di attivare la ricorsione per il calcolo di qualche f_i , con $i < n$, controlla se quel valore è stato calcolato precedentemente e posto in `F[i]`. In caso affermativo la ricorsione non viene effettuata ma viene restituito direttamente il valore `F[i]`. In questo modo l'algoritmo effettuerà esattamente n chiamate ricorsive (una sola chiamata per il calcolo di ogni f_i con $i < n$). Tenendo conto che ogni chiamata ricorsiva costa $O(1)$ il tempo di calcolo di `fib1` è $\Theta(n)$, un miglioramento esponenziale rispetto alla versione da cui eravamo partiti.

Al punto a cui siamo giunti possiamo anche eliminare la ricorsione ed ottenere la versione iterativa dell'algoritmo.

```

def Fib2(n):
    F=[-1]*(n+1)
    F[0]=F[1]=1
    for i in range(2, n+1):
        F[i]=F[i-2]+F[i-1]
    return F[n]

```

La complessità asintotica rimane $\Theta(n)$ ma abbiamo un risparmio di tempo e spazio (quello necessario alla gestione della ricorsione). La complessità di spazio è $O(n)$ a causa della lista F da memorizzare. Non è difficile ridurre questa occupazione a $\Theta(n)$ tenendo conto che dell'intera tabella basta conservare in memoria volta per volta solo gli ultimi due valori calcolati:

```
def Fib3(n):
    if n <= 1:
        return n
    a=b=1
    for i in range(2, n+1):
        a, b = b, a
    return b
```

Riassumendo: siamo partiti da un algoritmo ricorsivo e non efficiente ottenuto applicando la tecnica del divide et impera al problema in esame; ci siamo accorti che il motivo dell'inefficienza era la presenza di overlapping di sottoproblemi; abbiamo risolto il problema del ricalcolo di soluzioni allo stesso sottoproblema mediante la tecnica della memoizzazione e quindi ricorrendo a "tabelle" per conservare i risultati a sottoproblemi già calcolati; abbiamo sviluppato una versione dell'algoritmo iterativa che ha permesso di sbarazzarsi della ricorsione; abbiamo ottimizzato lo spazio di memoria mantenendo memorizzata nel corso dell'algoritmo solo la parte della tabella che sarebbe servita nel seguito.

Nota che nella versione ricorsiva dell'algoritmo si parte dal problema scomponendolo via via in sottoproblemi di dimensione sempre più piccola fino ad arrivare a problemi facilmente risolvibili. Si parla in questo caso di approccio top-down al problema. Nella versione iterativa dell'algoritmo si comincia col risolvere i sottoproblemi di dimensione sufficientemente piccola per poi passare a quelli di dimensione via via crescente fino ad arrivare alla soluzione del problema originario. Si parla in questo caso di approccio bottom-up al problema.

5.1.2 Sottoinsieme di file

Abbiamo n file di varie dimensioni ciascuna inferiore a C ed un disco di capacità C , bisogna trovare il sottoinsieme di file che può essere memorizzato sul disco e che massimizza lo spazio occupato. Progettare un algoritmo che, dati C e lista A dove $A[i]$ è la dimensione del file i , risolva il problema.

Ad esempio, per $C = 100$ e $A = [82, 15, 40, 95, 31, 50, 40, 28]$ la risposta deve essere 2, 4, 7 che occupa spazio $40 + 31 + 28 = 99$.

Non è difficile rendersi conto che algoritmi greedy del tipo "finché c'è spazio memorizza sul disco il file di massima dimensione" non trovano sempre la soluzione ottima. Per questo problema non si conoscono algoritmi efficienti. Migliori algoritmi si basano su un qualche tipo di ricerca esaustiva della soluzione. Per fortuna è altrettanto facile trovare algoritmi d'approssimazione efficienti con rapporti d'approssimazione costante.

Per semplicità di esposizione ci limiteremo ora a calcolare il valore della soluzione ottima, cioè il massimo spazio del disco che può essere occupato grazie agli n file.

Un algoritmo basato sul divide et impera può partire dalla seguenti osservazioni: Sia (A, C) l'istanza che vogliamo risolvere, se la lista dei file risulta vuota o la capacità C vale 0 la soluzione ottima vale 0. In caso contrario l'ultimo file della lista può appartenere o meno alla soluzione ottima. Se l'ultimo file non appartiene alla soluzione i rimanenti $n - 1$ file devono essere una soluzione ottima per $(A[: -1], C)$; se l'ultimo file appartiene alla soluzione i rimanenti $n - 1$ file devono essere una soluzione ottima per $(A[: -1], C - A[n - 1])$.

Possiamo quindi ricondurci al calcolo della soluzione ottima di due sottoproblemi di dimensione inferiore in quanto manca all'appello l'ultimo file e, una volta risolti questi due problemi, ottenuti i valori v_1 e v_2 delle loro soluzioni, la soluzione al problema di partenza sarà data da $\max(v_1, v_2 + A[-1])$.

```

def es(A, C):
    if len(A)==0 or C==0:
        return 0
    lascio = es(A[:-1], C)
    if A[-1] > C:
        return lascio
    prendo = A[-1] + es(A[:-1], C-A[-1])
    return max(lascio, prendo)

def es(A, i, C):
    if i==0 or C==0:
        return 0
    lascio = es(A, i-1, C)
    if A[i-1] > C:
        return lascio
    prendo = A[i-1] + es(A, i-1, C-A[i-1])
    return max(lascio, prendo)

```

Con la prima implementazione per la complessità si avrà:

$$T(n) = T(n-1, C) + T(n-1, C - A[n-1]) + \Theta(n)$$

il costo additivo $O(n)$ è dovuto al passaggio della copia della lista.

Con la seconda implementazione per la complessità si avrà:

$$T(n) = T(n-1, C) + T(n-1, C - A[n-1]) + \Theta(1)$$

Considera l'istanza in cui $C = n - 1$ e $A[i] = 1, 0 \leq i \leq n$. Per quest'istanza l'equazione di ricorrenza della seconda implementazione dà:

$$T(n) = T(n-1, C) + T(n-1, C - A[n-1]) + \Theta(1) \geq 2T(n-1, n-1) + \Theta(1) = 2T(n-1) + \Theta(1)$$

che risolta dà $T(n) = \Omega(2^n)$. Il motivo di questi tempi intrattabili è dovuto all'overlapping dei sottoproblemi generati dal partizionamento prodotto dal divide et impera.

Di seguito parte dell'albero di ricorsione generato durante l'esecuzione dell'istanza dove $C = 10$ e $lista = [1, 5, 3, 4, 2, 2]$ dove ciascun nodo è una coppia con al primo posto il numero di file dell'istanza ed al secondo la capacità del disco. In rosso sono marcate le chiamate ricorsive duplicate che l'algoritmo ricalcola. Più l'istanza è grande e più chiamate ricorsive duplicate saranno presenti.

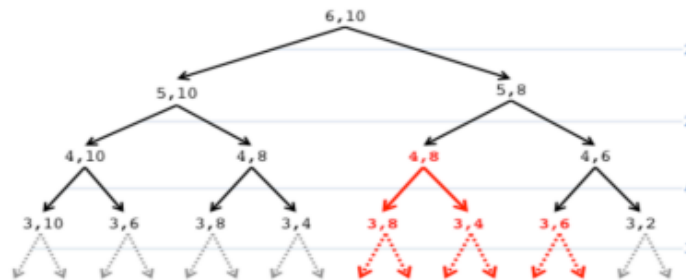


Figura 5.2: Albero di ricorsione del problema

Per evitare di fare questo inutile lavoro ricorriamo alla memoizzazione, utilizziamo una tabella in cui salviamo i risultati via via calcolati per poterli riutilizzare se necessario senza doverli ricalcolare. La tabella T avrà dimensione $n \times (C + 1)$ e nella cella $T[i][j]$ memorizzeremo il valore ottenuto dalla soluzione del sottoproblema in cui si hanno i soli primi i file e la dimensione del disco è j .

```

def es(A, C):
    T = [[-1] * (C+1) for i in range(len(A)+1)]
    return mem_es(A, len(A), C, T)

def mem_es(A, i, c, T):
    if T[i][c] == -1:
        if i==0 or C==0:
            T[i][c] = 0
        else:
            lascio = mem_es(A, i-1, c, T)
            T[i][c] = lascio
            if A[i-1] <= c:
                prendo = A[i-1] + mem_es(A, i-1, c-A[i-1], T)
                T[i][c] = max(lascio, prendo)
    return T[i][c]

```

La complessità dell'implementazione memoizzata è dunque $O(nC)$ (in realtà è $\Theta(nC)$ perché la tabella va inizializzata con i valori -1). Il tempo di calcolo della versione memoizzata è limitato dalla dimensione della tabella. Infatti, la funzione `mem_file()` esegue $O(1)$ operazioni in ogni chiamata e il numero totale di chiamate non può superare la dimensione della tabella.

La complessità della versione memoizzata è dunque $\Theta(nC)$, quella dell'algoritmo esaustivo è $O(2^n)$. C'è stato un risparmio? Dipende. Se C è molto grande, ad esempio quando supera 2^n , la versione memoizzata fa peggio. Però se C non è così grande, la versione memoizzata è più efficiente e a volte enormemente più efficiente.

Consideriamo un caso abbastanza realistico: $n = 100.000$ file e capacità $C = 1.000.000$ (assumiamo che le dimensioni dei file e la capacità siano espresse in multipli di MB, così C equivale a 1 TB) con dimensione massima dei file 1000 (cioè 1 GB) di modo che qualsiasi sottoinsieme di 1000 file può essere memorizzato sul disco. L'algoritmo non memorizzato eseguirà tutte le chiamate ricorsive (cioè sarà sempre possibile scegliere il k -esimo file) almeno relativamente ai primi 1000 file (da $n = 100.000$ a 99.000). Quindi la complessità dell'algoritmo non memorizzato sarà almeno dell'ordine di 2^{1000} . L'algoritmo memorizzato ha una complessità dell'ordine di $nC = 100.000 * 1.000.000 = 100.000.000.000$. Quindi un computer con sufficiente memoria (dell'ordine delle centinaia di GB) potrà eseguire l'algoritmo memorizzato in meno di un'ora. Mentre con l'algoritmo non memorizzato, dovendo eseguire almeno 2^{1000} operazioni, anche usando tutti i computer del pianeta, non riusciremo a vedere il risultato del calcolo neanche aspettando fino alla fine all'età stimata dell'universo.

Come mostra l'esempio appena visto la profondità dell'albero delle chiamate ricorsive (in entrambe le versioni dell'algoritmo) può essere molto grande e portare a stack overflow. Nella versione memoizzata ci si può concentrare direttamente sul calcolo della tabella T eliminando così la ricorsione.

Definiamo una tabella T di dimensioni $(n+1) \times (C+1)$ dove: $T[i, c]$ = spazio massimo con i primi i file su un disco di capacità c con $0 \leq i \leq n$ e $0 \leq c \leq C$. La soluzione al problema originario la troviamo in $T[n][C]$. Possiamo calcolare la tabella per righe grazie alla seguente formula ricorsiva:

$$T[i, c] = \begin{cases} 0 & \text{se } i = 0 \\ T[i-1, c] & \text{se } c < A[i-1] \\ \max(T[i-1, c], A[i-1] + T[i-1, c - A[i-1]]) & \text{altrimenti} \end{cases}$$

```
def es1(A, C):
    n=len(A)
    T=[[0]*(C+1) for i in range(n+1)]
    for i in range(1, n+1):
        for c in range(C+1):
            if c < A[i-1]:
                T[i][c] = T[i-1][c]
            else:
                T[i][c] = max(T[i-1][c], A[i-1]+T[i-1][c-A[i-1]])
    return T[n][C]
```

Ha complessità $O(nC)$. In termini dell'albero delle chiamate ricorsive, l'algoritmo iterativo, calcola i risultati a partire dalle foglie, cioè gli elementi $T[0, c]$, poi i risultati del livello superiore e così via risalendo l'albero di livello in livello fino alla radice $T[n, C]$ (approccio bottom-up al problema).

L'algoritmo che abbiamo ottenuto di complessità $O(nC)$ è un esempio di algoritmo pseudopolinomiale. Ricorda che la capacità C del disco data in input è codificata con $\log C$ bit e quindi per ottenere un algoritmo polinomiale nella complessità può comparire $\log^c C$ per una qualche costante c la presenza di C nella complessità mostra che questa può essere anche esponenziale nella reale dimensione dell'input.

Consideriamo la tabella T del problema relativamente all'istanza $C = 10$ e 6 file di dimensioni $A = [1, 5, 3, 4, 2, 2]$.

Tenendo conto che la tabella è stata calcolata per mezzo della seguente formula ricorsiva:

$$T[i, c] = \begin{cases} 0 & \text{se } i = 0 \\ T[i-1, c] & \text{se } c < A[i-1] \\ \max(T[i-1, c], A[i-1] + T[i-1, c - A[i-1]]) & \text{altrimenti} \end{cases}$$

$i \backslash c$	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	1	1	1	1	1	1	1	1	1	1
2	0	1	1	1	1	5	6	6	6	6	6
3	0	1	1	3	4	5	6	6	8	9	9
4	0	1	1	3	4	5	6	7	8	9	10
5	0	1	2	3	4	5	6	7	8	9	10
6	0	1	2	3	4	5	6	7	8	9	10

Figura 5.3: Tabella per il problema dei file

Nella tabella a partire da $T[n, C]$ da ogni elemento toccato della tabella $T[k, c]$ è tracciata una freccia verso l'elemento in base al quale è stato calcolato $T[k, c]$ (questo elemento sarà $T[k-1, c]$ o $T[k-1, c-A[k]]$). Quindi se la freccia è verso $T[k-1, c]$ (una freccia verticale) vuol dire che il k -esimo file non è scelto mentre se la freccia è verso $T[k-1, c-A[k]]$ (una freccia obliqua) vuol dire che il k -esimo file è stato scelto.

$i \backslash c$	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	1	1	1	1	1	1	1	1	1	1
2	0	1	1	1	1	5	6	6	6	6	6
3	0	1	1	3	4	5	6	6	8	9	9
4	0	1	1	3	4	5	6	7	8	9	10
5	0	1	2	3	4	5	6	7	8	9	10
6	0	1	2	3	4	5	6	7	8	9	10

Figura 5.4: Tabella per il problema dei file

Le frecce rosse evidenziano le decisioni a partire dall'elemento $T[n, C]$ che fornisce il valore della soluzione ottima. In base a queste è possibile dire quali file sono stati scelti nella soluzione ottima (i file di dimensioni 4, 5, e 1). Quindi se $T[k, c] = T[k-1, c]$, il k -esimo file non è scelto (nella soluzione del sotto-problema $T[k, c]$), altrimenti $T[k, c] > T[k-1, c]$ e il k -esimo file è stato scelto.

```
def es1(A, C):
    n=len(A)
    T=[[0]*(C+1) for i in range(n+1)]
    for i in range(1, n+1):
        for c in range(C+1):
            if c < A[i-1]:
                T[i][c]=T[i-1][c]
            else:
                T[i][c] = max(T[i-1][c], A[i-1] + T[i-1][c-A[i-1]])

    valore=T[n][C]
    sol=set()
    i=n
    while i > 0:
        if T[i][valore] != T[i-1][valore]:
            sol.add(i-1)
            valore-=A[i-1]
        i-=1
```

return $T[n][C]$, **sol**

Nella prima parte viene pagato $O(nC)$ per il calcolo della tabella. Nella seconda parte, alla ricerca dei file presenti nella soluzione ottima, la tabella viene visitata a partire dall'ultima cella $T[n, C]$, un riga in meno per volta. Il costo di questa visita è $O(n)$. La complessità è $O(nC)$.

5.1.3 Sequenza di zeri in stringhe binarie

Dato un intero n vogliamo contare le stringhe binarie lunghe n in cui non compaiono zeri consecutivi. L'algoritmo proposto deve avere complessità $O(n)$.

Ad esempio per $n = 1$ sono 2: (0, 1); per $n = 4$ sono 5: (010, 011, 101, 110, 111).

Utilizzeremo una tabella monodimensionale di dimensioni $n + 1$ e definiamo il contenuto delle celle come segue: $T[i]$ = numero di stringhe binarie lunghe i dove non compaiono 2 zeri consecutivi. Una volta riempita la tabella la soluzione al nostro problema la troveremo nella locazione $T[n]$. Resta da definire la regola ricorsiva con cui calcolare i valori $T[i]$ nella tabella.

$$T[i] = \begin{cases} 1 & \text{se } i = 0 \\ 2 & \text{se } i = 1 \\ T[i-1] + T[i-2] & \text{altrimenti} \end{cases}$$

La ricorrenza viene fuori dal seguente ragionamento: il conteggio può essere visto come la somma delle stringhe da contare lunghe i che terminano con 1 e le stringhe da contare lunghe i che terminano con 0. Le stringhe da contare lunghe i che terminano con 1 si ottengono accodando la cifra 1 a quelle da contare lunghe $i - 1$. Queste stringhe sono $T[i - 1]$. Le stringhe da contare lunghe i che terminano con 0 devono avere al penultimo posto la cifra 1, si ottengono dunque accodando 10 alle stringhe da contare lunghe $i - 2$. Queste stringhe sono dunque $T[i - 2]$.

```
def es(n):
    T=[0]*(n+1)
    T[0], T[1] = 1, 2
    for i in range(2, n+1):
        T[i] = T[i-1]+T[i-2]
    return T[n]
```

5.1.4 Persone in un albergo

Dato un intero n vogliamo contare quanti modi diversi ci sono di sistemare n persone in un albergo che dispone di camere singole e camere doppie. L'algoritmo proposto deve avere complessità $\Theta(n)$.

Ad esempio per $n = 2$ sono 2: $\{[1], [2]\}$ $\{[1, 2]\}$; per $n = 4$ sono 10: $\{[1], [2], [3], [4]\}$ $\{[1, 2], [3], [4]\}$ $\{[1, 3], [2], [4]\}$ $\{[1, 4], [2], [3]\}$ $\{[2, 3], [1], [4]\}$ $\{[2, 4], [1], [3]\}$ $\{[3, 4], [1], [2]\}$ $\{[1, 2], [3, 4]\}$ $\{[1, 3], [2, 4]\}$ $\{[1, 4], [2, 3]\}$.

Utilizzeremo una tabella monodimensionale di dimensioni $n + 1$ e definiamo il contenuto delle celle come segue: $T[i]$ = numero di modi in cui è possibile sistemare i persone nell'albergo. Una volta riempita la tabella la soluzione al nostro problema la troveremo nella locazione $T[n]$. Resta da definire la regola ricorsiva con cui calcolare il valore $T[i]$ calcolati i valori $T[j]$ che precedono nella tabella.

$$T[i] = \begin{cases} 1 & \text{se } i = 0 \\ 1 & \text{se } i = 1 \\ T[i-1] + (i-1) * T[i-2] & \text{altrimenti} \end{cases}$$

La ricorrenza viene fuori dal seguente ragionamento: possiamo vedere le disposizioni delle i persone come la somma di due diverse tipologie: quelle in cui la persona i finisce in camera da sola e quelle in cui la persona i finisce in camera con una delle altre $i - 1$ persone. Le differenti disposizioni in cui i finisce da solo sono $T[i - 1]$, sono infatti tutti i possibili modi di disporre le altre $i - 1$ persone. Le differenti disposizioni in cui i finisce in camera con una delle altre $i - 1$ persone sono $(i - 1) * T[i - 2]$ infatti ci sono $i - 1$ modi di scegliere il compagno di stanza e una volta scelto il compagno ci sono $T[i - 2]$ modi di disporre tutte le altre $i - 2$ persone nelle varie camere.

```

def es3(n):
    T=[0 for _ in range(n+1)]
    T[0]=T[1]=1
    for i in range(2, n+1):
        T[i] = T[i-1]+(i-1)*T[i-2]
    return T[n]

```

5.1.5 Somma massima per una sottosequenza

Dato un array A di $n \geq 2$ interi positivi vogliamo calcolare la somma massima per una sottosequenza di elementi di A non consecutivi. L'algoritmo proposto deve avere complessità $O(n)$.

Ad esempio per $A = [5, 4, 2, 10, 6, 8]$ la risposta è 23 infatti per A la sottosequenza di interi non consecutivi di valore massimo è quella di seguito evidenziata in rosso **5, 4, 2, 10, 6, 8**.

Utilizzeremo una tabella monodimensionale di dimensioni $n - 1$ e definiamo il contenuto delle celle come segue: $T[i] =$ la somma massima per una sottosequenza di elementi non consecutivi in $A[: i + 1]$. Una volta riempita la tabella la soluzione al problema sarà data da $T[n - 1]$. Resta da definire la regola ricorsiva con cui calcolare i valori $T[i]$ della tabella.

$$T[i] = \begin{cases} A[0] & \text{se } i = 0 \\ \max(A[0], A[1]) & \text{se } i = 1 \\ \max(T[i-1], T[i-2] + A[i]) & \text{altrimenti} \end{cases}$$

La ricorrenza viene fuori dal seguente ragionamento:

1. Il vettore contiene un solo elemento allora meglio prenderlo perché positivo $T[0] = A[0]$.
2. Il vettore contiene due soli elementi allora meglio prendere il più grande dei due $T[1] = \max(A[0], A[1])$.
3. Alla soluzione di $A[: i + 1]$ appartiene l'ultimo elemento $A[i]$. In questo caso l'elemento $A[i - 1]$ non può appartenere alla soluzione e quindi il massimo deve essere $T[i - 2] + A[i]$. Se al contrario $A[i]$ non appartiene alla soluzione allora per la soluzione deve aversi $T[i] = T[i - 1]$. In generale possiamo quindi dire che $T[i] = \max(T[i - 2] + A[i], T[i - 1])$.

```

def es(A):
    n=len(A)
    T[0]=A[0]
    T[1] = max(A[0], A[1])
    for i in range(2, n):
        T[i] = max(T[i-1], T[i-2] + A[i])
    return T[n-1]

```

Nota che per calcolare un nuovo valore della tabella bastano i due valori precedenti quindi il programma si può migliorare passando dalla complessità di spazio $\Theta(n)$ alla complessità di spazio $\Theta(1)$.

5.1.6 Tassellamenti di una superficie

Dato l'intero n vogliamo contare il numero di differenti tassellamenti di una superficie di dimensione $n \times 2$ tramite tessere di domino di dimensione 1×2 . L'algoritmo deve avere complessità $O(n)$.

Ad esempio per $n = 1$ la risposta dell'algoritmo deve ovviamente essere 1. Per $n = 2$ la risposta deve essere 2 perché sono possibili i soli due seguenti tassellamenti:

x	x	x	x
x	x	x	x

Utilizzeremo una tabella monodimensionale di dimensioni $n + 1$ e definiamo il contenuto delle celle come segue: $T[i] =$ numero di tassellamenti possibili per la superficie di dimensione $i \times 2$. Una

volta riempita la tabella la soluzione al nostro problema la troveremo nella locazione $T[n]$. Resta da definire la regola ricorsiva con cui calcolare i valori $T[i]$ nella tabella.

$$T[i] = \begin{cases} 1 & \text{se } i = 0 \\ 2 & \text{se } i = 1 \\ T[i-1] + T[i-2] & \text{altrimenti} \end{cases}$$

La ricorrenza viene fuori dal seguente ragionamento: possiamo vedere i tassellamenti della superficie di dimensione $i \times 2$ come la somma di due diverse tipologie. I tassellamenti in cui il tassello che copre l'ultima cella in basso a sinistra è posizionato in orizzontale e i tassellamenti in cui il tassello che copre l'ultima cella in basso a sinistra è posizionato in verticale. I tassellamenti in cui il tassello che copre l'ultima cella in basso a sinistra è posizionato in orizzontale sono $T[i-1]$, infatti il tassello occupa l' i -esima riga della superficie e resta da tassellare una superficie di dimensione $(i-1) \times 2$ (le prime $i-1$ righe della superficie) in tutti i modi possibili. Questi modi sono appunto $T[i-1]$. I tassellamenti in cui il tassello che copre l'ultima cella in basso a sinistra è posizionato in verticale sono $T[i-2]$ infatti in questo caso il vertice in basso a destra deve essere anch'esso coperto da un tassello in verticale, questi due tasselli lasciano da coprire una superficie di dimensione $(i-2) \times 2$ (la prime $i-2$ righe della superficie iniziale) in tutti i modi possibili. Questi modi sono appunto $T[i-1]$.

```
def es(n):
    m = max(3, n)
    T = [0]*(n+1)
    T[1], T[2] = 1, 2
    for i in range(3, n+1):
        T[i] = T[i-1] + T[i-2]
    return T[n]
```

5.1.7 Massimo vettore

Data una lista A di n interi, vogliamo trovare una sottolista (una sequenza di elementi consecutivi della lista) la somma dei cui elementi è massima.

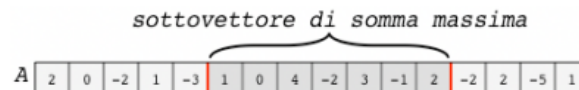


Figura 5.5: Sottolista che contiene valori positivi e negativi

Abbiamo già parlato di questo problema come esempio di applicazione della tecnica del divide et impera, lo riprendiamo ora per tentare un approccio basato sulla programmazione dinamica. Come è tipico di questa tecnica ci concentriamo prima sul calcolare il valore della soluzione.

Cominciamo con l'individuare i sottoproblemi dalla composizione delle cui soluzioni sarà poi possibile risolvere il problema originario. Una scelta che ci porta a definire n sottoproblemi è la seguente: $T[i]$ = massima somma possibile per le sottoliste di A che terminano nella posizione i . Poiché la sottolista a valore massimo deve terminare in una qualche posizione il valore della soluzione che cerchiamo sarà poi dato da $\max_{0 \leq i < n} T[i]$.

Vediamo ora come risolvere in modo efficiente gli n sottoproblemi. Il caso base è facile: $T[0] = A[0]$. Dobbiamo ora poter calcolare $T[i]$, con $i > 0$, sfruttando la conoscenza di $T[i-1]$:

$$T[i] = \max(A[i], A[i] + T[i-1])$$

Infatti la sottolista di valore massimo che termina con $A[i]$ può essere di due soli tipi: consiste del solo elemento $A[i]$ o ha lunghezza superiore ad 1. In questo caso vale $A[i] + S$ dove S è la massima somma di un sottovettore che termina in $i-1$ (il che significa che $S = T[i-1]$).

```
def es(A):
    T = [0]*n
    for i in range(1, n):
        T[i] = max(A[i], A[i]+T[i-1])
    return max(T)
```

Possiamo evitare di dover calcolare al termine il massimo in T , basta una variabile m che tiene traccia del massimo per il valore delle celle di T che via via calcoliamo. Possiamo evitare di tenere in memoria l'intera tabella T visto che per calcolare $T[i]$ basta disporre della sola cella $T[i-1]$ (e del valore $A[i]$) (in questo modo la complessità di spazio passa da $O(n)$ ad $O(1)$).

```
def es(A):
    n=len(A)
    m=t= 0
    for i in range(1, n):
        t = max(A[i], A[i]+t)
        m = max(m, t)
    return m
```

5.1.8 Sottosequenza crescente più lunga

Data una sequenza di S elementi una sottosequenza di S si ottiene eliminando zero o più elementi da S (non necessariamente le eliminazioni devono avvenire in testa o in coda alla sequenza). Ad esempio: data la sequenza 9, 3, 2, 4, 1, 5, 8, 6, 7, 2 una sua sottosequenza è 3, 1, 5, 6 (si ottiene eliminando dalla sequenza gli elementi di seguito indicati in rosso 9, 3, 2, 4, 1, 5, 8, 6, 7, 2). Le sottosequenze possibili per una sequenza di elementi sono $\Theta(2^n)$. Una sottosequenza è detta crescente se i suoi elementi risultano ordinati in modo crescente.

Data una sequenza S di n interi, vogliamo trovare la lunghezza massima per le sottosequenze crescenti presenti in S . Progettare un algoritmo che data una sequenza S di n elementi, in tempo $O(n^2)$ risolve il problema.

Ad esempio: per la sequenza $S = 9, 3, 2, 4, 1, 5, 8, 6, 7, 2$ la risposta è 5 infatti la sottosequenza più lunga in S è 3, 4, 5, 6, 7 (non è l'unica, un'altra possibile soluzione è 2, 4, 5, 6, 7).

Utilizzeremo una tabella monodimensionale di dimensioni n e definiamo il contenuto delle celle come segue: $T[i]$ = la lunghezza massima per una sottosequenza crescente che termina con l'elemento di S in posizione i . Poiché la sottosequenza di lunghezza massima da qualche parte dovrà pur terminare la soluzione al nostro problema sarà data da: $\max_{0 \leq i < n} (T[i])$. Resta da definire la regola ricorsiva con cui calcolare i valori $T[i]$ nella tabella.

$$T[i] = \begin{cases} 1 & \text{se } S_j > S_i \text{ per } 0 \leq j < i \\ \max_{0 \leq j < i \text{ con } S_j < S_i} (T[j]) + 1 & \text{altrimenti} \end{cases}$$

La ricorrenza viene fuori dal seguente ragionamento: se tutti gli elementi che precedono S_i sono maggiori di S_i allora l'unica sottosequenza crescente che termina in posizione i è data dal solo elemento S_i ; in caso contrario la sottosequenza che termina con S_i avrà come prefisso una sottosequenza crescente che termina in una posizione j con $j < i$ e $S_j \leq S_i$ e tra tutti i possibili "agganci" prendo quello che produce la sottosequenza più lunga.

```
def es(A):
    n=len(A)
    if n==0:
        return 0
    T=[1]*n
    for i in range(1, n):
        for j in range(i):
            if A[i] > A[j]:
                T[i] = max(T[i], T[j]+1)
    return max(T)
```

5.1.9 Quadrati di un numero

Un numero intero può sempre essere rappresentato come somma di quadrati di altri numeri interi. Infatti, usando il quadrato 1^n , il generico numero x possiamo sempre scomporlo come somma di n addendi tutti uguali a 1^2 (vale a dire $x = 1^2 + \dots + 1^2$).

Dato un intero n vogliamo sapere qual'è il numero minimo di quadrati necessari rappresentare n . Progettare un algoritmo che dato n in $\Theta(n^{\frac{3}{2}})$ calcola il numero minimo di quadrati che servono per rappresentarlo.

Ad esempio per $n = 0$ la risposta è 0. Per $n = 41$ la risposta è 2 infatti $41 = 5^2 + 4^2$. Nota che vale anche $41 = 6^2 + 2^2 + 1^2$ ma questa scomposizione usa 3 quadrati e non è minimale. Per $n = 6$ la risposta è 3 infatti $6 = 2^2 + 1^2 + 1^2$ e non è difficile vedere che 6 non può esprimersi come somma di due soli quadrati.

Utilizzeremo una tabella monodimensionale di dimensioni $n + 1$ e definiamo il contenuto delle celle come segue: $T[i]$ = numero minimo di quadrati per rappresentare l'intero i . Una volta riempita la tabella troveremo la soluzione in $T[n]$. Resta da definire la regola ricorsiva con cui calcolare i valori $T[i]$ nella tabella.

$$T[i] = \begin{cases} 0 & \text{se } i = 0 \\ \max_{0 \leq j \leq \sqrt{i}} (T[i - j^2]) + 1 & \text{altrimenti} \end{cases}$$

La ricorrenza viene fuori dal seguente ragionamento: per $i = 0$ ovviamente vale $T[0] = 0$. Per il caso generale dove $i > 0$ servirà utilizzare almeno un quadrato. Tra tutti i quadrati potenzialmente utili ad ottenere i servirà dunque trovare quello che poi permette di ottenere i con il minor numero di quadrati. I quadrati potenzialmente utili sono del tipo j con $1 \leq j \leq \sqrt{i}$ e, notando che se si usa il quadrato j resterà poi ancora da rappresentare il numero $i - j^2$ si ottiene la seguente formula: $T[i] = \min_{j \leq \sqrt{i}} T[i - j^2] + 1$.

```
def es(n):
    T=[0]*(n+1)
    for i in range(1, n+1):
        T[i]=n
        j=1
        while j**2 <= i:
            if T[i-j**2]+1 < T[i]:
                T[i] = T[i-j**2]+1
            j+=1
    return T[n]
```

5.1.10 Sequenze decimali non decrescenti

Dato un intero n vogliamo sapere quante sono le sequenze di cifre decimali non decrescenti lunghe n . Progettare un algoritmo che trova la risposta in tempo $O(n)$.

Ad esempio per $n = 1$ la risposta dell'algoritmo deve ovviamente essere 10, per $n = 2$ la risposta deve essere 55. Infatti alla cifra x al primo posto possono seguire $10 - x$ cifre diverse. Quindi si ha

$$\sum_{x=0}^9 (10 - x) = \sum_{i=1}^{10} i = \frac{10 * 11}{2} = 55$$

Utilizzeremo una tabella bidimensionale di dimensioni $n \times 10$ e definiamo il contenuto delle celle come segue: $T[i][j]$ = numero di sequenze decimali non decrescenti lunghe i che terminano con la cifra j . Una volta riempita la tabella la soluzione al nostro problema sarà data dalla somma degli elementi nell'ultima riga: $\sum_{j=0}^9 T[n][j]$. Resta da definire la regola ricorsiva con cui calcolare i valori $T[i][j]$ della tabella:

$$T[i][j] = \begin{cases} 1 & \text{se } i = 1 \\ \sum_{k=0}^j T[i-1][k] & \text{altrimenti} \end{cases}$$

La ricorrenza viene fuori dal seguente ragionamento: esiste un'unica sequenza lunga 1 che termina con la cifra j , la cifra j ed è ovviamente non decrescente. La sequenza lunga i non decrescente che termina con j è composta da una sequenza non decrescente lunga $i - 1$ cui si accoda la cifra j . Perché la sequenza risultante sia non decrescente la sequenza non decrescente lunga $i - 1$ deve terminare con una qualunque cifra k non superiore a j (vale a dire: deve essere una della sequenza contata con $T[i-1, k]$ e $k \leq j$).

```
def es(n):
    T = [[0]*10 for _ in range(n+1)]
    for j in range(10):
        T[1][j]=1
```

```

for i in range(2, n+1):
    for j in range(10):
        for k in range(j+1):
            T[i][j] += T[i-1][k]
return sum(T[n])

```

Inizializzare la tabella costa $\Theta(n)$ (nota che la tabella ha $n + 1$ righe ma un numero costante di colonne: 10). Dei 3 for annidati il primo viene iterato esattamente n volte, il secondo viene ogni volta iterato esattamente 10 volte ed il terzo viene iterato ogni volta al più 10 volte. Questo significa che il costo totale dei 3 for è $\Theta(n)$.

5.1.11 Percorso nella matrice

Data una matrice M binaria $n \times n$ vogliamo verificare se nella matrice è possibile raggiungere la cella in basso a destra partendo da quella in alto a sinistra senza mai toccare celle che contengono il numero 1. Si può assumere che $M[0][0] = 0$. Si tenga conto che ad ogni passo ci si può spostare solo di un passo verso destra o un passo verso il basso. Progettare un algoritmo che in tempo $\Theta(n^2)$ risolve il problema rispondendo True o False.

Ad esempio per la matrice A la risposta è sì mentre per la matrice B la risposta è no.

$A =$	0	0	0	0	0	1
	0	1	0	1	1	1
	0	0	0	1	0	1
	0	1	0	0	0	0
	0	0	0	0	1	0
	1	1	0	1	0	0
	1	1	0	1	0	0

$B =$	0	0	0	0	0	0
	0	1	1	1	0	0
	0	1	0	0	0	0
	0	1	0	1	1	1
	0	1	0	0	0	0
	0	1	0	0	0	0
	0	0	0	0	1	0

Utilizziamo una tabella T bidimensionale $n \times n$ dove $T[i][j] = \text{True}$ se esiste un cammino che in M va dalla cella $M[0][0]$ in alto a sinistra e arriva alla cella $M[i][j]$ senza toccare celle con valore 1 (False altrimenti). La soluzione al problema sarà nella cella $T[n-1][n-1]$.

Ecco di seguito la regola ricorsiva che permette di calcolare i valori della tabella

$$T[i][j] = \begin{cases} \text{False} & \text{se } M[i][j] = 1 \\ \text{True} & \text{se } i = j = 0 \\ T[i][j-1] & \text{se } i = 0 \\ T[i-1][j] & \text{se } j = 0 \\ T[i][j-1] \text{ or } T[i-1][j] & \text{altrimenti} \end{cases}$$

La ricorrenza viene fuori dal seguente ragionamento: la cella $M[0][0]$ è ovviamente raggiungibile; se la cella $M[i][j]$ contiene 1 allora non è possibile raggiungerla. Assumiamo quindi nel seguito che $M[i][j] = 0$. Una cella della prima riga può essere raggiunta solo dalla cella che la precede sulla riga. Una cella della prima colonna può essere raggiunta solo dalla cella che la precede sulla colonna. Una cella "interna" è raggiungibile se posso raggiungerla dalla cella che la precede sulla riga e/o dalla cella che la precede sulla colonna.

```

def es (M):
    n=len(M)
    T = [[True]*n for _ in range(n)]
    for j in range(1,n):
        T[0][j] = T[0][j-1]
        if M[0][j]==1:
            T[0][j] = False
    for i in range(1,n):
        T[i][0] = T[i-1][0]
        if M[i][0]==1:
            T[i][0] = False
    for i in range(1,n):
        for j in range(1,n):
            T[i][j] = T[i-1][j] or T[i][j-1]
            if M[i][j]==1:

```

```

    T[i][j] = False
    return T[n-1][n-1]

```

5.1.12 Sottomatrici quadrate di dimensione massima

Abbiamo una matrice quadrata binaria M di dimensione $n \times n$ e vogliamo sapere qual è la dimensione massima per le sottomatrici quadrate di soli uni contenute in M . Progettare un algoritmo che data la matrice M restituisce il massimo intero l per cui la matrice $l \times l$ è una sottomatrice di soli uni di M . L'algoritmo deve avere complessità $O(n^2)$.

Ad esempio, per la matrice M in figura la risposta è 3:

$$M = \begin{array}{ccccc} 1 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 & 1 \end{array}$$

Utilizziamo una tabella T bidimensionale $n \times n$ dove: $T[i][j]$ = il lato della matrice quadrata più grande contenente tutti uni e con cella in basso a destra $M[i][j]$ (se $M[i][j] = 0$ allora $T[i][j] = 0$). La soluzione al nostro problema sarà il massimo tra i valori della tabella. Ecco di seguito la regola ricorsiva che permette di calcolare i vari valori della tabella:

$$T[i][j] = \begin{cases} 0 & \text{se } M[i][j] = 0 \\ \min\{T[i][j-1], T[i-1][j-1], T[i-1][j]\} + 1 & \text{altrimenti} \end{cases}$$

La ricorrenza viene fuori dal seguente ragionamento: se $M[i][j] = 0$ allora il rettangolo di tutti uni che ha come cella in basso a destra quella cella ha lato zero. Se $M[i][j] = 1$ allora il lato del rettangolo di soli uni che ha come cella in basso a destra quella cella non può superare $T[i-1][j] + 1$ né $T[i][j-1] + 1$ né $T[i-1][j-1] + 1$ ne segue che il lato di quel rettangolo è al più $\min\{T[i][j-1], T[i-1][j-1], T[i-1][j]\} + 1$ d'altra parte non è difficile vedere che preso $l = \min\{T[i][j-1], T[i-1][j-1], T[i-1][j]\}$ allora il rettangolo di lato $l + 1$ con cella in basso a destra $M[i][j]$ è un rettangolo contenente solo uni.

```

def es(M):
    n=len(M)
    T = [[0]*n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if M[i][j]==0:
                T[i][j]=0
            else:
                T[i][j] = min(T[i][j-1], T[i-1][j-1], T[i-1][j])+1
    m=0
    for i in range(n):
        m = max(m, max(T[i]))
    return m

```

5.1.13 Problema dello zaino

Abbiamo uno zaino di capacità c ed n oggetti, ognuno con un peso p_i , e un valore v_i . Vogliamo sapere il valore massimo che si può inserire nello zaino. Progettare un algoritmo che, data la capacità c dello zaino e i vettori P (dei pesi) e V (dei valori) degli oggetti, risolve il problema in tempo $O(nc)$.

Ad esempio, si consideri l'istanza con $c = 11$ e $n = 5$ oggetti con peso e valore riportati nella seguente tabella:

oggetto	valore	peso
1	1	1
2	6	4
3	18	5
4	22	5
5	28	7

Il sottoinsieme di oggetti (3, 5) non è una soluzione ammissibile (perché di peso $12 > C$). Il sottoinsieme di oggetti (1, 2, 4) è una soluzione ammissibile (di peso 10 e valore 29) ma non ottima (perché ne esistono di valore maggiore). Il sottoinsieme di oggetti (1, 3, 4) è una soluzione ammissibile (di peso 11 e valore 41) si può dimostrare che è anche una soluzione ottima.

Utilizziamo una tabella bidimensionale T di dimensione $(n+1) \times (C+1)$ dove: $T[i][j]$ = massimo valore ottenibile dai primi i oggetti per uno zaino di capacità j . La soluzione al nostro problema sarà $T[n][c]$. Ecco di seguito la regola ricorsiva che permette di calcolare i vari valori della tabella:

$$T[i][j] = \begin{cases} 0 & \text{se } i = 0 \text{ o } j = 0 \\ T[i-1][j] & \text{se } p_i > j \\ \max\{T[i-1][j], v_i + T[i-1][j - p_i]\} & \text{altrimenti} \end{cases}$$

La ricorrenza viene fuori dal seguente ragionamento: se non si hanno oggetti (vale a dire $i = 0$) o la capacità dello zaino è nulla allora il valore della soluzione sarà 0. Se l' i -esimo oggetto ha un peso superiore alla capacità dello zaino (vale a dire $j < P[i-1]$), allora non può esservi inserito e quindi il valore della soluzione dipenderà da quello che si può ottenere dagli altri $i-1$ oggetti (vale a dire $T[i-1, c]$). In generale, se nello zaino c'è possibilità di inserire l'oggetto i (vale a dire $P[i-1] \leq j$) allora ci sono due possibilità: l'oggetto i non viene inserito nello zaino, in questo caso il massimo valore della soluzione sarà $T[i-1, c]$, oppure l'oggetto i viene inserito nello zaino. In questo caso c'è un guadagno $V[i]$ e per i rimanenti $i-1$ oggetti resta disponibile una capacità residua di $j - P[i]$. Di conseguenza il valore massimo della soluzione sarà $V[i-1] + T[i-1, j - P[i-1]]$. Pertanto il valore della soluzione sarà $\max(T[i-1, j], V[i-1] + T[i-1, c - P[i-1]])$.

```
def es(P, V, c):
    n=len(P)
    T = [[0]*(c+1) for i in range(n+1)]
    for i in range(1, n+1):
        for j in range(1, c+1):
            if j < P[i-1]:
                T[i][j] = T[i-1][j]
            else:
                T[i][j] = max(T[i-1][j], V[i-1]+T[i-1][j-P[i-1]])
    return T, T[n][c]
```

5.1.14 Problema del resto

Ho n diversi tagli di banconote ed un resto c da dare. Voglio sapere quanti modi ci sono di produrre il resto e se ho quantità illimitata di banconote dei vari tagli. Dato l'intero c e ed il vettore M con gli n tagli delle banconote ($M[i]$ è l' i -esimo taglio) progettare un algoritmo che risolve il problema. L'algoritmo proposto deve avere complessità $O(nc)$.

Ad esempio, con i tre tagli 1, 2 e 3 e $c = 5$ la risposta è 5 (infatti si hanno i seguenti possibili resti: 11111, 1112, 113, 23, 122).

Utilizzeremo una tabella bidimensionale di dimensioni $(n+1) \times (c+1)$ e definiamo il contenuto delle celle come segue: $T[i][j]$ = differenti modi di dare il resto j quando si dispone dei soli primi i tagli $0 \leq i \leq n, 0 \leq j \leq c$. Una volta riempita la tabella la soluzione al problema sarà data da $T[n][c]$. Ecco di seguito la ricorrenza che permette di riempire la tabella:

$$T[i][j] = \begin{cases} 1 & \text{se } j = 0 \\ 0 & \text{se } i = 0 \\ T[i-i][j] & \text{se } j < A[i-1] \\ T[i-1][j] + T[i][j - A[i-1]] & \text{altrimenti} \end{cases}$$

La ricorrenza viene fuori dal seguente ragionamento: c'è un solo modo di dare resto zero, $T[i][0] = 1$. Con zero tagli non c'è modo di dare un resto $j > 0$, $T[0][j] = 0$ (se $j > 0$). Se l' i -esimo taglio è maggiore del resto da dare allora le banconote di quel taglio non possono contribuire alla produzione del resto, $T[i][j] = T[i-1][j]$. In generale, se la banconota può contribuire al resto posso avere un resto in cui la uso ed uno in cui non la uso. La banconota i non viene usata per produrre il resto j , in questo caso il resto posso ottenerlo solo con le altre banconote $T[i][j] = T[i-1][j]$. La banconota

i viene usata per produrre il resto i , in questo caso dovrò produrre ancora il resto $j - M[i]$ e quindi $T[i][j] = T[i][j - M[i]]$. Pertanto i modi di dare il resto sono $T[i][j] = T[i - 1][j] + T[i][j - M[i]]$.

```
def es(A, c):
    n=len(A)
    T = [[0](c + 1) for _ in range(n+1)]
    for i in range(n+1):
        T[i][0]=1
    for j in range(1, c+1):
        T[0][j]=0
    for i in range(1, n+1):
        for j in range(1, c+1):
            if j < A[i-1]:
                T[i][j] = T[i-1][j]
            else:
                T[i][j] = T[i-1][j] + T[i][j-A[i-1]]
    return T, T[n][c]
```

5.1.15 Guadagno massimo con le transazioni

Una transazione è l'acquisto di un oggetto seguito dalla sua vendita (che non può ovviamente avvenire prima del giorno dell'acquisto). Disponiamo di un vettore A di interi dove $A[i]$ è la quotazione dell'oggetto nel giorno i . Dato il vettore A con le quotazioni dei prossimi n giorni e dovendo eseguire una singola transazione vogliamo sapere qual'è il guadagno massimo cui possiamo aspirare. Nota che se il vettore è decrescente il guadagno massimo è 0. Progettare un algoritmo che risolve il problema in tempo $O(n)$.

Ad esempio per $A = [7, 9, 5, 6, 3, 2]$ il guadagno massimo è 2 (basta acquistare a 7 e rivendere a 9). Per $B = [3, 2, 6, 10, 4, 8, 1]$ il guadagno massimo è 8 (basta acquistare a 2 e rivendere a 10). Utilizziamo una tabella T monodimensionale di lunghezza n a dove: $T[i]$ = il guadagno massimo che ottengo se vendo il giorno i . La soluzione al problema sarà $\max(T)$. La ricorrenza che permette di riempire la tabella è la seguente:

$$T[i] = \begin{cases} 0 & \text{se } i = 0 \\ A[i] - \min(A[:i+1]) & \text{altrimenti} \end{cases}$$

La ricorrenza viene fuori dal seguente ragionamento: se vendo il giorno iniziale non posso che comprare il giorno stesso e quindi il guadagno è 0. Se vendo il giorno i ottengo $A[i]$ per la vendita e per avere il guadagno massimo devo aver acquistato nel giorno tra 0 ed i in cui l'oggetto costava meno. Per ottenere tempo $O(n)$ devo essere in grado di calcolare ciascuna delle n celle in tempo $O(1)$ e per far ciò basta pre-calcolare per ciascun giorno i il costo minimo fino a quel giorno.

```
def es(A):
    B = [0]*n
    B[0] = A[0]
    for i in range(1, n):
        B[i] = min(A[i], B[i-1])
    T = [0]*n
    for i in range(1, n):
        T[i] = A[i]-B[i]
    return max(T)
```

5.1.16 Guadagno massimo con k transazioni

Una transazione è l'acquisto di un oggetto seguito dalla sua vendita (che non può ovviamente avvenire prima del giorno dell'acquisto). Disponiamo di un vettore A di interi dove $A[i]$ è la quotazione dell'oggetto nel giorno i . Dato il vettore A con le quotazioni dei prossimi giorni ed un intero k e dovendo eseguire al più k transazioni vogliamo sapere qual'è il guadagno massimo cui possiamo aspirare. Attenzione non è possibile cominciare una nuova transazione se non dopo aver completato la transazione precedente. Nota che se il vettore è decrescente il guadagno massimo è 0. Progettare un algoritmo che risolve il problema in tempo $O(nk)$.

Ad esempio per $A = [10, 22, 5, 75, 65, 80]$ il guadagno massimo è 87 (basta acquistare a 10 e rivendere a 22 e poi a 5 per rivendere a 80). Per $B = [100, 30, 15, 10, 8, 25, 80]$ il guadagno massimo è 72 (basta acquistare a 8 rivendere a 80).

Utilizziamo una tabella T bidimensionale di lunghezza $n \times (k + 1)$ dove: $T[i][j]$ = il guadagno massimo che ottengo se faccio j transazioni entro il giorno i . La soluzione al problema sarà $T[n - 1][k]$. La ricorrenza che permette di riempire la tabella è la seguente:

$$T[i][j] = \begin{cases} 0 & \text{se } i = 0 \text{ o } j = 0 \\ \max(T[i - 1][j], \max_{0 \leq s \leq i} ((A[i] - A[s]) + T[s][j - 1])) & \text{altrimenti} \end{cases}$$

La ricorrenza viene fuori dal seguente ragionamento: un numero arbitrario di transazioni eseguite nel giorno 0 fruttano 0. Conterò transazioni eseguite, indipendentemente dal giorno il guadagno è 0. Se devo calcolare $T[i][j]$ con j e i maggiori di zero posso effettuare o meno una delle j transazioni nel giorno i . Se non effettuo transazioni nel giorno i allora il mio guadagno sarà $T[i - 1][j]$. Se effettuo almeno una transazione nel giorno i allora devo aver acquistato in un giorno $s \leq i$ e quindi il mio guadagno sarà $(A[i] - A[s]) + T[s][j - 1]$ e per il guadagno massimo devo prendere $\max_{0 \leq s \leq i} ((A[i] - A[s]) + T[s][j - 1])$. Per ottenere il meglio delle due situazioni devo quindi prendere: $\max(T[i - 1][j], \max_{0 \leq s \leq i} ((A[i] - A[s]) + T[s][j - 1]))$.

```
def es(A, k):
    n=len(A)
    T = [[0]*(k+1) for i in range(n)]
    for i in range(1, n):
        for j in range(1, k+1):
            T[i][j] = T[i-1][j]
            for s in range(i+1):
                T[i][j] = max(T[i][j], (A[i]-A[s])+T[s][j-1])
    return T[n-1][k]
```

5.1.17 Problema della massima sottosequenza comune

Date due sequenze di simboli X e Y , vogliamo trovare una sottosequenza comune X e Y che abbia lunghezza massima. Progettate un algoritmo che risolva il problema in tempo $O(nm)$ dove n sono i simboli in X ed m quelli in Y .

Ad esempio, per $X = ABCBDAB$ e $Y = BDCABA$ una delle possibili sottosequenze comuni di lunghezza 3 è BCA ; una sottosequenza di lunghezza massima è $BCAB$; altre sottosequenze di lunghezza massima sono $BCBA$ e $BDAB$.

Concentriamoci preliminarmente sul calcolare il valore della soluzione ottima. Sia $X = x_1, x_2, \dots, x_n$ e $Y = y_1, y_2, \dots, y_m$. Definiamo una tabella bidimensionale T di dimensioni $(n + 1) \times (m + 1)$ dove: $T[i][j]$ = massima lunghezza possibile per sottosequenze comuni al prefisso di X di lunghezza i ed al prefisso di Y di lunghezza j . $T[i][j]$ con $i, j \geq 1$ conterrà dunque la lunghezza massima per sottosequenze comuni alle due stringhe $X' = x_1, x_2, \dots, x_i$ e $Y' = y_1, y_2, \dots, y_j$. La soluzione al nostro problema sarà nella cella $T[n][m]$. Resta ora da definire la formula ricorsiva che permette di calcolare la cella $T[i, j]$ a partire dalle celle precedenti già calcolate:

$$T[i][j] = \begin{cases} 0 & \text{se } i = 0 \text{ o } j = 0 \\ T[i - 1][j - 1] + 1 & \text{se } X[i - 1] = Y[j - 1] \\ \max\{T[i - 1][j], T[i][j - 1]\} & \text{altrimenti} \end{cases}$$

La ricorrenza viene fuori dal seguente ragionamento: se non possiamo usare i simboli di una delle due sequenze è ovvio che la sottosequenza comune avrà lunghezza 0. Se le due sequenze terminano con lo stesso simbolo allora la sottosequenza comune più lunga terminerà con quel simbolo e per il resto sarà la sequenza più lunga tra $x_1, x_2, \dots, x_{i-1}$ e $y_1, y_2, \dots, y_{j-1}$, vale a dire $T[i, j] = T[i - 1][j - 1] + 1$. Se le due sequenze terminano con simboli diversi, diciamo a per X e b per Y , allora deve verificarsi uno dei seguenti tre casi:

- la sottosequenza più lunga termina con a . In questo caso l'ultimo simbolo di Y non influisce e si ha: $T[i][j] = T[i][j - 1]$;

- la sottosequenza più lunga termina con b . In questo caso l'ultimo simbolo di X non influisce e si ha: $T[i][j] = T[i-1][j]$;
- la sottosequenza più lunga termina con un simbolo c diverso da a e b . In questo caso l'ultimo simbolo di X e l'ultimo simbolo di Y non influiscono e si ha $T[i][j] = T[i-1][j-1]$.

Dovendo prendere il caso che produce la sottosequenza più lunga deve aversi: $T = \max(T[i][j-1], T[i-1][j], T[i-1][j-1]) = \max(T[i][j-1], T[i-1][j])$, dove l'ultimo passaggio segue perché $T[i-1][j-1] \leq T[i][j-1]$ e $T[i-1][j-1] \leq T[i-1][j]$.

```
def es(X, Y):
    n, m = len(X), len(Y)
    T = [[0]*(m+1) for _ in range(n+1)]
    for i in range(n+1):
        for j in range(m+1):
            if i==0 or j==0:
                T[i][j]=0
            elif X[i-1]==Y[j-1]:
                T[i][j] = T[i-1][j-1]+1
            else:
                T[i][j] = max(T[i-1][j], T[i][j-1])
    return T[n][m]
```

5.1.18 Ricerca di cammini su grafi con pesi negativi

Un ciclo negativo è un ciclo diretto formato da archi con pesi la cui somma è negativa. Se in un cammino tra i nodi s e t è presente un nodo che appartiene ad un ciclo negativo allora tra s e t non esiste il cammino di costo minimo. Se per il ciclo W si ha $\text{costo}(W) < 0$, ripassando più volte attraverso il ciclo W possiamo abbassare arbitrariamente il costo del cammino da s a t .

Dato un grafo diretto e pesato G con archi i cui pesi possono essere anche negativi ma che non contiene cicli negativi e fissato un suo nodo s , vogliamo calcolare il costo minimo dei cammini che portano da s agli altri nodi del grafo (il costo è $+\infty$ se non c'è cammino).

E' un problema che abbiamo già affrontato nel caso in cui i pesi del grafo G sono tutti non negativi (algoritmo di Dijkstra). Con la presenza di archi di peso negativo l'algoritmo di Dijkstra può produrre errori. Vedremo ora un algoritmo basato sulla tecnica della programmazione dinamica che risolve il problema in tempo $O(n^2 + mn)$. L'algoritmo è noto in letteratura come algoritmo di Bellman-Ford.

Lemma 5.1.1 – Cammini di lunghezza $n - 1$

Se G non ha cicli negativi allora per ogni nodo t raggiungibile dalla sorgente s esiste un cammino di costo minimo che ha lunghezza al più $n - 1$.

Dimostrazione. Consideriamo il più corto cammino minimo e assumiamo per assurdo che abbia lunghezza superiore ad $n - 1$. C'è allora un nodo z toccato più volte dal cammino. Di conseguenza il cammino contiene un ciclo. Possiamo eliminare dal cammino il ciclo ed ottenere un cammino di lunghezza inferiore che è ancora minimo (ricorda che il ciclo eliminato non può avere costo negativo).

Per quanto appena visto, nella ricerca dei cammini minimi, possiamo restringerci a cammini di lunghezza al più $n - 1$. Tutto questo suggerisce di considerare i sottoproblemi che si ottengono limitando la lunghezza dei cammini. Definiamo così la seguente tabella di dimensione $n \times n$:

$$T[k][j] = \begin{cases} \text{costo di un cammino minimo da } s \text{ a } j \text{ di lunghezza al più } k & \text{se esiste} \\ +\infty & \text{altrimenti} \end{cases}$$

La soluzione al nostro problema sarà data dagli n valori che troviamo nell'ultima riga della tabella: $T[n-1][0], T[n-1][1], T[n-1][2], \dots, T[n-1][n-1]$. Infatti il costo minimo per andare da s al generico nodo t sarà $T[n-1][t]$.

Resta da definire la formula ricorsiva che permette di calcolare i valori delle celle $T[k][j]$ in funzione di celle già calcolate:

$$T[k][j] = \begin{cases} 0 & \text{se } j = s \\ +\infty & \text{se } k = 0 \\ \min_{\langle x,j \rangle \in E} (T[k-1][x] + \text{costo}(x,j)) & \text{altrimenti} \end{cases}$$

Per giustificare il caso $k > 0$ e $v \neq s$ si può ragionare in questo modo: se al contrario il cammino ha costo al più k al nodo $j \neq s$ devo arrivare provenendo da uno dei suoi vicini z attraversando l'arco $\langle x, z \rangle$, e a x devo essere arrivato con un cammino minimo che ha attraversato al più $k-1$ archi per cui: $T[k][v] = T[k-1][x] + \text{costo}(x, v)$. Tra tutti i nodi z per cui questo è possibile devo prendere quello di costo minimo, e allora $T[k][v] = \min_{\langle x,j \rangle \in E} (T[k-1][x] + \text{costo}(x, j))$.

```
def CostoCammini(G, s):
    from math import inf
    n=len(G)
    T = [[inf]*n for _ in range(n)]
    T[0][s]=0
    GT=Trasposto(G)
    for k in range(1,n):
        for j in range(n):
            if j==s:
                T[k][j]=0
            else:
                for x, costo in GT[j]:
                    T[k][j] = min(T[k][j], T[k-1][x]+costo)
    return T[n-1]

def Trasposto(G):
    n=len(G)
    GT = [[] for _ in G]
    for i in range(n):
        for j, costo in G[i]:
            GT[j].append((i, costo))
    return GT
```

L'inizializzazione della tabella T richiede tempo $\Theta(n^2)$. La costruzione del grafo trasposto GT richiede tempo $O(n+m)$ (nota che grazie a questo pre-processing avremo un accesso efficiente ai nodi che hanno un arco diretto in j). Per i tre for annidati ($\text{for } k \text{ in range}(1, n) \dots \text{for } j \text{ in range}(n) \dots \text{for } \langle x, \text{costo} \rangle \text{ in } GT[j]$) è ovvio il limite superiore $O(n^3)$. Tuttavia un'analisi più attenta permette di dare un limite più stretto: i due for più interni ($\text{for } j \text{ in range}(n) \dots \text{for } \langle x, \text{costo} \rangle \text{ in } GT[j]$) hanno costo totale $\Theta(m)$. Sostanzialmente il tempo richiesto è quello di scorrere tutte le liste di adiacenza del grafo GT che hanno lunghezza totale m .

Per ritrovare anche i cammini, oltre che il loro costo, con la tabella T bisogna calcolare anche l'albero P dei cammini minimi. Questo si può fare facilmente mantenendo per ogni nodo j il suo predecessore cioè il nodo u che precede j nel cammino. Il valore di $P[j]$ andrà aggiornato ogni volta che il valore di $T[k][j]$ cambia (ovvero diminuisce) in quanto abbiamo trovato un cammino migliore.

```
def CostoCammini1(G, s):
    from math import inf
    n=len(G)
    T = [[inf]*n for _ in range(n)]
    P=[-1]*n
    GT=Trasposto(G)
    P[s]=s
    T[0][s]=0
    for k in range(1,n):
        for i in range(n):
            if i==s:
                T[k][i]=0
```

```

else :
    for x, costo in GT[j]:
        if T[k-1][x] + costo < T[k][j]:
            T[k][j] = T[k-1][x] + costo
            P[i] = x
return T[n-1], P

```

Con questa implementazione al termine dell'algoritmo $T[n-1][j] \neq +\infty$ indica che j è raggiungibile a partire da s , in questo caso $P[j]$ conterrà il nodo che precede j nel cammino minimo da s a j . $T[n-1][j] = +\infty$ indica che j non è raggiungibile a partire da s , in questo caso $P[j]$ conterrà il valore -1.

Il contenuto di una cella della riga k dipende dal contenuto delle celle alla riga $k-1$, quindi: se la riga k della tabella T è identica alla riga $k-1$ anche le righe seguenti non varieranno, tanto vale allora terminare l'algoritmo senza aver calcolato le righe restanti della tabella. Questo accorgimento non modifica la complessità asintotica dell'algoritmo ma in pratica può contare molto. Non serve memorizzare l'intera tabella T bastano le ultime due righe. Perciò l'algoritmo può essere facilmente modificato in modo da utilizzare memoria $O(n)$ anziché $O(n^2)$.

L'algoritmo appena descritto è conosciuto con il nome di Algoritmo di Bellman-Ford. A priori non sappiamo se il grafo su cui applichiamo l'algoritmo ha cicli negativi (e quindi se i cammini restituiti sono effettivamente i cammini minimi o semplicemente i cammini minimi con lunghezza al più $n-1$). Una piccola modifica alla versione dell'algoritmo di Bellman-Ford appena descritto permette di scoprire se il grafo contiene cicli negativi raggiungibili da s o meno: calcola una riga in più della tabella (vale a dire la riga n) con il costo dei cammini minimi di lunghezza al più n . Le righe n e $n-1$ della tabella risultano identiche se e solo se nel grafo non ci sono cicli negativi raggiungibili da s . Implementare questo test ovviamente non cambia l'asintotica dell'algoritmo.

5.1.19 Gioco delle carte agli estremi

Si ha una sequenza di n carte, ciascuna carta ha come valore un numero intero. Due giocatori a turno prendono una delle due carte agli estremi della sequenza. Al termine del gioco il punteggio di ciascun giocatore è dato dalla somma dei valori delle carte da lui prese. Lo scopo del gioco è ottenere il punteggio superiore a quello dell'avversario. Data la sequenza dei valori delle n carte tramite la lista A (dove $A[i]$ è il valore dell' i -esima carta della sequenza), progettare un algoritmo che in tempo $O(n^2)$ determini qual'è la cifra massima che posso sempre vincere indipendentemente dalle mosse del mio avversario.

Ad esempio per $A = [10, 100, 2, 1]$ la risposta è 101 (basta prendere alla prima mossa la carta 1 e poi al secondo turno la carta 100).

Uso una tabella bidimensionale $n \times n$ dove: $T[i][j]$ = il valore massimo che posso guadagnare (indipendentemente dalle mosse del mio avversario) se si gioca sul sottovettore che inizia in $A[i]$ e termina in $A[j]$, $i \leq j$. La soluzione al problema sarà il valore $T[0][n-1]$. Resta ora da definire la formula ricorsiva che permette di calcolare la cella $T[i][j]$ a partire dalle celle precedenti già calcolate:

$$T[i][j] = \begin{cases} A[i][j] & \text{se } i = j \\ \max(A[i] + (S[i+1][j] - T[i+1][j]), A[j] + (S[i][j-1] - T[i][j-1])) & \text{altrimenti} \end{cases}$$

Dove con $S[i][j]$ denota la somma degli elementi della sequenza che vanno da $A[i]$ ad $A[j]$. Nota che $S[i][j] - T[i][j]$ rappresenta quanto si può al più guadagnare giocando sul sottovettore che inizia in $A[i]$ e termina in $A[j]$ quando tocca all'avversario muovere per primo. La ricorrenza viene fuori dal seguente ragionamento: se $i = j$ il gioco si svolge su una sequenza di un solo elemento e dovendo essere io a muovere per primo si ha $T[i][j] = A[i]$. Se $i < j$ posso scegliere una delle due carte agli estremi:

- Se scelgo $A[i]$ seguirà una partita sul vettore che va da $A[i+1]$ a $A[j]$ con il mio avversario che muove per primo. Guadagnerò quindi di certo $A[i] + (S[i+1][j] - T[i+1][j])$.
- Se scelgo $A[j]$ seguirà una partita sul vettore che va da $A[i]$ a $A[j-1]$ con il mio avversario che muove per primo. Guadagnerò quindi di certo $A[j] + (S[i][j-1] - T[i][j-1])$.

Posso dunque guadagnare $\max(A[i] + (S[i+1][j] - T[i+1][j]), A[j] + (S[i][j-1] - T[i][j-1]))$. Nota che la tabella è definita solo per $i \leq j$ e per poter calcolare $T[i][j]$ devo disporre dei valori $T[i+1][j]$ e $T[i][j-1]$ devo quindi riempire la tabella per diagonali a partire dalla diagonale principale.

```

def CalcolaS(A):
    n = len(A)
    S = [[0]*n for _ in range(n)]
    for i in range(n):
        for j in range(i,n):
            if i==j:
                S[i][j] = A[i]
            else:
                S[i][j] = S[i][j-1]+A[j]
    return S

def es(A):
    n = len(A)
    T = [[0]*n for _ in range(n)]
    S = CalcolaS(A)
    for i in range(n):
        T[i][i] = A[i]
    for l in range(1,n):
        for i in range(n-l):
            T[i][i+l] = max(A[i]+(S[i+1][i+l]-T[i+1][i+l]),
                           A[i+l]+(S[i][i+l-1]-T[i][i+l-1]))
    return T[0][n-1], T

```

Capitolo 6

Backtracking

6.1 Algoritmi di backtracking

6.1.1 Stringhe binarie con almeno k uni

Progettare un algoritmo che prende come parametro due interi n e k , con $0 \leq k \leq n$, e stampa tutte le stringhe binarie lunghe n che contengono al più k uni.

Ad esempio: per $n = 4$ e $k = 2$, delle $2^4 = 16$ stringhe lunghe n bisogna stampare le seguenti 11: 0000, 0001, 0010, 0100, 1000, 0011, 0101, 1001, 0110, 1010, 1100.

Un possibile algoritmo che risolve il problema in $\Theta(2^n * n)$.

```
def bk1(n, k, i=0, sol=[]):
    if i==n:
        if sol.count('1') <=k:
            print(''.join(sol))
        return
    sol.append('0')
    bk1(n, k, i+1, sol)
    sol.pop()
    sol.append('1')
    bk1(n, k, i+1, sol)
    sol.pop()
```

Indichiamo con $S(n, k)$ il numero di stringhe che bisogna stampare. Un buon algoritmo per questo problema dovrebbe avere una complessità proporzionale alle stringhe da stampare, vale a dire $O(S(n, k) * n)$ (poche stringhe = poco tempo). Ad esempio per $k = 2$ si ha

$$S(n, k) = 1 + n + \binom{n}{2} = \Theta(n^2)$$

e quindi un buon algoritmo per $k = 2$ dovrebbe avere complessità polinomiale $O(n^3)$ mentre l'algoritmo proposto ha complessità esponenziale $\Theta(2^n * n)$ (indipendente da k).

Osservazione: è inutile generare nell'albero di ricorsione i nodi che non hanno la possibilità di portare a soluzioni da stampare. Si ha quindi un algoritmo alternativo con un controllo sul numero di uni presenti nel prefisso delle stringhe che si vanno via via generando.

```
def bk2(n, k, i=0, tot1=0, sol=[]):
    if i==n:
        print(''.join(sol))
        return
    sol.append('0')
    bk2(n, k, i+1, tot1, sol)
    sol.pop()
    if tot1 < k:
        sol.append('1')
        bk2(n, k, i+1, tot1+1, sol)
        sol.pop()
```

Si consideri un algoritmo di enumerazione basato sul backtracking dove l'albero di ricorsione ha altezza h , il costo di una foglia è $g(n)$ e il costo di un nodo interno è $O(f(n))$. Se l'algoritmo gode della seguente proprietà: un nodo viene generato solo se ha la possibilità di portare ad una foglia da stampare. Allora la complessità dell'algoritmo è proporzionale al numero di cose da stampare $S(n)$, più precisamente la complessità dell'algoritmo è:

$$O(S(n) * h * f(n) + S(n) * g(n))$$

questo perchè il costo totale dei nodi foglia sarà $O(S(n) * g(n))$ (in quanto solo le foglie da enumerare verranno generate) e i nodi interni dell'albero che verranno effettivamente generati saranno $O(S(n) * h)$ (in quanto ogni nodo interno generato apparterrà ad un cammino che parte dalla radice e arriva ad una delle $S(n)$ foglie da enumerare).

Per l'algoritmo codificato con la funzione `bk2(n, k)`, la proprietà di generare un nodo solo se questo può portare ad una delle $S(n, k)$ foglie da stampare è rispettata. Inoltre $h = n$, $g(n) = \Theta(n)$, $f(n) = O(1)$. Quindi la complessità dell'algoritmo è:

$$S(n, k) * nO(1) + S(n, k) * \Theta(n) = \Theta(S(n, k) * n)$$

e l'algoritmo risulta ottimale. La complessità dell'algoritmo è $O(n^{k+1})$ infatti $S(n, k) = \binom{n}{0} + \binom{n}{1} + \binom{n}{2} + \dots + \binom{n}{k} < 2 * n^k$.

6.1.2 Stringhe binarie con esattamente k uni

Progettare un algoritmo che prende come parametro due interi n e k , con $0 \leq k \leq n$, e stampa tutte le stringhe binarie lunghe n che contengono esattamente k uni.

Ad esempio: per $n = 6$ e $k = 3$, delle $2^6 = 64$ stringhe lunghe n bisogna stampare le seguenti 20: 000111, 001011, 001101, 001110, 010011, 010101, 010110, 011001, 011010, 011100, 100011, 100101, 100110, 101001, 101010, 101100, 110001, 110010, 110100, 111000.

Rispetto all'esercizio precedente aggiungiamo ora una funzione di taglio anche nel caso in cui al prefisso viene aggiunto uno zero. Bisogna assicurarsi infatti che sia sempre possibile completare quel prefisso in modo da ottenere una stringa valida da stampare.

```
def bk3(n, k, i=0, tot1=0, sol=[]):
    if i==n:
        print(' '.join(sol))
        return
    if tot1+n-(i+1)>=k:
        sol.append('0')
        bk3(n, k, i+1, tot1, sol)
        sol.pop()
    if tot1<k:
        sol.append('1')
        bk3(n, k, i+1, tot1+1, sol)
        sol.pop()
```

Per l'algoritmo codificato con la funzione `bk3(n, k)` la proprietà di generare un nodo solo se questo può portare ad una delle $S(n, k)$ foglie da stampare è rispettata. Inoltre, l'altezza dell'albero di ricorsione è $h = n$, il tempo richiesto da una foglia è $g(n) = \Theta(n)$ e il tempo richiesto da un nodo interno è $f(n) = O(1)$. Quindi la complessità dell'algoritmo è:

$$S(n, k) * n * O(1) + S(n, k) * \Theta(n) = \Theta(S(n, k) * n)$$

e l'algoritmo risulta ottimale. La complessità dell'algoritmo è $O(n^{k+1})$ infatti $S(n, k) = \binom{n}{k} < n^k$.

6.1.3 Stringhe sull'alfabeto

Progettare un algoritmo che, dato un intero n , stampa tutte le stringhe sull'alfabeto dei 3 simboli a, b e c , in cui il numero delle b supera quello di ciascuno degli altri due simboli. L'algoritmo proposto deve avere complessità $O(nD(n))$ dove $D(n)$ è il numero di stringhe da stampare.

Ad esempio per $n = 2$ l'algoritmo deve stampare la sola stringa `bb` mentre per $n = 3$ le stringhe da stampare sono le seguenti: `bbb, abb, bab, bba, cbb, bcb, bbc`.

Per la stampa delle stringhe eseguiamo una funzione di backtracking. Al passo ricorsivo i -esimo possiamo potenzialmente inserire uno dei tre simboli b , a e c dell'alfabeto. Per ottenere una complessità proporzionale alle $D(n)$ stringhe da stampare la funzione di backtracking controlla che la scelta di inserire uno dei tre simboli venga fatta solo se la soluzione parziale che si ottiene porta ad un elemento da stampare. A questo proposito utilizziamo una semplice funzione di taglio che si avvale di due contatori (na con il numero delle a presenti nella soluzione parziale e nc con il numero delle c presenti nella soluzione parziale):

- inseriamo il simbolo b questo simbolo può sempre essere inserito;
- inseriamo il simbolo a solo se l'inserimento di a lascia spazio sufficiente per inserire eventualmente altri simboli di tipo b in modo da rendere alla fine il numero di b presenti nella stringa superiore a quello delle a e delle c . Il numero delle a a seguito dell'inserimento diventa $na + 1$ il numero delle b potenziali è $n - (na + nc + 1)$ il numero delle c resta nc . Deve quindi aversi $n - (na + nc + 1) > na + 1$ e anche $n - (na + nc + 1) > nc$.
- inseriamo il simbolo c solo se l'inserimento di c lascia spazio sufficiente per inserire eventualmente altri simboli di tipo b in modo da rendere alla fine il numero di b presenti nella stringa superiore a quello delle c e quello delle a . Il numero delle c a seguito dell'inserimento diventa $nc + 1$ il numero delle b potenziali è $n - (na + nc + 1)$ quello delle a resta na . Deve quindi aversi $n - (na + nc + 1) > nc + 1$ e anche $n - (na + nc + 1) > na$.

```
def es(n, i=0, na=0, nc=0, sol=[]):
    if i==n:
        print(''.join(sol))
        return
    sol.append('b')
    es(n, i+1, na, nc, sol)
    sol.pop()
    if n-(na+nc+1) > na+1 and n-(na+nc+1) > nc:
        sol.append('a')
        es(n, i+1, na+1, nc, sol)
        sol.pop()
    if n-(na+nc+1) > nc+1 and n-(na+nc+1) > na:
        sol.append('c')
        es(n, i+1, na, nc+1, sol)
        sol.pop()
```

Nota che nell'albero di ricorsione prodotto dall'esecuzione di `es2R` un nodo viene generato solo se porta ad una foglia da stampare. Possiamo quindi dire che la complessità dell'esecuzione di `es2R(0, n, 0, 0, [ ],)` richiederà tempo:

$$O(D(n) * h * f(n) + D(n) * g(n))$$

dove: $h = n$ è l'altezza dell'albero, $f(n) = O(1)$ è il lavoro di un nodo interno, $g(n) = O(n)$ è il lavoro di una foglia. Quindi il costo di `es2R(0, n, 0, 0, [ ],)` è $O(nD(n))$. Otteniamo quindi che il costo di `es2(n)` è $O(nD(n))$.

6.1.4 Sequenze valide

Fissiamo n , k e T positivi con $T \leq nk$. Definiamo valida una sequenza di lunghezza n contenente interi da 0 a k e la cui somma è T . Ad esempio per $n = 6$, $k = 4$ e $T = 12$ allora la sequenza 132042 è valida mentre 121213 non è valida.

*Trovate un algoritmo che dati n , k e T stampi tutte e sole le sequenze valide. L'algoritmo deve avere complessità $O(n * k * S(n, k))$ dove $S(n, k)$ è il numero di sequenze valide esistenti.*

Utilizziamo un algoritmo di backtracking per la stampa di tutte le stringhe lunghe n sull'alfabeto $\{0, \dots, k\}$ con l'aggiunta di una funzione di taglio. Per la funzione di taglio possiamo ragionare come segue: ad ogni inserimento nella sequenza che via via costruiamo teniamo traccia in una variabile *tot* della somma degli interi utilizzati. Dopo l'inserimento dei primi $i - 1$ valori nella sequenza, l'inserimento del valore j , con $0 \leq j \leq k$ avverrà se e solo se la sequenza potrà poi completarsi in

modo che la somma dei suoi elementi sia almeno T . In altri termini l'inserimento di j avviene se e solo se entrambi i seguenti vincoli sono soddisfatti: $tot + j + (n - i - 1) * k \geq T$ e $tot + j \leq T$. Grazie alla funzione di taglio la soluzione parziale via via costruita è sempre un prefisso di una soluzione da stampare.

```
def es (n, k, T, i=0, tot=0, sol=[]):
    if i==n:
        print(sol)
        return
    for j in range(k+1):
        if tot+j+(n-i-1)*k >= T and tot+j <= T:
            sol.append(j)
            es(n, k, T, i+1, tot+j, sol)
            sol.pop()
```

Nota che nell'albero di ricorsione prodotto dall'esecuzione dell'algoritmo un nodo viene generato solo se porta ad una foglia da stampare. Possiamo quindi dire che la complessità dell'esecuzione di `es3(n, k, T, i=0 tot=0, sol = [])` richiederà tempo

$$O(S(n, k) * hf(n) + S(n, k) * g(n))$$

dove: $h = n$ l'altezza dell'albero, $f(n) = \Theta(k)$ è il lavoro di un nodo interno, $g(n) = \Theta(n)$ è il lavoro di una foglia. Quindi il costo è $O(nkS(n, k))$.

6.1.5 Stringhe di * e di caratteri crescenti

Progettare un algoritmo che, data una stringa X lunga sull'alfabeto ternario $\{0, 1, 2\}$, stampa tutte le stringhe che è possibile ottenere da X sostituendo il simbolo '' ad alcuni dei suoi caratteri in modo che i caratteri rimanenti risultino in ordine strettamente crescente. L'algoritmo proposto deve avere complessità $O(nS(X))$ dove $S(X)$ è il numero di stringhe da stampare.*

Ad esempio per $X = 021$ l'algoritmo deve stampare, non necessariamente nello stesso ordine, le stringhe: `***`, `0**`, `02*`, `0*1`, `*2*`, `**1`, per $X = 2110$ l'algoritmo deve stampare, non necessariamente nello stesso ordine, le stringhe: `****`, `2***`, `*1**`, `**1*`, `***0`.

Sostanzialmente nelle soluzioni che si vanno costruendo in ciascuna posizione o inseriamo il simbolo * o lasciamo il simbolo della stringa X . Partiamo dunque dall'algoritmo per la stampa delle stringhe binarie e aggiungiamo delle funzioni di taglio che assicurino che uno dei due simboli viene aggiunto alla soluzione parziale solo se quella soluzione potrà poi completarsi in una stringa da stampare.

Il simbolo * può sempre essere aggiunto quindi non c'è bisogno di funzione di taglio. Il simbolo della stringa può essere aggiunto solo se è il primo di questa nuova stringa o è maggiore dell'ultimo simbolo diverso da * inserito finora. C'è dunque in questo caso bisogno di una funzione di taglio.

Per far sì che la funzione di taglio sia calcolabile in tempo $O(1)$ utilizziamo una variabile p che tiene traccia dell'ultimo simbolo diverso da * inserito nella stringa che si va costruendo ($p = *$ se la stringa non contiene simboli diversi da *).

```
def es(X, i=0, p='*', sol=[]):
    if i==len(X):
        print(' '.join(sol))
        return
    sol.append('*')
    es(X, i+1, p, sol)
    sol.pop()
    if p=='*' or X[i] > p:
        sol.append(X[i])
        es(X, i+1, X[i], sol)
        sol.pop()
```

Nota che nell'albero di ricorsione prodotto dall'esecuzione dell'algoritmo un nodo viene generato solo se porta ad una foglia da stampare. Possiamo quindi dire che la complessità dell'algoritmo richiederà tempo

$$O(S(X) * hf(n) + S(X) * g(n))$$

dove: $h = n$ è l'altezza dell'albero, $f(n) = O(1)$ è il lavoro di un nodo interno, $g(n) = \Theta(n)$ è il lavoro di una foglia. Quindi il costo è $\Theta(n * S(X))$.

6.1.6 Matrici binarie $n \times n$

Progettare un algoritmo che prende come parametro un intero n e stampa tutte le matrici binarie $n \times n$.

Ad esempio: per $n = 2$, bisogna stampare le seguenti $2^4 = 16$ matrici:

<table><tr><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td></tr></table>	0	0	0	0	<table><tr><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td></tr></table>	0	0	0	1	<table><tr><td>0</td><td>0</td></tr><tr><td>1</td><td>0</td></tr></table>	0	0	1	0	<table><tr><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td></tr></table>	0	0	1	1	<table><tr><td>0</td><td>1</td></tr><tr><td>0</td><td>0</td></tr></table>	0	1	0	0	<table><tr><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td></tr></table>	0	1	0	1	<table><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	0	1	1	0	<table><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td></tr></table>	0	1	1	1
0	0																																						
0	0																																						
0	0																																						
0	1																																						
0	0																																						
1	0																																						
0	0																																						
1	1																																						
0	1																																						
0	0																																						
0	1																																						
0	1																																						
0	1																																						
1	0																																						
0	1																																						
1	1																																						
<table><tr><td>1</td><td>0</td></tr><tr><td>0</td><td>0</td></tr></table>	1	0	0	0	<table><tr><td>1</td><td>0</td></tr><tr><td>0</td><td>1</td></tr></table>	1	0	0	1	<table><tr><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td></tr></table>	1	0	1	0	<table><tr><td>1</td><td>0</td></tr><tr><td>1</td><td>1</td></tr></table>	1	0	1	1	<table><tr><td>1</td><td>1</td></tr><tr><td>0</td><td>0</td></tr></table>	1	1	0	0	<table><tr><td>1</td><td>1</td></tr><tr><td>0</td><td>1</td></tr></table>	1	1	0	1	<table><tr><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	1	1	1	0	<table><tr><td>1</td><td>1</td></tr><tr><td>1</td><td>1</td></tr></table>	1	1	1	1
1	0																																						
0	0																																						
1	0																																						
0	1																																						
1	0																																						
1	0																																						
1	0																																						
1	1																																						
1	1																																						
0	0																																						
1	1																																						
0	1																																						
1	1																																						
1	0																																						
1	1																																						
1	1																																						

```
def es(n):
    sol = [[0]*n for _ in range(n)]
    es1(n, sol)

def es1(n, sol, i=0, j=0):
    if i==n:
        for riga in range(n):
            print(sol[riga])
        print()
        return
    sol[i][j]=0
    if j < n-1:
        es1(n, sol, i, j+1)
    else:
        es1(n, sol, i+1, 0)
    sol[i][j]=1
    if j < n-1:
        es1(n, sol, i, j+1)
    else:
        es1(n, sol, i+1, 0)
```

L'albero di ricorsione è binario e di altezza 2^n ha dunque $2^{n^2} - 1$ nodi interni e 2^{n^2} foglie. Ciascun nodo interno richiede tempo $O(1)$ e ciascuna foglia $O(n^2)$. L'algoritmo ha complessità $O(2^{n^2} n^2)$.

Poiché le matrici da stampare sono 2^{n^2} e la stampa di una matrice richiede $\Theta(n^2)$. Qualunque algoritmo per questo problema richiede tempo $\Omega(2^{n^2} n^2)$. L'algoritmo è ottimo.

6.1.7 Matrici binarie $n \times n$ senza uno adiacenti

Progettare un algoritmo che prende come parametro un intero n e stampa tutte le matrici binarie $n \times n$ in cui non compaiono uno adiacenti (in orizzontale, verticale o diagonale). La complessità dell'algoritmo deve essere $O(n^2 S(n))$ dove $S(n)$ è il numero di matrici da stampare.

Ad esempio per $n = 3$ delle $2^9 = 512$ matrici quadrate 3×3 e binarie bisogna stampare le seguenti 34:

0	0	0
0	0	0
0	0	0

0	0	0
0	0	0
0	0	1

0	0	0
0	0	0
0	1	0

0	0	0
0	0	0
1	0	0

0	0	0
0	0	0
1	0	1

0	0	0
0	0	1
0	0	0

0	0	0
0	0	1
1	0	0

| | | | | |---|---|---| | 0 | 0 | 0 | | 0 | 1 | 0 | | 0 | 0 | 0 | | | | | | |---|---|---| | 0 | 0 | 0 | | 1 | 0 | 0 | | 0 | 0 | 0 | | | | | | |---|---|---| | 0 | 0 | 0 | | 1 | 0 | 0 | | 0 | 0 | 1 | | | | | | |---|---|---| | 0 | 0 | 0 | | 1 | 0 | 1 | | 0 | 0 | 0 | | | | | | |---|---|---| | 0 | 0 | 1 | | 0 | 0 | 0 | | 0 | 0 | 0 | | | | | | |---|---|---| | 0 | 0 | 1 | | 0 | 0 | 0 | | 0 | 0 | 1 | | | | | | |---|---|---| | 0 | 0 | 1 | | 0 | 0 | 0 | | 0 | 1 | 0 | |
| | | | | |---|---|---| | 0 | 0 | 1 | | 0 | 0 | 0 | | 1 | 0 | 0 | | | | | | |---|---|---| | 0 | 0 | 1 | | 0 | 0 | 0 | | 1 | 0 | 1 | | | | | | |---|---|---| | 0 | 0 | 1 | | 1 | 0 | 0 | | 0 | 0 | 0 | | | | | | |---|---|---| | 0 | 0 | 1 | | 1 | 0 | 0 | | 0 | 0 | 1 | | | | | | |---|---|---| | 0 | 1 | 0 | | 0 | 0 | 0 | | 0 | 0 | 0 | | | | | | |---|---|---| | 0 | 1 | 0 | | 0 | 0 | 0 | | 0 | 1 | 0 | | | | | | |---|---|---| | 0 | 1 | 0 | | 0 | 0 | 0 | | 1 | 0 | 0 | |

0	0	0	0	0	1	0	1	1	1
0	0	0	1	1	1	1	1	1	1

Per la stampa delle matrici eseguiamo una funzione di backtracking. Nella generica locazione $sol[i][j]$ è potenzialmente consentito inserire 0 o 1. Per ottenere una complessità proporzionale alle $S(n)$ matrici da stampare basta controllare che il bit inserito consenta sempre di ottenere poi una matrice da stampare. Il bit 1 può sempre essere inserito mentre per il bit 0 c'è bisogno di una funzione di taglio che controlli che nelle caselle già riempite di sol adiacenti a $sol[i][j]$ in orizzontale o verticale non sia stato inserito un 1. Le potenziali caselle adiacenti già riempite sono le seguenti 2: $sol[i-1][j]$ e $sol[i][j-1]$. Inseriamo dunque il simbolo 0 in $sol[i][j]$ solo se

($i = 0$ or $sol[i-1][j] = 0$) and ($j = 0$ or $sol[i][j-1] == 0$)

```
def es(n):
    sol = [[0]*n for _ in range(n)]
    es1(n, sol)

def es1(n, sol, i=0, j=0):
    if i==n:
        for riga in range(n):
            print(sol[riga])
        print()
        return
    if (i==0 or sol[i-1][j]==0) and (j==0 or sol[i][j-1]==0):
        sol[i][j]=0
        if j < n-1:
            es1(n, sol, i, j+1)
        else:
            es1(n, sol, i+1, 0)
        sol[i][j]=1
        if j < n-1:
            es1(n, sol, i, j+1)
        else:
            es1(n, sol, i+1, 0)
```

6.1.9 Permutazioni da 0 a $n-1$

Progettare un algoritmo che prende come parametro l'intero n e stampa tutte le permutazioni dei numeri da 0 a $n-1$.

Ad esempio per $n = 4$ bisogna stampare $4! = 24$ permutazioni: $[0, 1, 2, 3]$ $[0, 1, 3, 2]$ $[0, 2, 1, 3]$ $[0, 2, 3, 1]$ $[0, 3, 1, 2]$ $[0, 3, 2, 1]$ $[1, 0, 2, 3]$ $[1, 0, 3, 2]$ $[1, 2, 0, 3]$ $[1, 2, 3, 0]$ $[1, 3, 0, 2]$ $[1, 3, 2, 0]$ $[2, 0, 1, 3]$ $[2, 0, 3, 1]$ $[2, 1, 0, 3]$ $[2, 1, 3, 0]$ $[2, 3, 0, 1]$ $[2, 3, 1, 0]$ $[3, 0, 1, 2]$ $[3, 0, 2, 1]$ $[3, 1, 0, 2]$ $[3, 1, 2, 0]$ $[3, 2, 0, 1]$ $[3, 2, 1, 0]$.

L'albero delle permutazioni ha $\Theta(n!)$ foglie e $\Theta(n!)$ nodi interni.

$$\sum_{i=0}^n \frac{n!}{i!} \sum_{i=0}^{\infty} \frac{1}{i!} \sum_{i=0}^{\infty} \frac{2}{2^i} = n! \frac{2}{1 - \frac{1}{2}} = 4n!$$

ho usato $i! \geq \frac{2^i}{2}$ e $\sum_{i=0}^{\infty} x^i = \frac{1}{1-x}$ per $x < 1$.

```
def es(n):
    preso = [0]*n
    es1(n, 0, [], preso)

def es1(n, i, sol, preso):
    if i==n:
        print(sol)
        return
    for j in range(n):
        if preso[j]==0:
```

```

sol.append(j)
preso[j]=1
es1(n, i+1, sol, preso)
sol.pop()
preso[j]=0

```

L'algoritmo ha complessità $\Theta(n!n)$: l'albero delle permutazioni ha $\Theta(n!)$ nodi interni e $n!$ foglie $= 2^{\Theta(n \log n)}$; ciascun nodo interno richiede tempo $\Theta(n)$ e ciascuna foglia richiede tempo $\Theta(n)$.

Qualunque algoritmo per questo problema richiede tempo $\Omega(n!n)$: le soluzioni da stampare sono $n!$ e il tempo per stampare ciascuna di queste è $\Theta(n)$. L'algoritmo proposto è ottimo.

6.1.10 Permutazioni da 0 a $n - 1$ con posizioni e numeri pari

Progettare un algoritmo che prende come parametro l'intero n e stampa tutte le permutazioni dei numeri da 0 a $n - 1$ dove nelle posizioni pari compaiono numeri pari. La complessità dell'algoritmo deve essere $O(S(n)n^2)$ dove $S(n)$ è il numero di permutazioni da stampare.

Ad esempio per $n = 5$ delle $5! = 120$ permutazioni bisogna stampare le seguenti 12: [0, 1, 2, 3, 4] [0, 1, 4, 3, 2] [0, 3, 2, 1, 4] [0, 3, 4, 1, 2] [2, 1, 0, 3, 4] [2, 1, 4, 3, 0] [2, 3, 0, 1, 4] [2, 3, 4, 1, 0] [4, 1, 0, 3, 2] [4, 1, 2, 3, 0] [4, 3, 0, 1, 2] [4, 3, 2, 1, 0].

Per la stampa delle permutazioni eseguiamo una funzione di backtracking. Al passo ricorsivo i -esimo possiamo potenzialmente inserire uno degli n interi non ancora inseriti. Per ottenere una complessità proporzionale alle $S(n)$ permutazioni da stampare la funzione di backtracking controlla che la scelta di inserire un intero non ancora inserito j venga fatta solo se j è pari e la posizione in cui inserire è pari o j è dispari e la posizione in cui inserire j è dispari (vale a dire $j \bmod 2 == i \bmod 2$).

```

def es(n):
    preso = [0]*n
    es1(n, 0, [], preso)

def es1(n, i, sol, preso):
    if i==n:
        print(sol)
        return
    for j in range(n):
        if preso[j]==0 and j%2==i%2:
            sol.append(j)
            preso[j]=1
            es1(n, i+1, sol, preso)
            sol.pop()
            preso[j]=0

```

Sia $S(n)$ il numero di permutazioni da stampare. L'algoritmo ha complessità $O(S(n)n^2)$: l'albero di ricorsione è di altezza n ; solo i nodi che portano ad una delle $S(n)$ soluzioni vengono effettivamente generati; i nodi interni effettivamente generati saranno $O(S(n)n)$ e le foglie effettivamente generate saranno $S(n)$; ciascun nodo interno richiede tempo $\Theta(n)$ e ciascuna foglia richiede tempo $\Theta(n)$; il tempo in totale sarà $O(S(n)n)\Theta(n) + S(n)\Theta(n) = O(S(n)n^2)$.

6.1.11 Problema di ottimizzazione dello Zaino

Abbiamo n oggetti, ciascuno caratterizzato da un peso p_i ed un valore v_i . Abbiamo inoltre uno zaino di capacità C . Tra tutti i possibili sottoinsiemi degli n oggetti vogliamo trovare quello il cui peso non supera C e il cui valore sia massimo. Il peso ed il valore di un insieme di oggetti sono dati dalla somma dei pesi e dalla somma dei valori degli oggetti dell'insieme rispettivamente. Progettare un algoritmo che prende in input la lista dei valori degli n oggetti e la lista dei pesi degli n oggetti e la capacità dello zaino e restituisce la soluzione al problema (la lista con gli oggetti da inserire nello zaino per massimizzarne il valore).

Ad esempio consideriamo la seguente istanza con $C = 166$ e $n = 10$ oggetti con peso e valore riportati in tabella:

oggetto	peso	valore
0	23	92
1	31	57
2	29	49
3	89	87
4	44	68
5	53	60
6	38	43
7	63	67
8	85	84
9	82	72

La soluzione ottima pesa 165, vale 309 ed il suo vettore caratteristico è $SOL = [1, 1, 1, 0, 1, 0, 1, 0, 0, 0]$.

Possiamo risolvere il problema ricorrendo ad un algoritmo esaustivo: generiamo tutti i possibili sottoinsiemi degli n oggetti. Per ognuno verifichiamo se è una soluzione ammissibile (vale a dire se il peso del sottoinsieme di oggetti non eccede la capacità dello zaino) e, in questo caso, ne calcoliamo il valore. Teniamo traccia della soluzione ammissibile di valore massimo via-via trovata. Al termine diamo in output il massimo trovato.

Poiché i sottoinsiemi da considerare per n oggetti sono 2^n l'algoritmo esaustivo avrà complessità esponenziale $\Omega(2^n)$.

```
def es(P, V, c):
    def es1(P, V, c, i, Sol):
        n=len(V)
        nonlocal Sol0, val0
        if i==n:
            if sum([P[i] for i in range(n) if Sol[i] == 1]) <= c:
                val = sum([V[i] for i in range(n) if Sol[i]==1])
                if val > val0:
                    Sol0 = Sol.copy(); val0 = val
            else:
                Sol.append(0)
                es1(P, V, c, i+1, Sol)
                Sol.pop()
                Sol.append(1)
                es1(P, V, c, i+1, Sol)
                Sol.pop()

        val0=-1; Sol0=[]
        es1(P, V, c, 0, [])
        print(val0, Sol0)
```

Non è difficile modificare il codice in modo che ogni nodo sappia il valore e il peso parziale della soluzione che si sta costruendo. In questo modo i nodi foglia possono scoprire se bisogna aggiornare la soluzione ottima in tempo $O(1)$. Nota che il numero di nodi interni da generare resta $\Theta(2^n)$ e l'asintotica dell'algoritmo risulta ora $\Theta(2^n)$. Cosa più importante, in questo modo ogni nodo interno sa quanto vale e quanto pesa la soluzione parziale costruita fino a quel momento. Queste informazioni possono risultare utili in vista dell'introduzione di eventuali funzioni di taglio.

```
def es(P, V, c):
    def es1(P, V, c, i, Sol, val, peso):
        n=len(V)
        nonlocal Sol0, val0
        if i==n:
            if peso <= c:
                if val > val0:
                    Sol0 = Sol.copy(); val0 = val
            else:
                pass
```

```

        Sol.append(0)
        es1(P, V, c, i+1, Sol, val, peso)
        Sol.pop()
        Sol.append(1)
        es1(P, V, c, i+1, Sol, val+V[i], peso+P[i])
        Sol.pop()

    val0=-1; Sol0=[]
    es1(P, V, c, 0, [], 0, 0)
    print(val0, Sol0)

```

Trasformiamo l'algoritmo di ricerca esaustiva in un algoritmo di backtracking introducendo funzioni di taglio che permetteranno di potare alcuni dei $\Theta(2^n)$ nodi presenti nell'albero di ricorsione. Nel nostro caso procederemo come segue:

1. potiamo i nodi che non portano a soluzioni ammissibili;
2. potiamo nodi che se anche portassero a soluzioni ammissibili queste non sarebbero tali da migliorare la soluzione ottima fino a quel momento ottenuta.

La funzione di taglio del punto 1 verrà applicata al figlio di destra di ogni nodo interno (dove si decide se prendere l'oggetto). La funzione di taglio del punto 2 verrà applicata al figlio di sinistra di ogni nodo interno (dove si decide di non prendere l'oggetto).

```

def es(P, V, c)

    def es1(P, V, c, i, Sol, val, peso):
        n=len(V)
        nonlocal Sol0, val0
        if i==n:
            if val > val0:
                Sol0=Sol.copy(); val0=val
        else:
            Sol.append(0)
            es1(P, V, c, i+1, Sol, val, peso)
            Sol.pop()
            if peso+P[i] <= c:
                Sol.append(1)
                es1(P, V, c, i+1, Sol, val+V[i], peso+P[i])
                Sol.pop()

    val0=-1; Sol0=[]
    es1(P, V, c, 0, [], 0, 0)
    print(val0, Sol0)

```

Resta da implementare la seconda funzione di taglio: potare quei nodi che se anche portassero a soluzioni ammissibili queste non sarebbero tali da migliorare la soluzione ottima fino a quel momento ottenuta.

Un modo poco costoso per raggiungere parzialmente quest'obiettivo vien fuori dalla seguente semplice osservazione: se al livello i della ricorsione decido di non inserire l'oggetto i nello zaino, nessuna delle soluzioni ammissibili che sarà possibile ottenere potrà valere più di $val + \sum_{j=i+1}^{n-1} v_j$. Di conseguenza se $val + \sum_{j=i+1}^{n-1} v_j \leq val0$ possiamo potare l'albero di ricorsione evitando di porre $sol[i] = 0$.

L'obiettivo è raggiunto solo parzialmente perché $val + \sum_{j=i+1}^{n-1} v_j > val0$ non garantisce che le soluzioni ammissibili che sarà possibile raggiungere saranno realmente in grado di migliorare la soluzione ottima.

Per rendere efficiente il calcolo di questa funzione di taglio nel codice che segue faremo sì che ogni nodo dell'albero a livello i riceva, una variabile $valR$ con l'informazione del valore totale degli oggetti restanti (vale a dire $\sum_{j=i}^{n-1} v_j$).

```

def es(P, V, c):

    def es1(P, V, c, i, Sol, val, valR, peso):
        n=len(V)
        nonlocal Sol0, val0
        if i==n:
            if val > val0:
                Sol0=Sol.copy(); val0=val
        else:
            if val+valR-V[i] > val0:
                Sol.append(0)
                es1(P, V, c, i+1, Sol, val, valR-V[i], peso)
                Sol.pop()
            if peso+P[i] <= c:
                Sol.append(1)
                es1(P, V, c, i+1, Sol, val+V[i], val-RV[i], peso+P[i])
                Sol.pop()
        val0=-1; Sol0=[]; valR=sum(V)
        es1(P, V, c, 0, [], 0, valR, 0)
    print(val0, Sol0)

```

Per il problema dello zaino quindi abbiamo visto 3 algoritmi:

- Zaino(P,V,C): algoritmo di ricerca esaustiva.
- Zaino1(P,V,C): algoritmo di backtraking con una funzione di taglio.
- Zaino2(P,V,C): algoritmo di backtraking con due funzioni di taglio.

L'efficacia delle funzioni di taglio per questo tipo di algoritmi può essere validata sperimentalmente. Ad esempio possiamo introdurre nei codici dei 3 algoritmi un contatore dei nodi interni dell'albero di ricorsione effettivamente generati nel corso della loro esecuzione.

Eseguendo i tre algoritmi sull'istanza d'esempio con gli $n = 10$ oggetti, vale a dire:

```

P=[23, 31, 29, 89, 44, 53, 38, 63, 85, 82]
V=[92, 57, 49, 87, 68, 60, 43, 67, 84, 72]
C=166

```

per i $2^n - 1 = 1023$ nodi interni dell'albero di ricorsione si ha:

- Per Zaino(P,V,C) vengono visitati 1023 su 1023 nodi interni
- Per Zaino1(P,V,C) vengono visitati 102 su 1023 nodi interni
- Per Zaino2(P,V,C) vengono visitati 371 su 1023 nodi interni

In genere migliore è la funzione di taglio più questa risulta computazionalmente onerosa. Considerando che ogni nodo dell'albero di ricorsione dovrà applicarla, non sempre funzioni in grado di tagliare di più producono algoritmi più veloci.

L'effettività di alcune funzioni di taglio può dipendere dalla qualità della soluzione ottima prodotta fino a quel momento. Un esempio di ciò si ha con la seconda funzione di taglio che abbiamo utilizzato per lo zaino. In questi casi può essere fruttuoso far partire il backtraking da una soluzione di buona qualità che poi il backtraking tenterà di migliorare. Per ottenere "buone" soluzioni da cui partire è possibile ricorrere, in una fase di preprocessing, ad algoritmi approssimanti che garantiscono di produrre soluzioni di buona qualità (anche se non ottime) in tempi contenuti.

6.1.12 Problema della 3-colorazione

Sia G un grafo non diretto, una 3-colorazione di G è una colorazione dei suoi nodi con 3-colori (0, 1 e 2) tale che non risultino nodi adiacenti con lo stesso colore. Se una colorazione esiste possiamo rappresentarla con un vettore sol dove $sol[i]$ è il colore assegnato al nodo i . Progettare un algoritmo che, preso un grafo G restituisce una sua 3-colorazione, **None** se il grafo non è 3-colorabile.

Possiamo risolvere il problema con un algoritmo di backtracking che va alla ricerca delle eventuali 3-colorazioni del grafo e appena ne trova una termina. Al passo ricorsivo i si sceglie un colore per il nodo i tra quelli non già utilizzati per i nodi a lui adiacenti e se esiste si procede a colorare il nodo $i + 1$. Se si raggiunge $i = n$ l'algoritmo restituisce la colorazione.

```
def col3(G, i=0, sol=[]):
    if i==len(G):
        return sol
    C=[]
    for j in G[i]:
        if j < i and sol[j] not in C:
            C.append(sol[j])
    for c in range(3):
        if c not in C:
            sol.append(c)
            if col3(G, i+1, sol):
                return sol
            sol.pop()
```

La funzione di taglio introdotta (vale a dire la scelta di non colorare un vertice intermedio con un colore uguale a quello di un vertice adiacente già colorato) non garantisce che quella colorazione porterà ad una foglia che rappresenta una colorazione. La complessità quindi non sarà necessariamente proporzionale al numero di 3-colorazioni esistenti. La complessità dell'algoritmo è $O(3^n)$: l'albero di ricorsione è un albero ternario di altezza n che ha dunque al più $\frac{3^n-1}{3-1} = \Theta(3^n)$ nodi (ricorda infatti che un albero t -ario completo, con $t > 1$, ha esattamente $\sum_{i=0}^n t^i = \frac{t^{n+1}-1}{t-1}$ nodi); ogni nodo interno richiede tempo $O(n)$ mentre al più una foglia verrà raggiunta e richiederà tempo $O(1)$. Il tempo totale è $O(3^n)$.

Dare una buona stima della complessità è piuttosto difficile proprio perché dipende fortemente dal grafo di input. Possiamo però vedere come l'algoritmo si comporta su alcuni input.

6.1.13 Il problema del ciclo Hamiltoniano

In un grafo un ciclo si dice *Hamiltoniano* se tocca tutti i nodi del grafo. Dato un grafo G vogliamo sapere se G ha o meno un ciclo Hamiltoniano. Progettare un algoritmo che preso in input il grafo G restituisce un ciclo hamiltoniano se nel grafo è presente, nulla altrimenti.

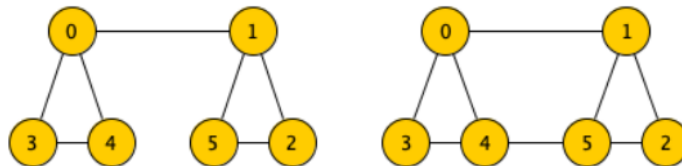


Figura 6.1: Il grafo a sinistra non ha ciclo hamiltoniano, il grafo di destra sì

Ogni nodo del grafo viene toccato una sola volta dal ciclo quindi un ciclo hamiltoniano è una permutazione P degli n vertici del grafo con la proprietà che esistono nel grafo tutti gli archi:

$$(P[0], P[1]), (P[1], P[2]), \dots, (P[n-2], P[n-1]), (P[n-1], P[0])$$

per risolvere il problema possiamo quindi andare alla ricerca di una tale permutazione. Per potare lo spazio di ricerca nell'albero delle permutazioni possiamo: imporre che la permutazione inizi con il vertice 0; imporre che i prefissi della permutazione rappresentino cammini del grafo.

```
def es(G):
    presi=[0]*len(G)
    presi[0]=1
    sol=[0]
    if es1(G, 1, sol, presi):
        return sol
```



```

def es1(G, i, sol, presi):
    if i==len(G):
        if 0 in G[sol[-1]]:
            return True
        return False
    for j in G[sol[-1]]:
        if presi[j]==0:
            sol.append(j)
            presi[j]=1
            if es1(G, i+1, sol, presi):
                return True
            presi[j]=0
            sol.pop()
    return False

```

La funzione di taglio introdotta (vale a dire la scelta di non generare solo permutazioni che cominciano con 0 e i cui prefissi siano cammini del grafo) non garantisce che i nodi generati porteranno ad una foglia che rappresenta un ciclo hamiltoniano. La complessità quindi non sarà necessariamente proporzionale al numero di cicli hamiltoniani esistenti. La complessità dell'algoritmo è $O(n!)$: l'albero di ricorsione è un albero delle permutazioni di altezza $n - 1$ che ha dunque al $O((n - 1)!)$; ogni nodo interno richiede tempo $\Theta(n)$ mentre al più $(n - 1)!$ foglie verranno raggiunte e ciascuna richiederà tempo $O(1)$. Il tempo totale è $O((n - 1)!n) = O(n!)$.

La complessità del programma dipende fortemente dal grafo di input. Ci sono grafi per i quali trova subito un ciclo Hamiltoniano (ad es. se il grafo è un ciclo) o determina che non ce ne sono (ad es. se il grafo è un albero). Ma ci sono tantissimi grafi per i quali impiega tempo esponenziale per dare una risposta. Basti pensare che se il grafo G non ha cicli Hamiltoniani, tutti i cammini dal nodo 1 saranno comunque visitati e se questi sono tanti la complessità sarà molto elevata.

6.1.14 Il problema delle n regine

Data una scacchiera $n \times n$ vogliamo calcolare quanti diversi modi ci sono di disporre sulla scacchiera n regine in modo che nessuna di esse sia in grado di catturarne un'altra in base alle regole degli scacchi. Perciò una disposizione dovrà prevedere che nessuna delle n regine abbia colonna, riga o diagonale in comune con un'altra.

Ad esempio, ecco due delle possibili disposizioni per $n = 8$:

							x
	x						
			x				
x							
						x	
				x			
		x					
					x		

			x				
	x						
						x	
		x					
					x		
							x
x							
				x			

Bisogna quindi progettare una funzione di backtracking che prende come parametro l'intero n e restituisce il numero di disposizioni lecite possibili. Non è difficile convincersi che per $n = 2$ ed $n = 3$ non esiste alcuna disposizione lecita. Per i primi valori di n la funzione deve dare le seguenti risposte:

n	disposizioni
1	1
2	0
3	0
4	2
5	10
6	4
7	40
8	92
9	352
10	724
11	2680
12	14200
13	73712
14	365596
15	2279184

Le disposizioni che vogliamo contare soddisfano 3 vincoli:

1. le n regine sono su righe distinte;
2. le n regine sono su colonne distinte;
3. le n regine sono su diagonali distinte.

I primi due vincoli fanno sì che in ciascuna delle disposizioni da contare in ogni riga della scacchiera ci sarà esattamente una regina e lo stesso varrà per ogni colonna. Questo significa che posso vedere ciascuna delle disposizioni da contare come una permutazione dei numeri da 0 a $n - 1$ dove in posizione i compare la colonna in cui è stata sistemata la regina della i -esima riga.

							x	7
	x							1
			x					3
x								0
						x		6
				x				4
		x						2
					x			5

$= [7, 1, 3, 0, 6, 4, 2, 5]$

Una disposizione che soddisfa i primi due vincoli del problema (vale a dire regine in righe e colonne distinte) può dunque essere vista come una permutazione dei primi n interi. Non tutte le permutazioni corrispondono alle disposizioni lecite che vogliamo contare. Esempio di permutazione che non corrisponde ad una disposizione lecita per il problema delle n regine (la regina sulla riga 2 e quella sulla riga 5 condividono la stessa diagonale)

7							x
1		x					
3				x			
0	x						
4					x		
6						x	
2			x				
5						x	

$[7, 1, 3, 0, 4, 6, 2, 5] =$

Per risolvere il problema possiamo modificare il programma che stampa tutte le permutazioni dei primi n interi in modo che, anziché stamparle, conti quelle che soddisfano i vincoli richiesti.

Cominciamo a scrivere il programma per il conteggio delle "soluzioni" che soddisfanno solo i primi due vincoli quando si richiede alle regine di non essere mai nelle stesse righe o stesse colonne (in pratica conta le permutazioni lunghe n restituendo $n!$), aggiungeremo poi a questa versione del programma delle funzioni di taglio in modo che vengano generate (e quindi contate) solo permutazioni corrispondenti a disposizioni lecite.

```

def es(n):
    presi=[0]*n
    return es1(n, 0, [], presi)

def es1(n, i, sol, presi):
    if i==n:
        return 1
    tot = 0
    for j in range(n):
        if presi[j]==0:
            sol.append(j)
            presi[j]=1
            tot += es1(n, i+1, sol, presi)
            sol.pop()
            presi[j]=0
    return tot

```

Bisogna ora aggiungere al codice i vincoli attinenti alle diagonali di modo che la posizione che si vuole assegnare ad una nuova regina non deve essere su una diagonale già occupata. Ogni cella della scacchiera appartiene a due diagonali: una diagonale che va da sinistra a destra (diagonale principale DP) ed una diagonale che va da destra a sinistra (diagonale secondaria DS). Ecco due proprietà che permettono di ricavare da una cella la diagonale a cui questa appartiene.

Due celle (i, j) e (i', j') sono sulla stessa diagonale principale DP se e solo se $i - j = i' - j'$, due celle (i, j) e (i', j') sono sulla stessa diagonale secondaria DS se e solo se $i + j = i' + j'$.

In un insieme DP posso tener traccia delle diagonali già occupate che vanno da sinistra a destra mentre in un insieme DS posso tener traccia delle diagonali già occupate che vanno da destra a sinistra. Per sapere se la diagonale DP della cella (i, j) è stata già occupata basta vedere se $i - j$ è in DP, analogamente, per sapere se la diagonale DS della cella (i, j) è stata già occupata basta vedere se $i + j$ è in DS.

```

def es(n):
    presi=[0]*n
    return es1(n, 0, [], presi, set(), set())

def es1(n, i, sol, presi, DP, DS):
    if i==n:
        return 1
    tot = 0
    for j in range(n):
        if presi[j]==0 and (i-j not in DP) and (i+j not in DS):
            sol.append(j)
            presi[j]=1; DP.add(i-j); DS.add(i+j)
            tot += es1(n, i+1, sol, presi, DP, DS)
            sol.pop()
            presi[j]=0; DP.remove(i-j); DS.remove(i+j)
    return tot

```